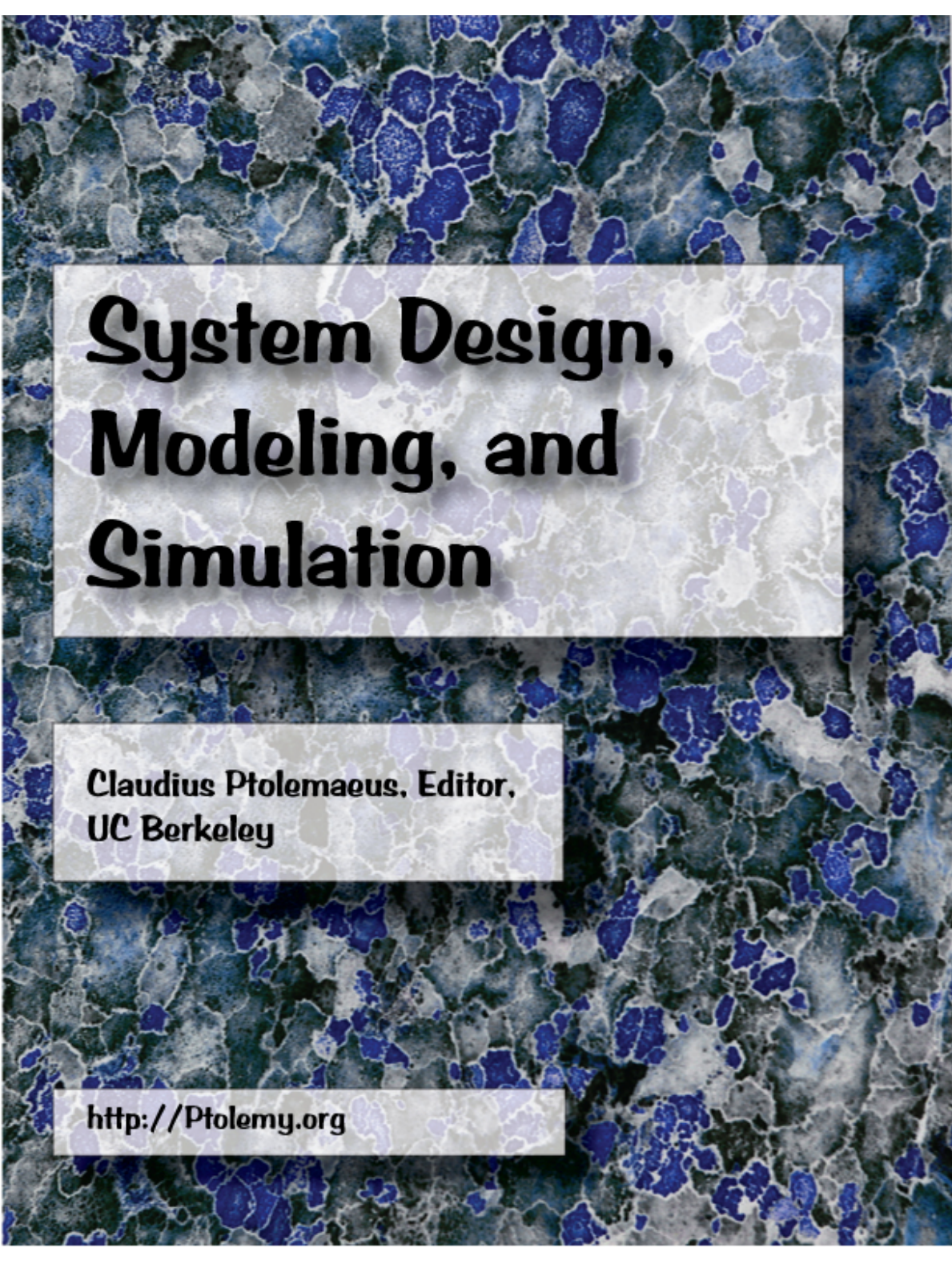




System Design, Modeling, and Simulation

**Claudius Ptolemaeus, Editor,
UC Berkeley**

<http://Ptolemy.org>

Copyright ©2011
Regents of the University of California
All rights reserved

First Edition, Version 0.01

Please cite this book as:

Claudius Ptolemaeus, Editor,
System Design, Modeling, and Simulation Using Ptolemy II, Ptolemy.org, 2011.

Contents

Preface	v
1 Heterogeneous Modeling	1
1.1 Plurality of Models	2
1.2 Heterogeneity of Models	4
1.3 Abstract Syntax – The Structure of Models	6
1.4 The Actor Abstract Semantics	10
1.5 Models of Computation	18
1.6 Conclusion	21
2 Visual Models	23
2.1 Getting Started	24
2.2 Tokens and Data Types	36
2.3 Hierarchy	40
2.4 Annotations and Parameterization	46
2.5 Navigating Larger Models	49
2.6 Classes and Inheritance	52
2.7 Higher-Order Components	64

2.8	Conclusion	70
3	Discrete Event Models	71
3.1	Time in DE Models	76
3.2	Scheduling	80
3.3	Using DE with Dataflow	94
3.4	Wireless and Sensor Network Systems	95
3.5	Summary	97
	Exercises	99
4	FSMs and Modal Models	101
4.1	Finite State Machines	103
4.2	Modal Models	121
4.3	Conclusion	152
	Exercises	153
A	Signal Display	155
A.1	Overview of Available Plotters	155
A.2	Customizing a Plot	156
	Bibliography	163
	Actor Index	171
	Index	171

Preface

FIXME: Preface goes here.

NOTE: If you are reading this document on-line and have a network connection, then you can click on figures showing Ptolemy II models to execute and experiment with those models on line.

Heterogeneous Modeling

Edward A. Lee

Contents

1.1	Plurality of Models	2
1.2	Heterogeneity of Models	4
	<i>Sidebar: About the term Actors</i>	4
1.3	Abstract Syntax – The Structure of Models	6
	<i>Sidebar: Relationship to UML, SysML, and MARTE</i>	7
1.4	The Actor Abstract Semantics	10
1.4.1	Execution Control	10
	<i>Sidebar: About Heterogeneous Models</i>	13
1.4.2	Communication	14
1.4.3	Models of Time	15
	<i>Sidebar: Time in Ptolemy II</i>	17
1.5	Models of Computation	18
	<i>Sidebar: Time in Ptolemy II (Continued)</i>	18
1.6	Conclusion	21
	<i>Sidebar: Tools Supporting Heterogeneous Models</i>	22

1.1 Plurality of Models

Occam's razor is a widely used principle in science and engineering that prefers theories and hypotheses with the fewest assumptions, postulates, or entities that are sufficient to accomplish the goal. The principle can be expressed as “entities must not be multiplied beyond necessity” (*entia non sunt multiplicanda praeter necessitatem*) or as “plurality should not be posited without necessity” (*pluralitas non est ponenda sine necessitate*) ([Encyclopedia Britannica, 2010](#)). The principle is attributed to 14th-century English logician, theologian and Franciscan friar William of Ockham.

Despite its compelling value, the principle has its limitations. Immanuel Kant, for example, felt a need to moderate the effects of Occam's razor, stating “the variety of beings should not rashly be diminished.” (*entium varietates non temere esse minuendas*) ([Smith, 1929](#)). Einstein allegedly remarked, “everything should be made as simple as possible, but not simpler” ([Shapiro, 2006](#)).

When applied to design techniques, Occam's razor biases us towards using fewer and simpler design languages and notations. However, experience indicates that redundancy and diversity can both be beneficial. For example, there is benefit to using UML class diagrams even if the information they represent is already encoded in a C++ program, making the diagrams redundant. There is also value in use-case diagrams, which express concepts that are not encoded at all in a C++ program that realizes a design. The use-case diagrams represent aspects of the design not (directly) represented in a class diagram, or even in the program itself.

The plurality of notations in UML and its derivatives runs distinctly counter to the principle in Occam's razor.¹ To take an extreme position, arguably, everything that can be expressed in UML can be expressed in natural language. In fact, historically, natural language specifications of design are the precursors of UML specifications. The syntax of natural languages (at least the Indo-European languages) is simpler and less diverse than the syntax of UML, and it is already familiar to anyone who would engage in design. So what is gained by this anti-razor?

One might postulate that UML diagrams offer more precision than natural language descriptions. This is partly true of some diagrams. Consider class diagrams, for example. These have a rather simple syntax, but are not as precise as a set of C++ header files, for

¹The **unified modeling language (UML)**, interestingly, originated in the 1990s to *reduce* the diversity of notations used to express object-oriented software architectures ([Booch et al., 1998](#)).

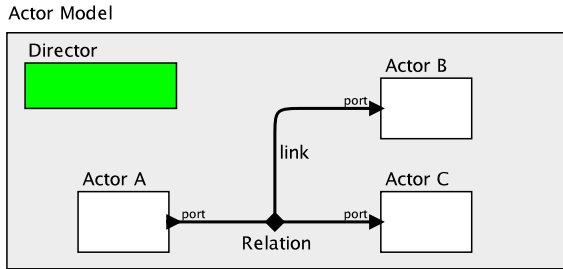


Figure 1.1: In an actor model, components called actors communicate via ports.

example. In particular, the varied interpretations of associations and the extension mechanisms make it possible to create standard-compliant class diagrams that mean almost anything. In general, UML standards emphasize syntax over semantics, and compliance with the standard is about the structure and appearance of diagrams, and not so much about their meaning. So improved precision can't possibly be the justification for the anti-razor.

Design of software systems is essentially a creative process. Engineers create artifacts that did not previously exist. Occam's razor should not be applied to creative processes. A plurality of media for expression enrich process, even when the plurality is not driven by necessity.

This book is about a class of design representations called **actor-oriented models**, or more simply, **actor models**. It explains how to realize a multiplicity of such models using Ptolemy II, and how to get the models to interoperate.

Actors are concurrent components that share data by sending messages via ports. Consider for example the model in Figure 1.1. That model shows three actors, each with one port. Actor A sends messages to actors B and C via its port. The messages traverse links. The sum total of all messages communicated via a port is called a **signal**. We will have more to say about the other features of this model.

An amazing variety of techniques have evolved that fit the general pattern of actor models. These techniques differ in how actors execute and how the communication occurs. Were we to apply Occam's razor, we would seek the single unifying actor-oriented modeling language. However, there are significant advantages to having available a plurality

of distinct actor-oriented modeling languages, together with mechanisms for composing models in these languages, each with a strong and clear semantics. To support the design of complex systems, Ptolemy II provides for heterogenous composition of such models.

The techniques described in this book have been developed in the Ptolemy Project, and have largely previously presented in (Buck et al., 1994; Eker et al., 2003; Lee et al., 2003). The key idea is to use a common *abstract syntax* (to be explained below) for a diverse set of actor-oriented models, and to hierarchically compose those models using an *abstract semantics*.

1.2 Heterogeneity of Models

Complex systems demand diversity in the modeling mechanisms. We see this diversity of models very clearly with **cyber-physical systems (CPS)**, which combine computing and networking with physical dynamics. Cyber-physical systems require model combinations that integrate the continuous dynamics of physical processes (often described using differential equations) with models of software. We also see diversity of models in applications where timed interactions between components are combined with conventional algorithmic computations,² such as in networked computer games. We even see it in

²An **algorithm** is a finite description of a sequence of steps to be taken to solve a problem. Physical processes are rarely structured as a sequence of steps. They are rather structured as continuous interactions between concurrent components.

Sidebar: About the term Actors

Our notion of actor-oriented modeling is related to the term actor as introduced in the 1970's by Hewitt to describe the concept of autonomous reasoning agents (Hewitt, 1977). The term evolved through the work of Agha and others to describe a formalized model of concurrency (Agha et al., 1997). Agha's actors each have an independent thread of control and communicate via asynchronous message passing.

The Ptolemy II version embraces a larger family of models of concurrency that are often more constrained than general message passing. Our actors are still conceptually concurrent, but unlike Agha's actors, they need not have their own thread of control. Moreover, although communication is still through some form of message passing, it need not be strictly asynchronous.

traditional software systems when we have concurrent³ interactions between algorithmic components.

One way to deal with a diversity of models is to create very flexible or underspecified modeling frameworks that can be adapted to cover models of interest. The downside of this approach is a weakening of the **semantics** of models. The semantics of a model is its meaning. A weaker semantics admits more meanings than a stronger semantics. Weaker semantics makes models less understandable, less interoperable, and less analyzable. The same model may mean different things to different people, and important aspects of the design may be missing altogether. The Ptolemy II approach is to embrace heterogeneity and to provide mechanisms for a diversity of models to interact concurrently.

Model diversity arises in various ways (Brooks et al., 2008). In **multi-view modeling**, distinct and separate models of the same system are constructed to model different aspects of a system. For example, one model may describe dynamic behavior, while another describes physical design and packaging. In **amorphous heterogeneity**, distinct modeling styles are combined in arbitrary ways within the same model without the benefit of structure. For example, some component interactions in a model may use rendezvous while others use publish-and-subscribe. In **hierarchical multimodeling**, hierarchical compositions of distinct modeling styles are combined to take advantage of the unique capabilities and expressiveness of the distinct modeling styles. A familiar example is Statecharts (Harel, 1987), which hierarchically combines **synchronous** concurrent composition with **finite state machines**.

This book focuses on a disciplined form of hierarchical multimodeling studied in the Ptolemy project and implemented in the Ptolemy II software environment. Using hierarchy, one can effectively divide a complex model into a tree of nested submodels, which are at each level composed to form a network of interacting actors. Our approach constrains each of these networks to be locally homogeneous, using a common execution and communication semantics called a **model of computation (MoC)**. The key is to use constrained MoCs with strong semantic properties as much as possible, and to allow hi-

³**Concurrency**, from the Latin verb *concurrere* meaning “run together,” is often taken in computer science to mean the arbitrary interleaving of two or more sequences of steps. However, this is a rather specialized interpretation of a basic concept. In this book, we take concurrency to mean simultaneous operation, with no implication of either interleaving nor sequences of steps. In particular, two continuous processes can operate concurrently without being directly representable as sequences of steps. Consider for example a resistive heating element immersed in a vat of water. Increasing the current through the heating element will cause the temperature of the water to rise. The electrical flow is one continuous process, as is the temperature of the water, and these processes are interacting. But neither process is reasonably representable as a sequence of steps, nor is the overall process an interleaving of such steps.

erarchical composition of distinct MoCs to overcome the constraints that are the price for strong semantics.

1.3 Abstract Syntax – The Structure of Models

The Ptolemy approach uses a common **abstract syntax** across diverse models. An abstract syntax defines the structure of models and may be described by a meta model. A **meta model** is a model of a modeling language or notation. A meta model for the Ptolemy II abstract syntax is shown in Figure 1.2 as a **UML class diagram**, which shows the relationships between object-oriented classes. Instances of these classes comprise a **Ptolemy II model**.

Every component in a Ptolemy II model is an instance of the `NamedObj` class, which has a name and a container (the container is null the **top-level** object in a Ptolemy II model). There are four specific kinds of `NamedObj`: `Attribute`, `Entity`, `Port`, and `Relation`. A `NamedObj` contains a (possibly empty) collection of instances of `Attribute`. An `Entity` also contains a (possibly empty) collection of instances of `Port`. Ports are associated with instances of `Relation`, which mediate connections between ports. A `CompositeEntity` is an `Entity` that contains instances of `Entity` and `Relation`. An `AtomicActor` is an executable `Entity`, whereas a `CompositeActor` is executable `CompositeEntity`. In Ptolemy II, an **atomic entity** (or **atomic actor**) (from the ancient Greek **atomos**, meaning indivisible), is simply one that cannot contain other entities (or actors), whereas a **composite entity** (or **composite actor**) is one that can. Finally, a `Director` is an executable `Attribute`.

The description we have just given of a Ptolemy II model almost entirely about its abstract syntax. The structure of a Ptolemy II model (what contains what, what is associated with what) conforms with the abstract syntax defined in Figure 1.2. The fact that some objects in a model are executable, however, is not part of the abstract syntax. That is part of the semantics of the model. Whereas an abstract syntax constrains the structure of a model, a **concrete syntax** gives a way to describe a model in text or pictures.

An example of a Ptolemy II model is shown in Figure 1.3 using the concrete **visual syntax** of *Vergil*, a visual editor for Ptolemy II models. The figure actually shows three distinct submodels and their hierarchical relationships. In that figure, the top-level of the hierarchy is labeled “Model: CompositeActor,” which means that its name is `Model` and that it is an

Sidebar: Relationship to UML, SysML, and MARTE

The **Object Management Group (OMG)** has standardized a number of notations that relate strongly to the [visual syntax](#) of Ptolemy II models. The actor-oriented models we consider here are syntactically related to **composite structure diagrams** of UML 2 ([Bock, 2006](#); [Booch et al., 1998](#)), or more directly its derivative **SysML** ([Object Management Group \(OMG\), 2008a](#)). The **internal block diagram** notation of SysML, which is based on the UML 2 composite structure diagrams, particularly with the use of flow ports, is closely related to actor models. In SysML, the actors are called “blocks,” (the term “actor” is used in UML for another purpose).

SysML, however, emphasizes how model diagrams are rendered (their visual syntax), and leaves many details open about what the diagrams mean and how the models operate (their semantics). For example, although the standard states that “flow ports are intended to be used for asynchronous, broadcast, or send-and-forget interactions” ([Object Management Group \(OMG\), 2008a](#)), there is nothing like a **MoC** given in SysML. Different SysML tools may give different behavior to flow ports and still be compliant with the standard. Therefore, the same SysML diagrams may in fact represent many different designs, and the behavior of the model may depend on the tools used to interpret the model. The emphasis of SysML is on standardizing the notation.

In contrast, the emphasis in Ptolemy II is on what the structure of models is (the *abstract* syntax), rather than on how it is rendered (visually or otherwise), and on the semantics of models. The visual notation is incidental, and in fact is not the only representation for a Ptolemy II model. Moreover, Ptolemy II **directors** give a model a very specific meaning. This concrete meaning helps to ensure that a model means the same thing to different observers, it enables interoperation of heterogeneous models.

MARTE (modeling and analysis of real-time and embedded systems) puts more emphasis than SysML on the behavior of models ([Object Management Group \(OMG\), 2008b](#)), but specifically avoids “constraining” the execution semantics. This makes the standard flexible, enabling representation of many real-time techniques that are prevalent today in practice. This flexibility accounts for some of its success, since existing practices can be captured. But the price paid is imprecision.

In contrast, the emphasis in Ptolemy II is less on capturing existing design practices, and more on providing precise and well-defined models of system behavior. The goal is to improve the practice of design of real-time and embedded systems.

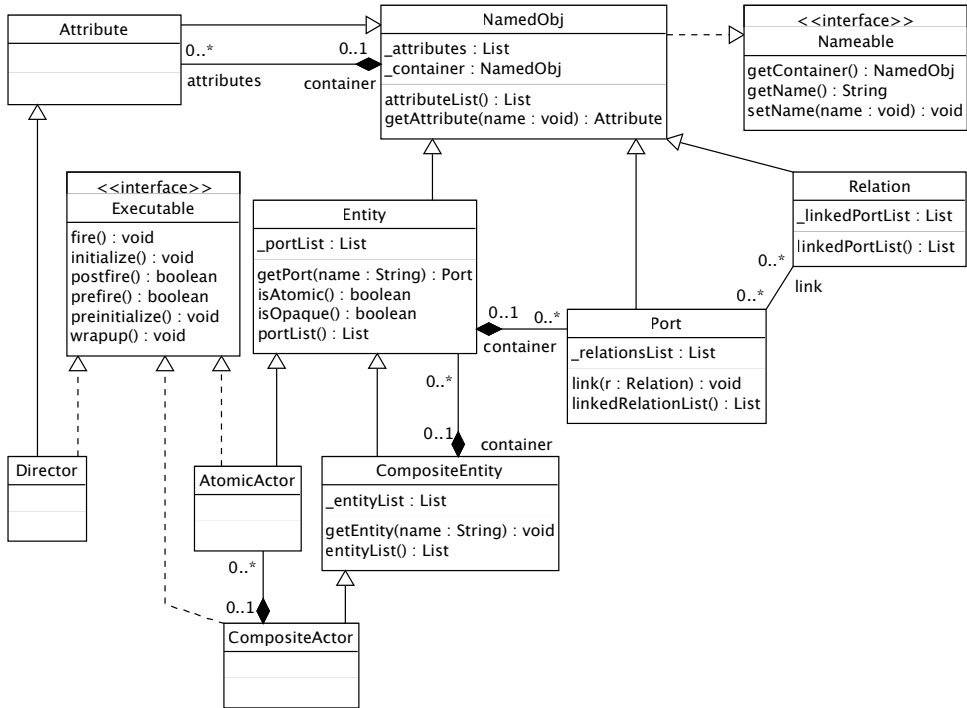


Figure 1.2: A meta model for Ptolemy II. This is a UML class diagram. The boxes represent classes with the class name, key members, and key methods shown. The lines with triangular arrowheads represent inheritance. The lines with diamond ends represent containment.

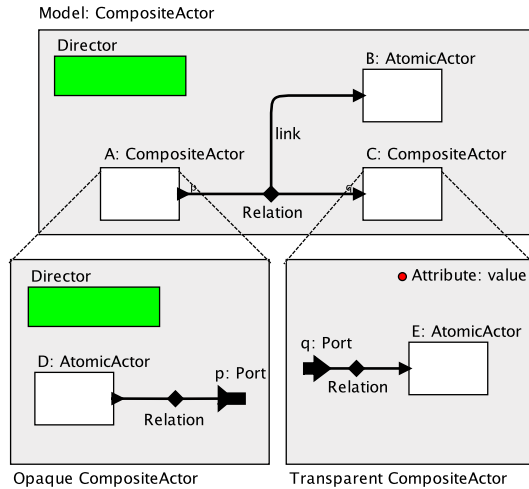


Figure 1.3: A hierarchical model in Ptolemy II consisting of a top-level composite actor and two submodels, each of which is also a composite actor. Each of the three submodels here is rendered in Vergil, the visual editor for Ptolemy II models.

instance of `CompositeActor`. `Model` contains an instance of `Director`, three actors, and one **relation**. Actors `A` and `C` are composite, whereas actor `B` is atomic. The ports of the three actors are linked to the relation.

In Figure 1.3, actor `A` contains one `Port` named `p`, one `Director`, one actor `D`, and one `Relation`. The port of `D` is linked to the port `p` via the relation.⁴ Actor `C` is similar except that it does not contain an instance of `Director`, and it contains an `Attribute` (indicated visually with a bullet). In Ptolemy II, some attributes have values given in an [expression language](#).

A composite actor that contains a director is said to be **opaque**, and one that does not is said to be **transparent**. As we will explain below, opaque composite actors are key to hierarchical heterogeneity. From outside, an opaque composite actor looks just like an atomic actor. Its inside structure is not seen by its container. It is a black box. In contrast,

⁴Vergil will typically not show this relation as a black diamond, and instead only show a line. But the relation is there in the model structure nonetheless.

a transparent composite actor is simply a syntactic grouping with no semantic meaning. The inside structure is fully visible from outside. It is a white box.

The block diagram in Figure 1.3 uses one of many possible concrete syntaxes for the same model. The model can also be defined in Java syntax, as shown in Figure 1.4, or in an XML schema known as **MoML** (Lee and Neuendorffer, 2000), as shown in Figure 1.5. All three syntaxes describe model structure (which conforms with the abstract syntax). We will next give the structure some meaning (which conforms with an abstract semantics).

1.4 The Actor Abstract Semantics

In an actor-oriented model, components called **defnnactoractors** execute concurrently, receiving data from other actors (or from themselves in feedback systems) at their input ports and sending data to other actors (or themselves) via their output ports. What exactly it means to “execute concurrently” and to send or receive data depends on the specific model of computation. The MoC has a **concrete semantics**. To permit hierarchical composition of distinct MoCs, the concrete semantics conforms to an **abstract semantics**. We describe this abstract semantics informally. A formal framework can be found in Lee and Sangiovanni-Vincentelli (1998).

The abstract semantics has three distinct aspects, execution control, communication, and models of time. We discuss these in order.

1.4.1 Execution Control

The semantics of execution and communication is governed by a **director** in a composite actor, which implements a particular MoC. As shown in Figure 1.2, a `Director` is an `Attribute` that implements the `Executable` interface. It is rendered in Vergil as a green rectangle, as shown in Figure 1.3. Inserting a `Director` into a composite actor makes the composite actor executable. Atomic actors also implement the `Executable` interface.

An execution of an executable actor has the following phases: **setup**, **iterate**, and **wrapup**, where each phase has more fine-grained phases. The setup phase is divided into **preinitialize** and **initialize** phases, implemented by methods of the `Executable` interface. In the preinitialize subphase, an actor performs any actions that may influence static analysis (including scheduling, type inference and checking, code generation, etc.). For example, a composite actor may alter its own internal structure, creating internal actors for exam-

```

1  import ptolemy.actor.AtomicActor;
2  import ptolemy.actor.CompositeActor;
3  import ptolemy.actor.Director;
4  import ptolemy.actor.IOPort;
5  import ptolemy.actor.IORelation;
6  import ptolemy.kernel.Relation;
7  import ptolemy.kernel.util.IllegalActionException;
8  import ptolemy.kernel.util.NameDuplicationException;
9
10 public class Model extends CompositeActor {
11     public Model()
12         throws IllegalActionException, NameDuplicationException {
13         super();
14         // Construct top level.
15         new Director(this, "Director");
16         CompositeActor A = new CompositeActor(this, "A");
17         IOPort p = new IOPort(A, "p");
18         AtomicActor B = new AtomicActor(this, "B");
19         IOPort r = new IOPort(B, "r");
20         CompositeActor C = new CompositeActor(this, "C");
21         IOPort q = new IOPort(C, "q");
22         Relation relation = connect(p, q);
23         r.link(relation);
24
25         // Populate composite actor A.
26         new Director(A, "Director");
27         AtomicActor D = new AtomicActor(A, "D");
28         IOPort D_p = new IOPort(D, "p");
29         Relation D_r = new IORelation(A, "r");
30         D_p.link(D_r);
31         p.link(D_r);
32
33         // Populate composite actor C.
34         AtomicActor E = new AtomicActor(C, "E");
35         IOPort E_p = new IOPort(E, "p");
36         Relation E_r = new IORelation(C, "r");
37         E_p.link(E_r);
38         q.link(E_r);
39     }
40 }

```

Figure 1.4: The model of Figure 1.3 given in the concrete syntax of Java.


```

1  <?xml version="1.0" standalone="no"?>
2  <!DOCTYPE entity PUBLIC "-//UC Berkeley//DTD MoML 1//EN"
3    "http://ptolemy.eecs.berkeley.edu/xml/dtd/MoML_1.dtd">
4  <entity name="Model" class="ptolemy.actor.CompositeActor">
5    <property name="Director" class="ptolemy.actor.Director"/>
6    <entity name="A" class="ptolemy.actor.CompositeActor">
7      <property name="Director" class="ptolemy.actor.Director"/>
8      <port name="p" class="ptolemy.actor.IOPort"/>
9      <entity name="D" class="ptolemy.actor.AtomicActor">
10        <port name="p" class="ptolemy.actor.IOPort"/>
11      </entity>
12      <relation name="r" class="ptolemy.actor.IORelation"/>
13      <link port="p" relation="r"/>
14      <link port="D.p" relation="r"/>
15    </entity>
16    <entity name="B" class="ptolemy.actor.AtomicActor">
17      <port name="r" class="ptolemy.actor.IOPort"/>
18    </entity>
19    <entity name="C" class="ptolemy.actor.CompositeActor">
20      <property name="Attribute"
21        class="ptolemy.kernel.util.Attribute"/>
22      <port name="q" class="ptolemy.actor.IOPort"/>
23      <entity name="E" class="ptolemy.actor.AtomicActor">
24        <port name="p" class="ptolemy.actor.IOPort"/>
25      </entity>
26      <relation name="r" class="ptolemy.actor.IORelation"/>
27      <link port="q" relation="r"/>
28      <link port="E.p" relation="r"/>
29    </entity>
30    <relation name="r" class="ptolemy.actor.IORelation"/>
31    <link port="A.p" relation="r"/>
32    <link port="B.r" relation="r"/>
33    <link port="C.q" relation="r"/>
34  </entity>

```

Figure 1.5: The model of Figure 1.3 given in the concrete syntax of MoML.

ple, during this phase. The initialization phase initializes parameters, resets local state, and sends initial messages on output ports, if any. Typically, the preinitialization of an actor is performed exactly once during the actor’s lifetime in a model execution, and before any other execution of the actor. The initialization of an actor is performed once after the preinitialization and type resolution. It may be performed again if the semantics requires that the actor to be re-initialized, for example, in the [hybrid system](#) formalism ([Lee and Zheng, 2005](#)).

Actors perform atomic executions (called **iterations**) in the iterate phase. An iteration is a (typically finite) computation that leads the actor to a quiescent state. The MoC of a composite actor determines how the iteration of one actor is related to the iterations of other actors in the same composite (whether it is concurrent or interleaved, whether and when it is scheduled, etc.).

In order to coordinate the iterations among actors, an iteration is further broken down into three subphases called **prefire**, **fire**, and **postfire**. Prefire (optionally) tests the preconditions for the actor to execute, such as the presence of sufficient inputs to complete the iteration. The computation of the actor is typically performed during the fire phase, where

Sidebar: About Heterogeneous Models

Some authors use the term **multi-paradigm modeling** for the mixing of models of computation ([Mosterman and Vangheluwe, 2004](#)). Ptolemy II focuses on techniques that combine [actors](#) with multi-paradigm modeling. An early systematic approach to such mixed models was realized in Ptolemy Classic ([Buck et al., 1994](#)). Ptolemy II is a successor to Ptolemy Classic ([Eker et al., 2003](#)). Influenced by the Ptolemy approach, SystemC is capable of realizing multiple MoCs ([Patel and Shukla, 2004](#); [Herrera and Villar, 2006](#)). So are ModHel’X [Hardebolle and Boulanger \(2007\)](#) and ForSyDe [Jantsch \(2003\)](#); [Sander and Jantsch \(2004\)](#).

Another approach supports mixing concurrency and communication mechanisms without the structural constraints of hierarchy ([Goessler and Sangiovanni-Vincentelli, 2002](#); [Basu et al., 2006](#)). A number of other researchers have tackled the problem of heterogeneity in creative ways [Burch et al. \(2001\)](#); [Feredj et al. \(2009\)](#).

For each MoC, a Ptolemy model contains a director, which annotates the model to assert the MoC being used, and provides either a code generator or an interpreter for the MoC (or both). An interesting alternative is given by “42” ([Maraninchi and Bhounhadiba, 2007](#)), which integrates with the model a specification of a customized MoC.

it will read input data from ports, process data, and produce output data on output ports. An actor may have persistent state that evolves during execution. The postfire phase is used to update that state in response to any inputs.

Note that despite the fact that computation occurs during the fire phase, the state of the actor is not updated until postfire. This supports fixed-point iteration in some MoCs, such as [synchronous reactive](#) models and [continuous time](#) differential equations. Directors implementing an MoC with a fixed-point semantics compute the fixed point of actor outputs while keeping the state of each actor constant. The state of an actor can only be updated after the fixed point has been reached. This requires the firing of each actor several times before the actor is postfired. Such a strategy helps to ensure that the MoC is determinate.

This strategy is only followed by actors conforming to the strictest form of the actor abstract semantics. [Goderis et al. \(2009\)](#) classify actor-oriented MoCs into three categories of abstract semantics call **strict**, **loose**, and **loosest**. In the strict actor semantics, prefire, fire, and postfire are all finite computations where only postfire makes any changes to the state of the actor. In the loose actor semantics, changes to the state may be made in the fire phase. In the loosest actor semantics, the fire phase may not be finite. It could represent a non-terminating computation.

An actor that conforms with the strict actor semantics can be used with any actor-oriented director. Such an actor is said to be **domain polymorphic**. An actor that conforms only with the loose actor semantics can be used with fewer directors. An actor that conforms only with the loosest actor semantics can be used with still fewer. Some actors are designed to work only with a single specific type of director. These actors are not domain polymorphic at all.

A director also implements the same phases of execution. When put within a composite actor, making it [opaque](#), the director endows that composite actor with an executable semantics. If the director conforms to the strict actor semantics, then an opaque composite actor with that director is domain polymorphic. Such directors support the most flexible form of **hierarchical heterogeneity** in Ptolemy II because a plurality of directors with distinct MoCs may be combined hierarchically within a single model.

1.4.2 Communication

A director determines how actors communicate. It does this by creating an object called a **receiver** and placing that object in input ports. There is one receiver for each commu-

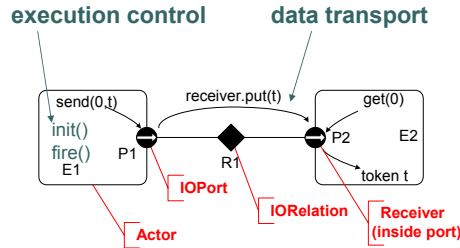


Figure 1.6: Communication mechanism in Ptolemy II.

nication channel. Receivers can implement FIFO queues, mailboxes, proxies for a global queue, rendezvous, etc. They do this by conforming to the meta model shown in Figure 1.7. When an actor sends data t to an output port p_1 , the mechanics of the send are delegated to the receiver in each input port p_2 that p_1 is connected to, as shown in Figure 1.6. When an actor gets data from an input port, that action is also delegated to the receiver. Thus, what it means to send data to an output port of get data from an input port is determined by the director, which controls what it means by providing an appropriate receiver.

1.4.3 Models of Time

Some models of computation have a notion of time, meaning that they assume that when they are fired, their director provides a value representing the time of that firing. A part of the actor abstract semantics provides a mechanism for accessing and controlling the advancement of time.

When an actor fires, it can ask its director for the current time. The time returned by the director is the call the **current model time**. If it asks for the current model time only in its *postfire* phase of execution, then it is assured that time is nondecreasing. If it does it in the *fire* phase, then there is no such assurance, because some directors, notably the *Continuous* director, speculatively advance time while converging to a *fixed point* (see Chapter ??). In an invocation of `fire()`, the current model time may actually be *less* than what it was in a *prior* invocation of `fire()`.

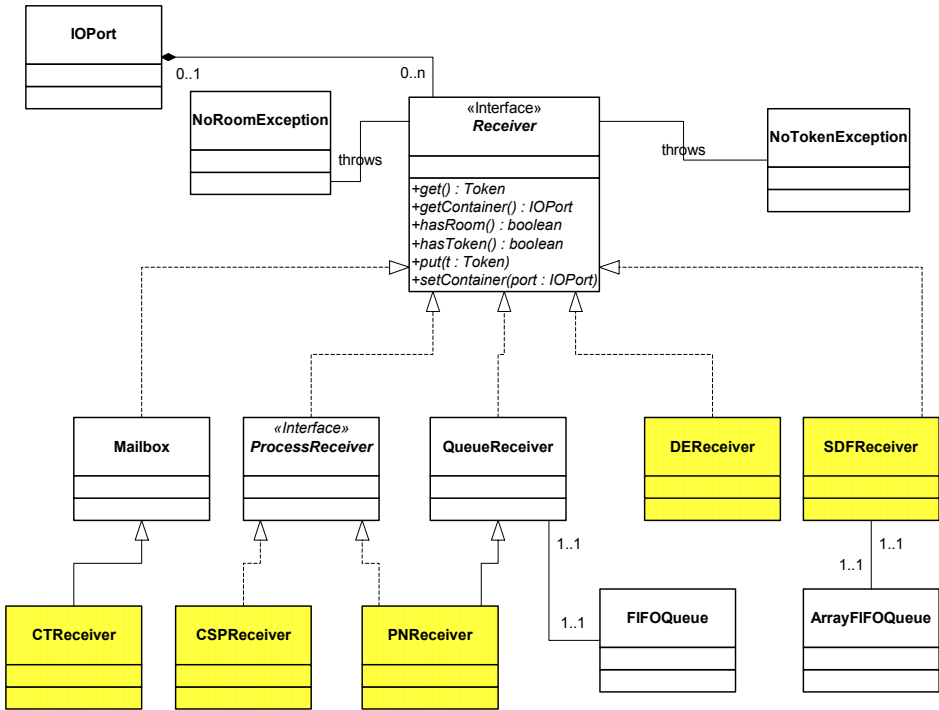


Figure 1.7: Meta model for communication in Ptolemy II.

An actor can request that it be fired at some future time by invoking a `fireAt()` method of its director. The director is responsible for ensuring that the actor is fired at the requested future time. Note, however, that the actor cannot assume that the *next* firing will be at that future time. There may be an arbitrary number of intervening firings before the one at that future time. Moreover, if the execution of the model ends before time advances to the requested future time, then the actor will not be fired at that future time.

The model hierarchy is central to the management of time. Typically, only the top-level director advances time. Other directors in a model obtain current model time from their enclosing director. If the top-level director does not implement a timed model of computation, then time does not advance.

Timed models of computation may be interleaved in the hierarchy with untimed models of computation (see below). There are certain combinations that do not make sense, however. For example, if the top-level director never advances time, and an actor requests a firing at a future time, then the request cannot be honored. Such combinations result in an exception at run time that indicates that a model contains an actor that is incompatible with an enclosing director.

Time can also advance non-uniformly in a model. In particular, [modal models](#) support a notion of local time where advancement of time can be temporarily suspended ([Lee and Tripakis, 2010](#)). Within the submodel there is a monotonically non-decreasing gap between local time and environment time. This mechanism is used to model temporary suspension of a submodel. For details, see Chapter 4.

Sidebar: Time in Ptolemy II

Ptolemy II uses a model of time known as **superdense time** ([Manna and Pnueli, 1993](#); [Maler et al., 1992](#); [Lee and Zheng, 2005](#); [Cataldo et al., 2006](#)). A superdense time value is a pair (t, n) , called a **time stamp**, where t is **model time** and n is a **microstep** (also called an **index**). The model stamp represents the time at which some event occurs, and the microstep represents the sequencing of events that occur at the same time. Two time stamps (t, n_1) and (t, n_2) can be interpreted as being **simultaneous** (in a weak sense) even if $n_1 \neq n_2$. A stronger notion of **simultaneity** would require the time stamps to be equal (both in model time and microstep). If an event with time stamp (t, n_1) causes an event with time stamp (t, n_2) , then we require that $n_1 < n_2$. Thus we can represent causally-related, but (weakly) simultaneous events. If an event with time stamp (t, n_1) occurs in the same [signal](#) as a distinct event with time stamp (t, n_2) , then we require that $n_1 \neq n_2$. A signal may therefore include weakly simultaneous, but distinct, events, but not strongly simultaneous events. This is useful, for example, for representing discontinuities in continuous-time signals, or sequences of zero-time events in discrete signals.

In Ptolemy II, the microstep is represented as an `int`, and the model time $t = mr$ is represented as an arbitrarily large integer multiple m of a time resolution r . The multiple m is realized as a Java `BigInteger` (an arbitrarily large integer). The **time resolution** r , a `double`, is a parameter shared by all the directors in a model. A model, therefore, has the same time resolution throughout its hierarchy.

Continued on page 18.

1.5 Models of Computation

In Ptolemy II an implementation of a model of computation is called a **domain**.⁵ A domain defines the communication semantics and the execution of actors within the domain.

⁵The term “domain” comes from a fanciful notion in astrophysics, that there are regions of the universe with different sets of laws of physics. A model of computation represents the “laws of physics” of the submodel governed by it.

Sidebar: Time in Ptolemy II (Continued)

To understand the role of the microstep, consider for example Newton’s cradle, a toy with five steel balls suspended by strings. If you lift one ball and release it, it strikes the second ball, which does not move. Instead, the fifth ball reacts by rising. Consider the momentum of the second ball as a function of time. We might convey this momentum in a signal in an actor model. The second ball does not move, so its momentum must be everywhere zero. But the momentum of the first ball is somehow transferred to the fifth ball, passing through the second ball. So the momentum cannot be always zero.

Let $\mathbb{R}$ represent the real numbers. Let $m: \mathbb{R} \rightarrow \mathbb{R}$ be a function that represents the momentum of this second ball, and let τ be the time of the collision. Then

$$m(t) = \begin{cases} M & \text{if } t = \tau \\ 0 & \text{otherwise} \end{cases}$$

for all $t \in \mathbb{R}$. Infinitesimally before and after time τ , the momentum of the ball is zero, but at time τ , it is not zero.

Such infinitesimals are elegantly represented using superdense time. Specifically, the momentum of the second ball can be unambiguously represented by a sequence where $m(\tau, 0) = 0$, $m(\tau, 1) = M$, and $m(\tau, 2) = 0$. That is, instead of a function of form $m: \mathbb{R} \rightarrow \mathbb{R}$, we represent the momentum as a function of the form $m: \mathbb{R} \times \mathbb{N} \rightarrow \mathbb{R}$, where $\mathbb{N}$ represents the natural numbers. At time τ , the signal m has three distinct values, ordered by their microsteps.

Superdense time is ordered lexicographically (like a dictionary), which means that $(t_1, n_1) < (t_2, n_2)$ if either $t_1 < t_2$, or $t_1 = t_2$ and $n_1 < n_2$. Thus, an event is considered to occur before another if its model time is less or, if the model times are the same, if its microstep is lower. Time stamps are a particular realization of **tags** in the tagged-signal model of [Lee and Sangiovanni-Vincentelli \(1998\)](#).

We present here some domains that we have realized in Ptolemy II. This is far from a complete list. The intent is to show the diversity of the models of computation under study. These and others are described in subsequent chapters in much more detail.

A theme throughout these domains is to ensure **determinism** unless the model explicitly includes **nondeterminism**. A domain is said to be **determinate** if the **signals** sent between actors are exactly the same for all allowed executions of a model. That is, the values of tokens, their order, and their **time stamps** (if they have time stamps) do not depend on arbitrary scheduling decisions, despite the concurrency in the model. Ensuring determinism is far from trivial in concurrent **MoCs**. And providing reasonable **nondeterminate** mechanisms is also challenging. When these mechanisms are provided, the goal is that the nondeterminism be explicitly specified by the builder of the model; it should not appear accidentally, nor should it surprise the user.

Process networks. In the **process network (PN)** domain, actors represent concurrent processes that communicate by (conceptually infinite capacity) FIFO queues ([Lee and Parks, 1995](#)). Receivers in this model implement these FIFO queues. Writing to the queues always succeeds, while reading from an empty queue blocks the reader process. The simple blocking-read, nonblocking-write rule ensures the determinacy of the model ([Kahn and MacQueen, 1977](#)). This domain is untimed. We have extended the model with specific support for certain forms of nondeterminism. Each actor executes in its own Java thread, so on multi-core machines they can execute in parallel.

Dataflow. Ptolemy II includes several **dataflow** domains, all restricted special cases of PN ([Lee and Matsikoudis, 2009](#)). The **synchronous dataflow (SDF)** domain ([Lee and Messerschmitt, 1987](#)) is a particularly restricted special case with strong, decidable, formal properties. When an actor is executed in this model, it consumes a fixed number of tokens from each input port, and produces a fixed number of tokens to each output port. A valuable property of SDF models is that deadlock and boundedness can be statically analyzed, and schedules (including parallel schedules) can be statically computed. Receivers in this domain represent FIFO queues with fixed finite capacity, and the execution order of components is statically scheduled. SDF can be timed or untimed. The **dynamic dataflow (DDF)** domain is more flexible, but computes schedules on the fly.

Discrete events. In the **discrete event (DE)** domain, actors communicate through events placed on a time line. Each event has a value and a time stamp. Actors process events in

chronological order. The output events produced by an actor are required to be no earlier in time than the input events that were consumed. In other words, actors in DE are causal.

The execution of this model uses a global event queue. When an actor generates an output event, the event is placed in the queue, and sorted according to its time stamp. Receivers in this domain are proxies for the global event queue. During each iteration of a DE model, the events with the smallest time stamp are removed from the global event queue, and their destination actor is fired. The DE domain supports simultaneous events. At each time where at least one actor fires, the director computes a fixed point (Lee and Zheng, 2007). The semantics of DE is given in (Lee, 1999).

Finite-state machines. The *finite-state machine* (FSM) domain is the only one of the domains discussed here that is not actor oriented (Lee and Tripakis, 2010). The entities in this domain represent *states*, and the relations represent transitions between states. Attributes are used to represent *guards*, which determine when a transition is taken from one state to another. Each state has one port for incoming transitions and one for outgoing transitions. Thus, although FSM models can be represented with the same *abstract syntax* as actor models, the ports are not used for communication but rather for sequential control.

The FSM domain interoperates with all other domains hierarchically, providing a powerful construct called a *modal model* (Lee, 2009). In a modal model, each state of an FSM represents a mode of execution. The state can have one or more *refinements*, which are submodels with a director that are active when the FSM is in that state. When a submodel is not active, its local time does not advance.

Continuous time. The *Continuous* domain (Lee and Zheng, 2005) models ordinary differential equations (ODEs), extended to allow the handling of discrete events. Special actors that represent *integrators* are connected in feedback loops in order to represent the ODEs. Each connection in this domain represents a continuous-time function, and the components denote the relations between these functions.

Each receiver in the Continuous domain is a buffer with size one. It contains the value of the continuous function of the corresponding connection at a specific time instant. The execution of a Continuous model involves the computation of a numerical solution to the ODEs. In an iteration of a Continuous model, time is advanced by a certain amount determined by the solver. At each instant, the director computes a fixed point for all signal values using the principles of synchronous/reactive models (see below) (Lee and

Zheng, 2007). To advance time, the director chooses a time step with the help of a solver and speculatively executes actors through this time step. If the time step is sufficiently small (key events such as level crossings, mode changes, or requested firing times are not skipped over), then the director commits the time increment and proceeds to the postfire phase.

The Continuous director conforms to the [strict actor semantics](#), and hence interoperates with all other timed Ptolemy II domains. Combining it with FSMs yields a particular form of modal model known as a [hybrid system](#) (Lee and Zheng, 2005; Lee, 2009). Combinations with discrete-event and synchronous/reactive are also particularly useful (Lee and Zheng, 2007).

Rendezvous. In the [Rendezvous](#) domain, actors represent concurrent processes that communicate by atomic instantaneous rendezvous. Receivers in this domain implement the rendezvous points. An attempt to put a token into a receiver will not complete until a corresponding attempt is made to get a token from the same receiver, and vice versa. As a consequence, the process that first reaches a rendezvous point will stall until the other process also reaches the same rendezvous point (Hoare, 1978). Like PN, this domain supports explicit nondeterminism.

1.6 Conclusion

Ptolemy II focuses on actor-oriented modeling of complex systems, providing a disciplined approach to heterogeneity. The central notion in hierarchical model decomposition is that of a [domain](#), which implements a particular model of computation. Technically, a domain serves to separate the flow of control and data between components from the actual functionality of individual components. Besides facilitating hierarchical models, this factoring potentially also dramatically increases the reusability of components and models. The rest of this book shows how to build Ptolemy II models and how to leverage the properties of each of the models of computation.

Sidebar: Tools Supporting Heterogeneous Models

Several widely used tools provide fixed combinations of a few MoCs. Commercial tools include Simulink/StateFlow (from The MathWorks), which combines continuous and discrete-time actor models with finite-state machines, and LabVIEW (from National Instruments), which combines dataflow actor models with finite-state machines and with a time-driven MoC. Statemate (Harel et al., 1990) and SCADE (Berry, 2003) combine finite-state machines with a synchronous/reactive formalism (Benveniste and Berry, 1991). Giotto (Henzinger et al., 2001) and TDL (Pree and Templ, 2006) combine FSMs with a time-driven MoC. Several hybrid systems modeling and simulation tools combine continuous-time dynamical systems with FSMs (Carloni et al., 2006).

The Y-chart approach supports heterogeneous modeling and is popular for hardware-software codesign (Kienhuis et al., 2001). This approach separates modeling of the hardware architecture that will realize a system from modeling of application behavior (a form of multi-view modeling), and provides mechanisms for bringing these disparate models together. These mechanisms support explorations of the design space that can trade off hardware cost and complexity with software design. Metropolis is a particularly elegant tool realizing this approach (Goessler and Sangiovanni-Vincentelli, 2002). It introduces a notion called a “quantity manager” that mediates interactions between functionality and the resources required to implement the functionality.

Modelica (Fritzson, 2003) also has actor-like semantics in that components are concurrent and communicate via ports, but the ports are neither inputs nor outputs. Instead, the connections between ports declare equation constraints on variables. This approach has significant advantages, particularly for specifying physical models based on differential-algebraic equations (DAEs). However, the approach also appears to be harder to combine heterogeneously with other MoCs.

Visual Models

Christopher Brooks, Edward A. Lee, Steve Neuendorffer, and Hideo John Reekie

Contents

2.1	Getting Started	24
2.1.1	Executing a Pre-Built Model: A Signal Processing Example	24
2.1.2	Creating a New Model	28
2.1.3	Running the Model	31
2.1.4	Making Connections	33
2.2	Tokens and Data Types	36
2.3	Hierarchy	40
2.3.1	Creating a Composite Actor	40
2.3.2	Adding Ports to a Composite Actor	41
2.3.3	Setting the Types of Ports	44
2.4	Annotations and Parameterization	46
2.4.1	Parameters in Hierarchical Models	46
2.4.2	Decorative Elements	49
2.4.3	Creating Custom Icons	49
2.5	Navigating Larger Models	49
2.6	Classes and Inheritance	52
2.6.1	Creating and Using Actor-Oriented Classes	52
2.6.2	Overriding Parameter Values in Instances	57
2.6.3	Subclassing and Inheritance	57
2.6.4	Sharing Classes Across Models	59

2.7	Higher-Order Components	64
2.7.1	MultiInstance Composite	64
2.7.2	IterateOverArray	66
2.7.3	Mobile Code	67
2.7.4	Lifecycle Management Actors	68
2.8	Conclusion	70

There are many ways to use Ptolemy II. It can be used as a framework for assembling software components, as a modeling and simulation tool, as a block-diagram editor, as a system-level rapid prototyping application, as a toolkit supporting research in component-based design, or as a toolkit for building Java applications. This chapter introduces its use as a modeling and simulation tool.

In this chapter, we describe how to graphically construct models using **Vergil**, a graphical user interface (GUI) for Ptolemy II. Figure 2.1 shows a simple Ptolemy II model in Vergil, showing the graph editor, one of several editors available in Vergil. Keep in mind as you read this document that graphical entry of models is only one of several possible entry mechanisms available in Ptolemy II. For example, you can define models in Java (as shown in Figure 1.4), or in XML (as shown in Figure 1.5).

2.1 Getting Started

For the remainder of this chapter, we assume that you have installed a version of Ptolemy II¹, and that you can start Vergil so that you see the initial welcome window, which looks like the one in Figure 2.2. Feel free to explore the links in this window. The “Tour of Ptolemy II” link takes you to the page shown in Figure 2.3. Alternatively, if you are reading this book on a computer screen, you can click on (most) figures to follow a hyperlink that allows you to browse (and even modify and execute) the models.

2.1.1 Executing a Pre-Built Model: A Signal Processing Example

The first example (Spectrum) listed under “Basic Modeling Capabilities” on the tour page is the model shown in Figure 2.1. It creates a sinusoidal signal, multiplies it by a sinusoidal carrier, adds noise, and then estimates the power spectrum. You can execute this

¹See <http://ptolemy.org/ptolemyII/ptIILatest> for the latest version.

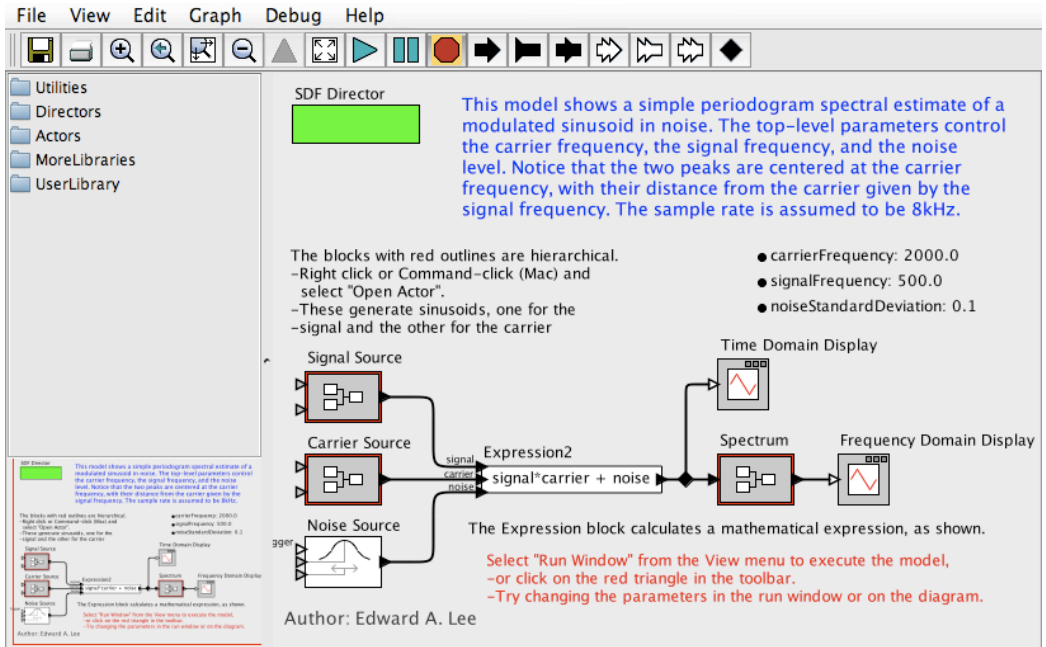


Figure 2.1: Example of a Vergil window.

model in either of two ways. First, you can on the run button in the toolbar, which is indicated by a blue triangle pointing to the right. For this model, this will cause two signal plots to be displayed in their own windows.

The second way to run a model is to select Run Window in the View menu, and then click on Go. The result is shown in Figure 2.4. The upper plot shows the power spectrum and the lower plot shows the time-domain signal. Note the four peaks, which indicate the modulated sinusoid. In the run window you can adjust the frequencies of the signal and the carrier as well as the amount of noise. These can also be adjusted in the block diagram in Figure 2.1 by double clicking on the bulleted parameters near the upper right of the window.

You can study the way the model is constructed in Figure 2.1. Note the Expression actor in the middle, whose icon indicates the expression being calculated:

```
signal*carrier + noise
```

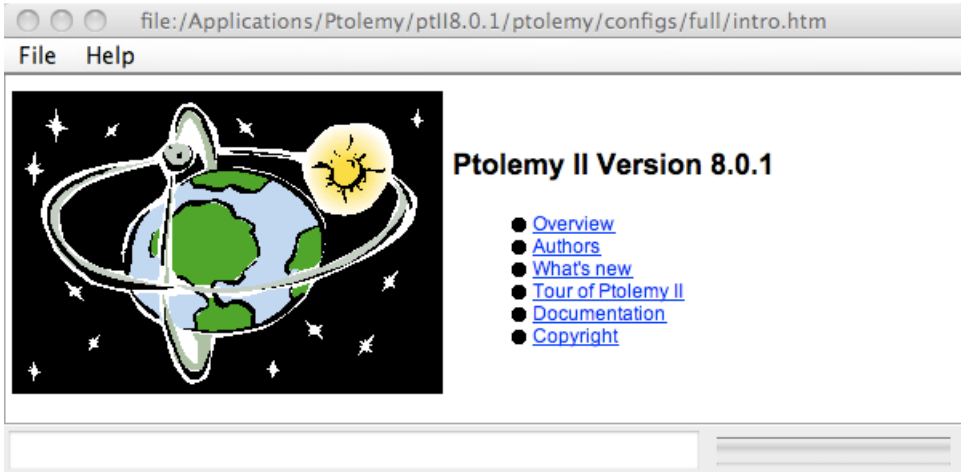


Figure 2.2: Initial welcome window.

The identifiers in this expression, `signal`, `carrier`, and `noise`, refer to the input ports by name. The names of these ports are shown in the diagram. The Expression actor is a very flexible actor in the Ptolemy II actor library. It can have any number of input ports, with arbitrary names, and uses a rich and expressive [expression language](#) to specify the value of the output as a function of the inputs (and parameters of the containing model, if desired). The expression language is described in Appendix ??.

By right clicking on an actor and selecting

```
[Documentation->Get Documentation]
```

you can view documentation for an actor. Figure 2.5 illustrates obtaining documentation for the [Expression](#) actor.

Three of the actors in Figure 2.1 are [composite actors](#), which means that their implementation is itself a Ptolemy II model. Composite actors are indicated visually by the red outline. You can invoke the `Open Actor` context menu² choice to reveal the implementation, as shown in Figure 2.6, which shows the implementation of the Signal Source

²A **context menu** that is particular to the object under the cursor. It is obtained by right clicking the mouse, if your mouse has a right button, or control clicking, if not, while the cursor is on the icon.

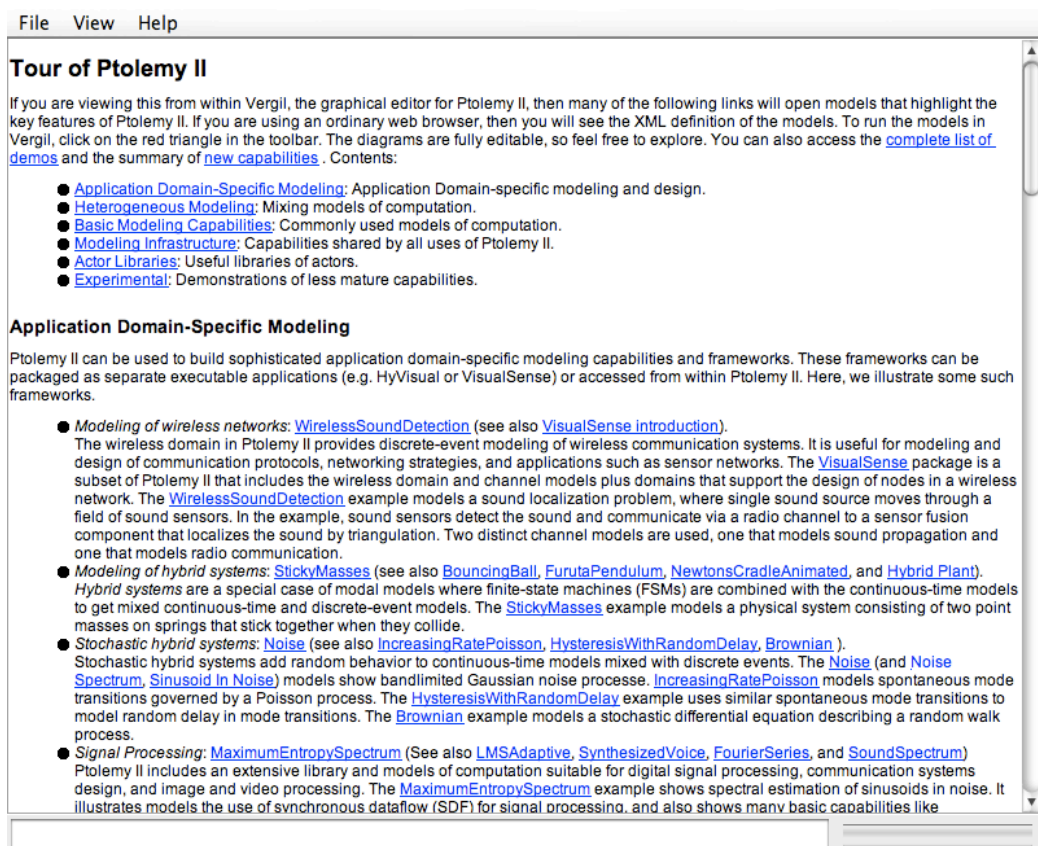


Figure 2.3: The tour of Ptolemy II page.

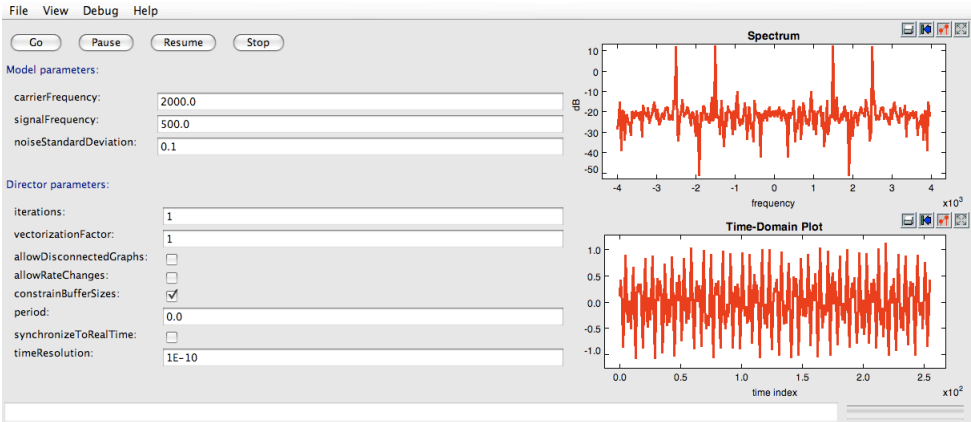


Figure 2.4: The Run Window for the model shown in Figure 2.1.

actor used in Figure 2.1. It is evident from the block diagram how a sinusoidal signal is generated.

2.1.2 Creating a New Model

Create a new model by selecting [File->New->Graph Editor] in the welcome window menu bar. You should see something like the window shown in Figure 2.7. Ignoring the menus and toolbar for a moment, on the left is a palette of objects that can be dragged onto the page on the right. To begin with, the page on the right is blank. Open the Actors library in the palette, and go into the Sources library. Find the **Const** actor under GenericSources and drag an instance over onto the blank page. Then go into the Sinks->GenericSinks and drag a **Display** actor onto the page. Each of these actors can be dragged around on the page. However, we would like to connect one to the other. To do this, drag a connection from the output port on the right of the **Const** actor to the input port of the **Display** actor. Lastly, open the Directors library and drag an SDFDirector onto the page. The **director** gives a meaning (semantics) to the graph, but for now we don't have to be concerned about exactly what that meaning is.

Now you should have something that looks like Figure 2.8. The Const actor is going to create our string, and the Display actor is going to print it out for us. We need to take care of one small detail to make it look like Figure 2.8: we need to tell the Const actor that

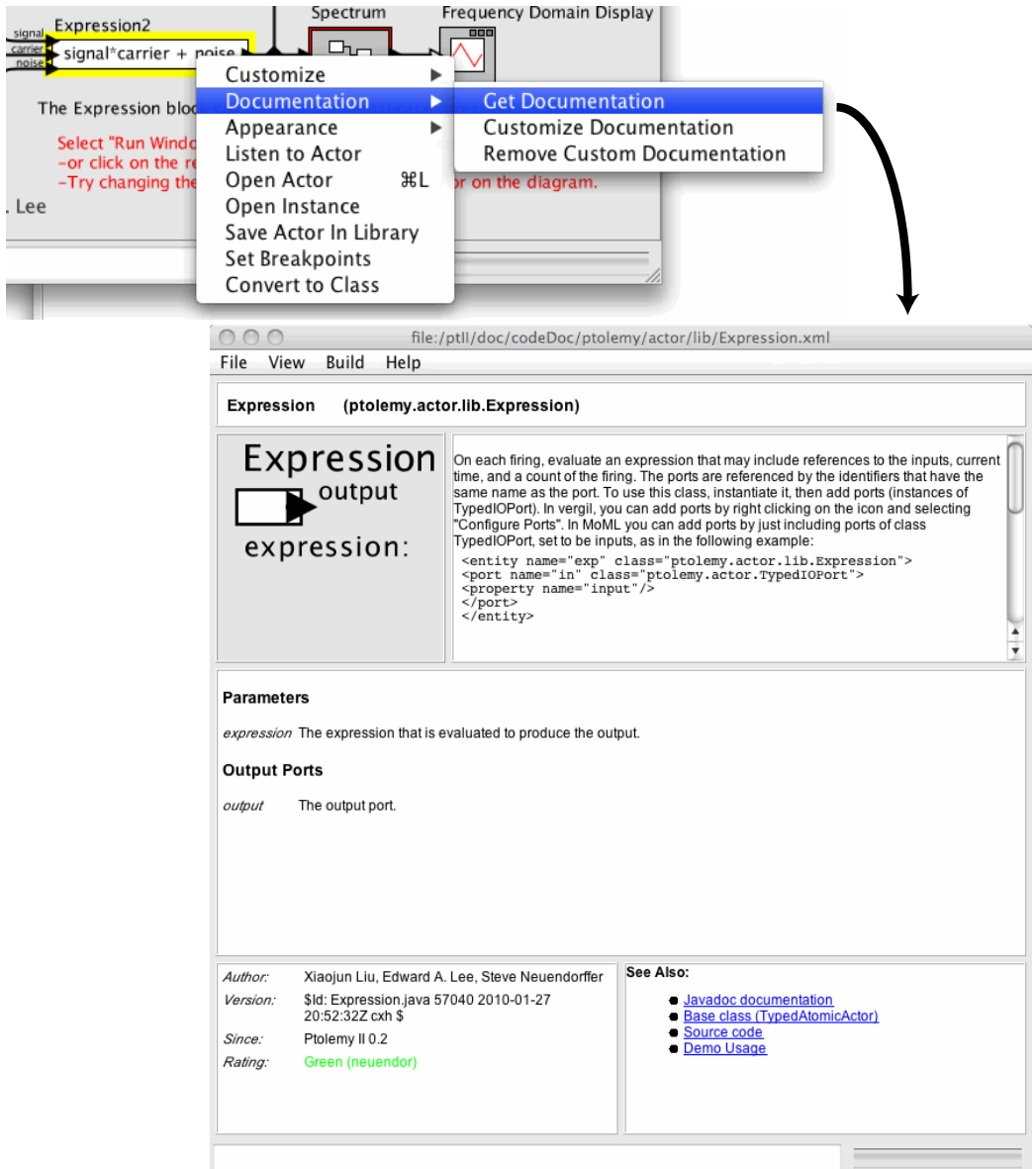


Figure 2.5: Viewing documentation for actors.

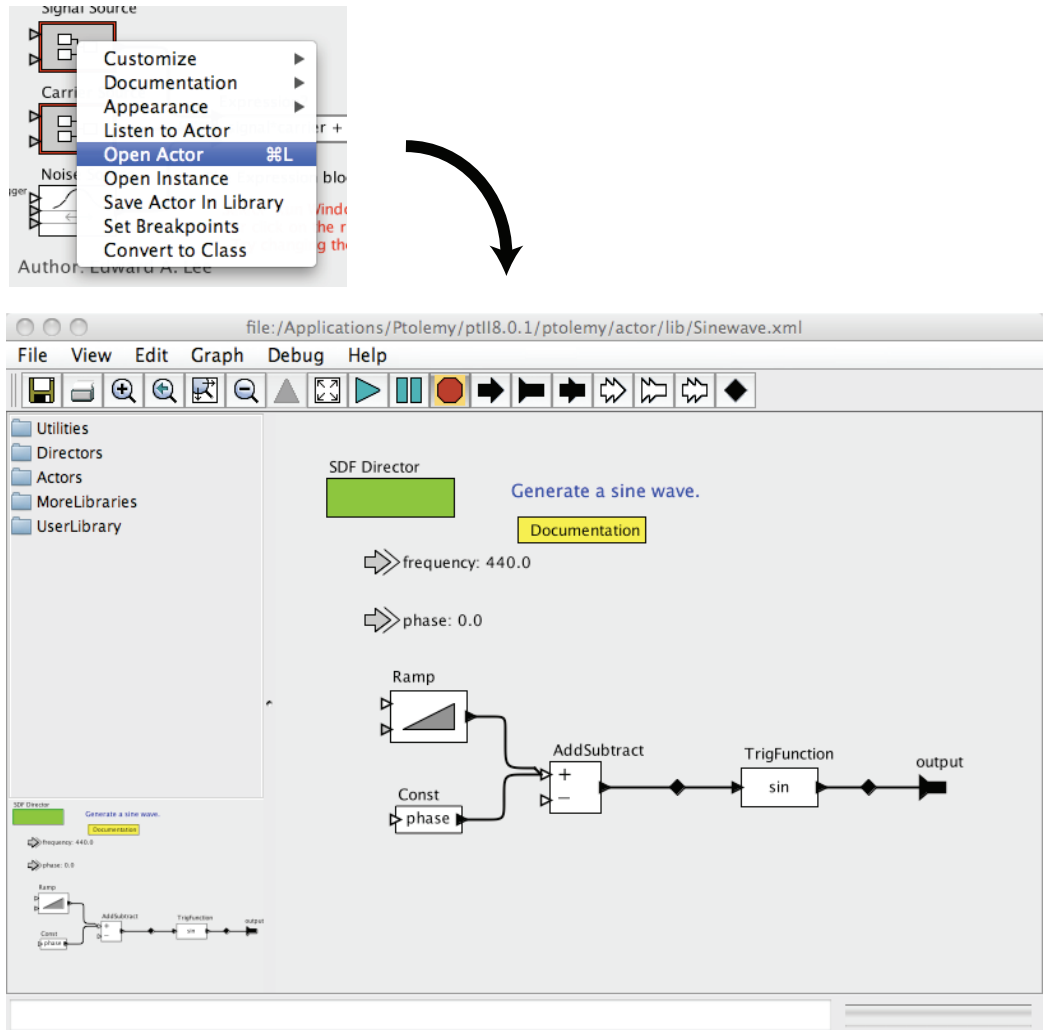


Figure 2.6: Invoke `Open Actor` on composite actors to reveal their implementation.

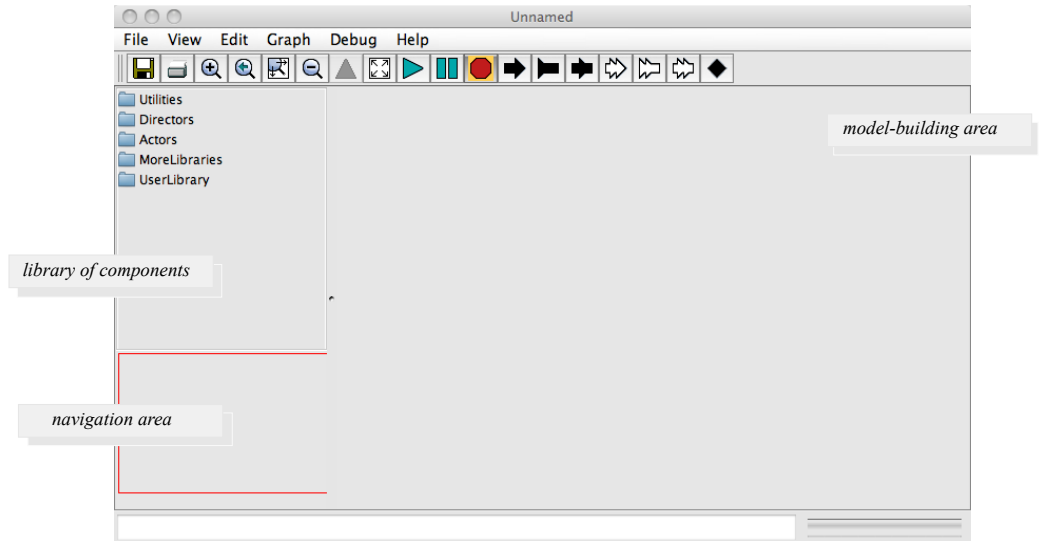


Figure 2.7: An empty Vergil Graph Editor.

we want the string “Hello World.” To do this we need to edit one of the parameters of the Const. To do this, either double click on the Const actor icon, or right click³ on the Const actor icon and select [Customize->Configure]. You should see the dialog box in Figure 2.9. Enter the string “Hello World” (with the quotation marks) for the *value* parameter and click the Commit button. Including the quotation marks ensures that the expression is interpreted as a string. You may wish to save your model, using the File menu. File names for Ptolemy II models should end in “.xml” or “.moml” so that Vergil will properly process the file the next time you open that file.

2.1.3 Running the Model

To run the example, go to the View menu and select the Run window. **Important:** Set the *iterations* parameter to something other than 0, say 1. This specifies how many times the model should repeat, and zero is interpreted by the SDFDirector as “run forever.” The model execution can be difficult to stop, as the executing thread will compete with the user interface thread, and there could be a considerable delay before the director responds

³If you have only one mouse button, instead of right clicking, hold the control key and click.

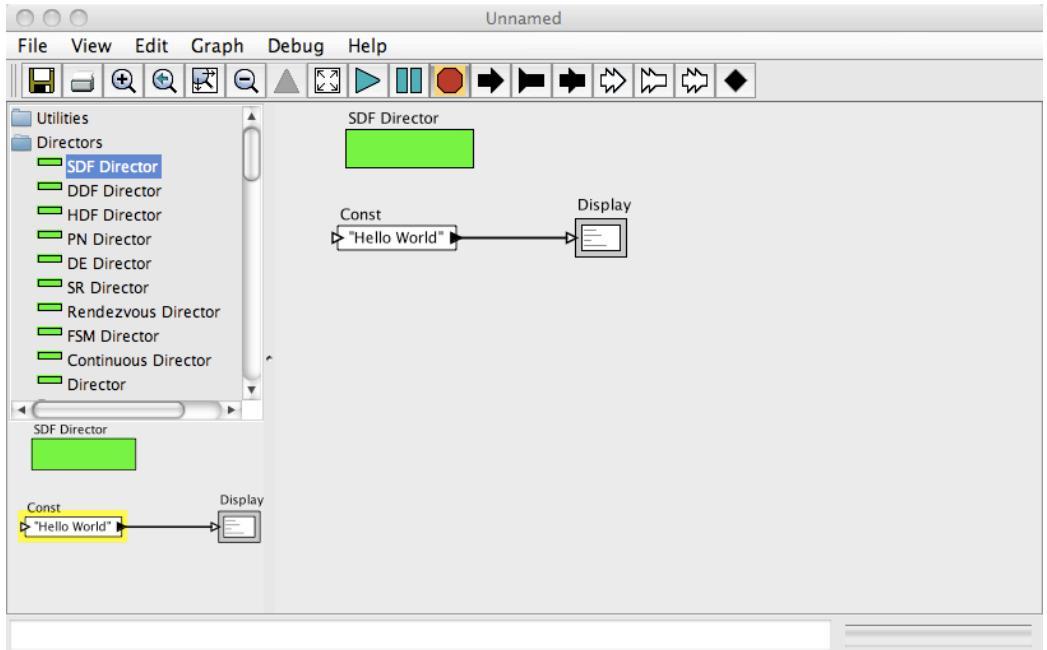


Figure 2.8: The Hello World example.

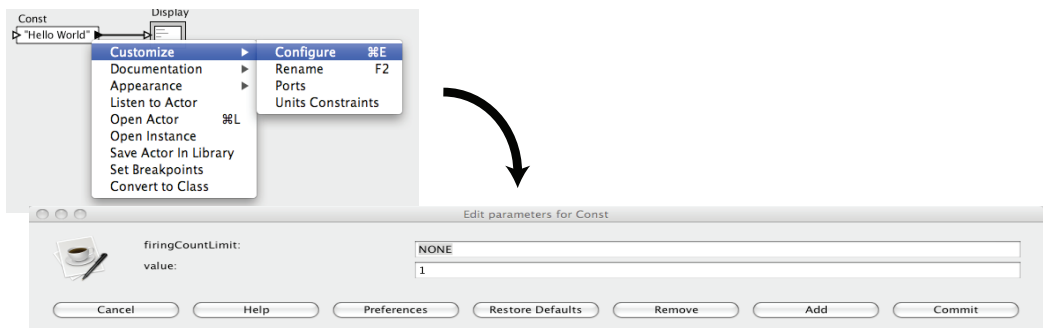


Figure 2.9: The Const parameter editor.

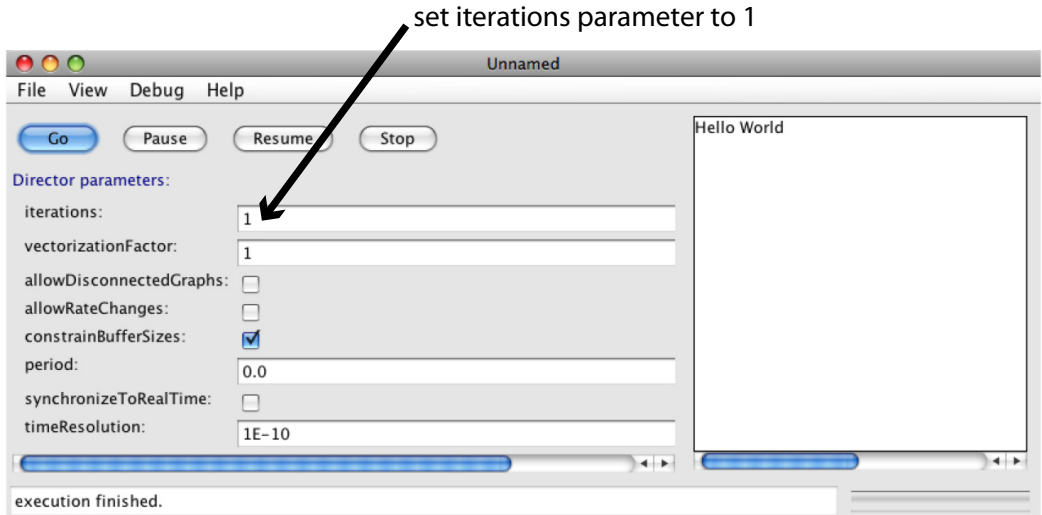


Figure 2.10: Execution of the Hello World example.

to the stop directive. You can set this parameter in the run window, or by double clicking on the director. If you click the “Go” button, you will see the string “Hello World” in the display at the right.

2.1.4 Making Connections

The model constructed above contains only two actors and one connection between them. If you move either actor (by clicking and dragging), you will see that the connection is routed automatically. We can now explore how to create and manipulate more complicated connections. First create a model in a new graph editor that includes an `SDFDirector`, a **Ramp** actor (found in the `Sources->SequenceSources` library, a `Display` actor, and a `SequencePlotter` actor, found in the `Sinks->SequenceSinks` library, as shown in Figure 2.11.

Suppose we wish to route the output of the Ramp to both the Display and the `SequencePlotter`. So how do we get the output of the Ramp to the other two actors? We need an explicit `relation` in the diagram. A relation is represented in the diagram by a black diamond, as shown in Figure 2.13. It can be created by any one of the ways below:

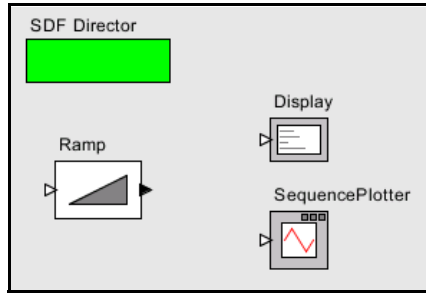


Figure 2.11: Three unconnected actors in a model.

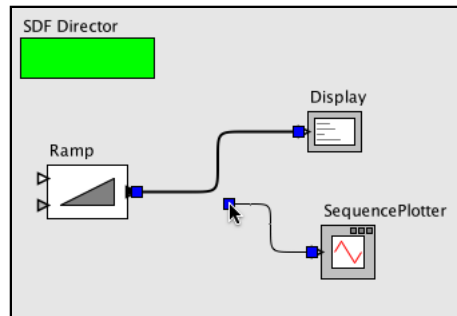


Figure 2.12: Dragging a connection to an existing one will create a three-way connection by creating an explicit relation.

- Drag the endpoint of a link into the middle of an existing line, as shown in Figure 2.12.
- Control-click⁴ on the background. A relation will appear.
- Click on the button in the toolbar with the black diamond on it, as shown in Figure 2.13.

⁴On a Macintosh, command-click.

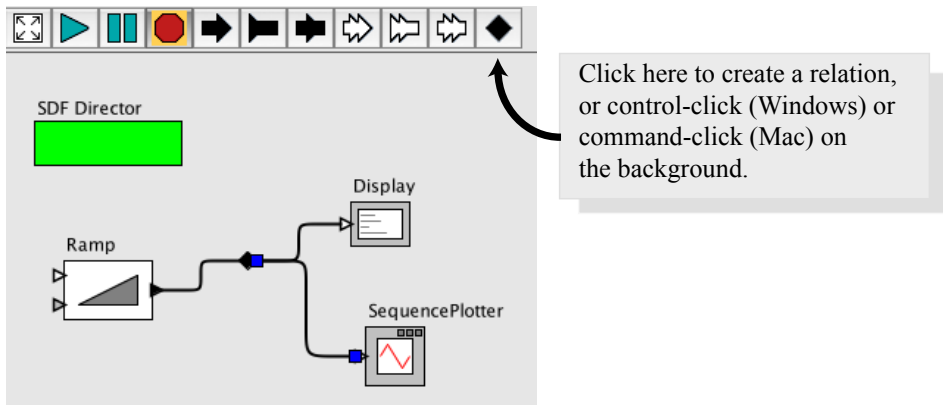


Figure 2.13: A relation can also be created by clicking on the black diamond in the toolbar.

Making a connection to a relation can be tricky, since if you just click and drag on the relation, the relation gets selected and moved. To make a connection, hold the control key while clicking and dragging on the relation.⁵

In the model shown in Figure 2.13, the relation is used to broadcast the output from a single port to a number of places. The single port still has only one connection to it, a connection to a relation. Relations can also be used to control the routing of wires in the diagram. Relations may be linked to other relations. Any two relations that are linked are said to be members of the same **relation group**. Semantically, a relation group has the same meaning as a single relation. In a relation group, there is no significance to the order in which relations are linked.

Ptolemy II supports two distinct flavors of ports, indicated in the diagrams by a filled triangle or an unfilled triangle. The output port of the Ramp actor is a **single port**, indicated by a filled triangle, which means that it can only support a single connection. The input port of the Display and SequencePlotter actors are **multiports**, indicated by unfilled triangles, which means that they can support multiple connections. Each connection is treated as a separate **channel**, which is a path from an output port to an input port (via relations) that can transport a single stream of tokens.

⁵ On a Macintosh, hold the command key rather than the control key.

To explore multiports, try putting some other signal source in the diagram and connecting it to the SequencePlotter or to the Display. If you explore this fully, you will discover that the SequencePlotter can only accept inputs of type *double*, or some type that can be losslessly converted to *double*, such as *int*. These data type issues are explored next.

2.2 Tokens and Data Types

In the example of Figure 2.8, the Const actor creates a sequence of values on its output port. The values are encapsulated as **tokens**, and sent to the Display actor, which consumes them and displays them in the Run Window. A token in Ptolemy II is a generic wrapper for a unit of data that is communicated between actors via ports.

The tokens produced by the Const actor can have any value that can be expressed in the Ptolemy II [expression language](#) (see Appendix ??). Try giving the value 1 (the integer with value one), or 1.0 (the floating-point number with value one), or {1.0} (an array containing a one), or {value=1, name="one"} (a record with two elements: an integer named “value” and a string named “name”), or even [1,0;0,1] (the two-by-two identity matrix). These are all expressions.

The Const actor is able to produce data with different **types**, and the Display actor is able to display data with different types. Most actors in the actor library are **polymorphic actors**, meaning that they can operate on or produce data with multiple types. The behavior may even be different for different types. Multiplying matrices, for example, is not the same as multiplying integers, but both are accomplished by the **MultiplyDivide** actor in the Math library. Ptolemy II includes a sophisticated **type system** that allows this to be done efficiently and safely (Xiong, 2002).

To explore data types a bit further, try creating the model in Figure 2.14. The Ramp actor is listed under the Sources->SequenceSources sublibrary, and the **AddSubtract** actor is listed under the Math library. Set the *value* parameter of the Const to be 0 and the *iterations* parameter of the director to 5. Running the model should result in 5 numbers between 0 and 4, as shown in the figure. These are the values produced by the Ramp, which are having the value of the Const actor subtracted from them. Experiment with changing the value of the Const actor and see how it changes the 5 numbers at the output.

Now for the real test: change the value of the Const actor back to "Hello World". When you execute the model, you should see an **exception** window, as shown in Figure 2.15. Do not worry; exceptions are a normal part of constructing (and debugging) models.

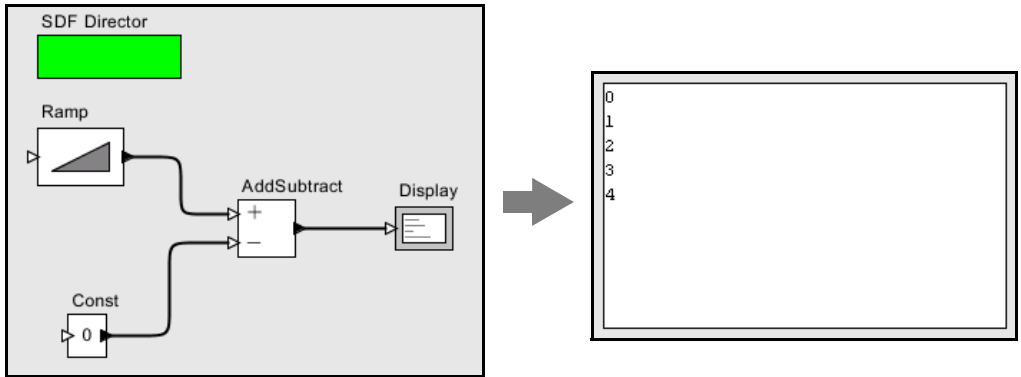


Figure 2.14: Another example, used to explore data types in Ptolemy II.

In this case, the exception window is telling you that you have tried to subtract a string value from an integer value, which doesn't make much sense at all (following Java, adding strings is allowed, but not subtracting them). This is an example of a **type error**.

The actor that caused the exception is highlighted and the name of the actor, “.helloWorld-SubtractError.AddSubtract” is mentioned in the exception. In Ptolemy II models, all objects have a dotted name. The dots separate elements in the hierarchy. Thus, “.helloWorldSubtractError.AddSubtract” is an object named “AddSubtract” contained by a object named “.helloWorldAddSubtractError”. This assumes that the model was saved as a file called `helloWorldAddSubtract.xml`.

Exceptions can be a very useful debugging tool, particularly if you are developing your own components in Java. To illustrate how to use them, click on the Display Stack Trace button in the exception window of Figure 2.15. You should see a stack trace like that shown in Figure 2.15. This window displays the execution sequence that resulted in the exception. For example, if you scroll towards the bottom, you will see a line like

```
at ptolemy.data.StringToken._subtract (StringToken.java:359)
```

indicating that the exception occurred within the `subtract()` method of the class `ptolemy.data.StringToken`, at line 359 of the source file `StringToken.java`. Since Ptolemy II is distributed with source code (most installation mechanisms at least offer the

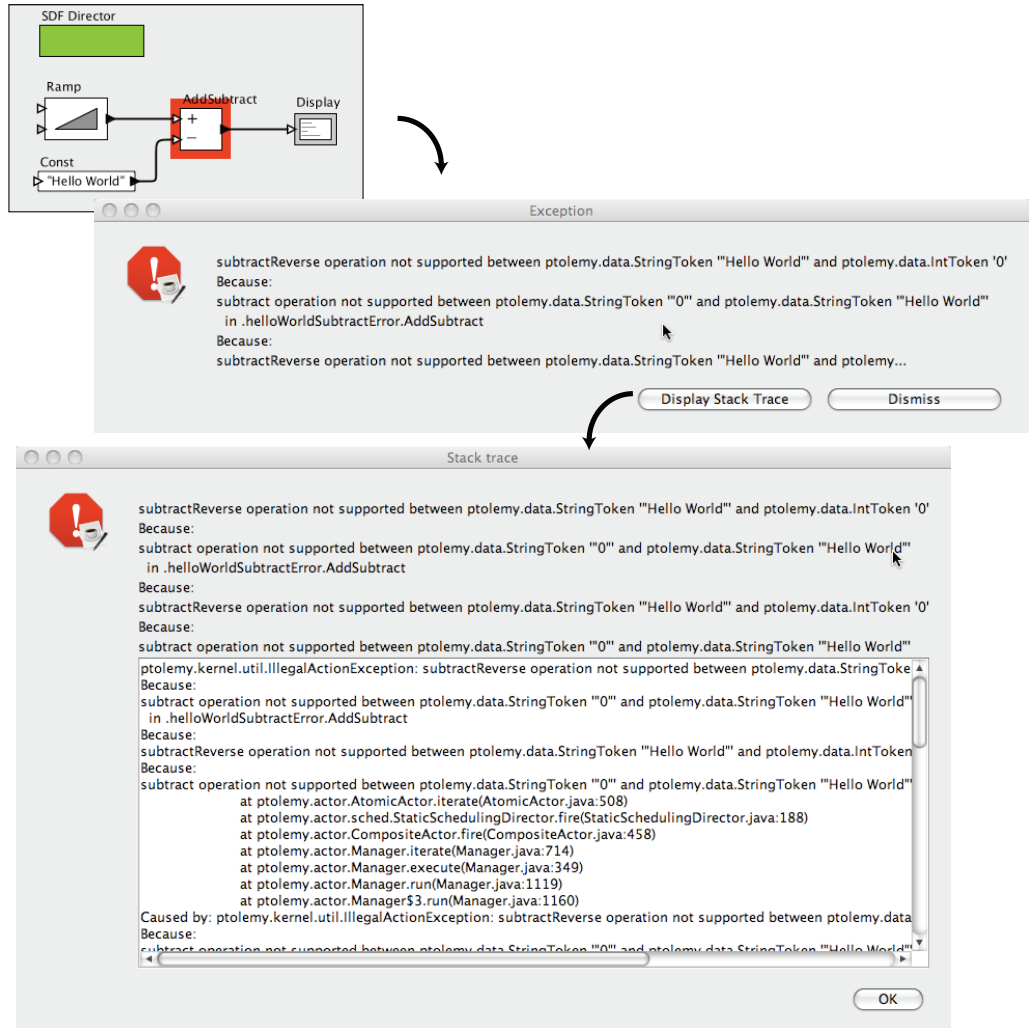


Figure 2.15: An example that triggers an exception when you attempt to execute it. Strings cannot be subtracted from integers. Examining the stack trace for such exceptions can sometimes be useful.

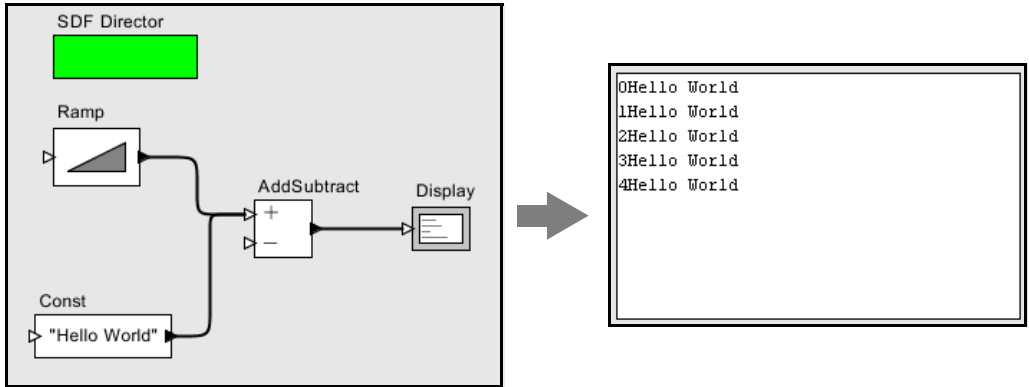


Figure 2.16: Addition of a string to an integer.

option of installing the source), this can be very useful information. For type errors, you probably do not need to see the stack trace, but if you have extended the system with your own Java code, or you encounter a subtle error that you do not understand, then looking at the stack trace can be very illuminating.

To find the file `StringToken.java` referred to above, find the Ptolemy II installation directory. If that directory is `$PTII`, then the location of this file is given by the full class name, but with the periods replaced by slashes; in this case, it is at

```
$PTII/ptolemy/data/StringToken.java
```

The slashes might be backslashes under Windows.

Let's try a small change to the model to get something that does not trigger an exception. Disconnect the Const from the lower port of the AddSubtract actor and connect it instead to the upper port, as shown in Figure 2.16. You can do this by selecting the connection and deleting it (using the delete key), then adding a new connection, or by selecting it and dragging one of its endpoints to the new location. Notice that the upper port is an unfilled triangle; this indicates that it is a [multiport](#), meaning that you can make more than one connection to it. Now when you run the model you should see strings like "0HelloWorld", as shown in the figure.

There are two interesting things going on here. The first is that, as in Java, strings are added by concatenating them. The second is that the integers from the Ramp are converted to strings and concatenated with the string “Hello World”. All the connections to a multiport must have the same type. In this case, the multiport has a sequence of integers coming in (from the Ramp) and a sequence of strings (from the Const).

Ptolemy II automatically converts the integers to strings when integers are provided to an actor that requires strings. But in this case, why does the AddSubtract actor require strings? Because it would not work to require integers; the string “Hello World” would have to be converted to an integer. As a rough guideline, Ptolemy II will perform automatic type conversions when there is no loss of information. An *int* can be converted to a *string*, but not vice versa. An *int* can be converted to a *double*, but not vice versa. An *int* can be converted to a *long*, but not vice versa. The details are explained in Appendix ??, but many users will not need to understand the full sophistication of the system. You should find that most of the time it will just do what you expect.

To further explore data types, try modifying the Ramp so that its parameters have different types. For example, try making *init* and *step* strings.

2.3 Hierarchy

Ptolemy II supports (and encourages) hierarchical models. These are models that contain components that are themselves models. Such components are called **composite actors**. Consider a small signal processing problem, where we are interested in recovering a signal based only on noisy measurements of it. We will create a composite actor modeling a communication channel that adds noise, and then use that actor in a model.

2.3.1 Creating a Composite Actor

First open a new graph editor and drag in a **CompositeActor** from the Utilities library. This actor is going to add noise to our measurements. First, using the context menu (obtained by right clicking⁶ over the composite actor), select [Customize->Rename], and give the composite a better name, like `Channel`, as shown in Figure 2.17. (Note that you can alternatively give a *Display name*, which is arbitrary text that will be displayed instead of the name of the actor.) Then, using the context menu again, select Open

⁶On a Macintosh, control-click.

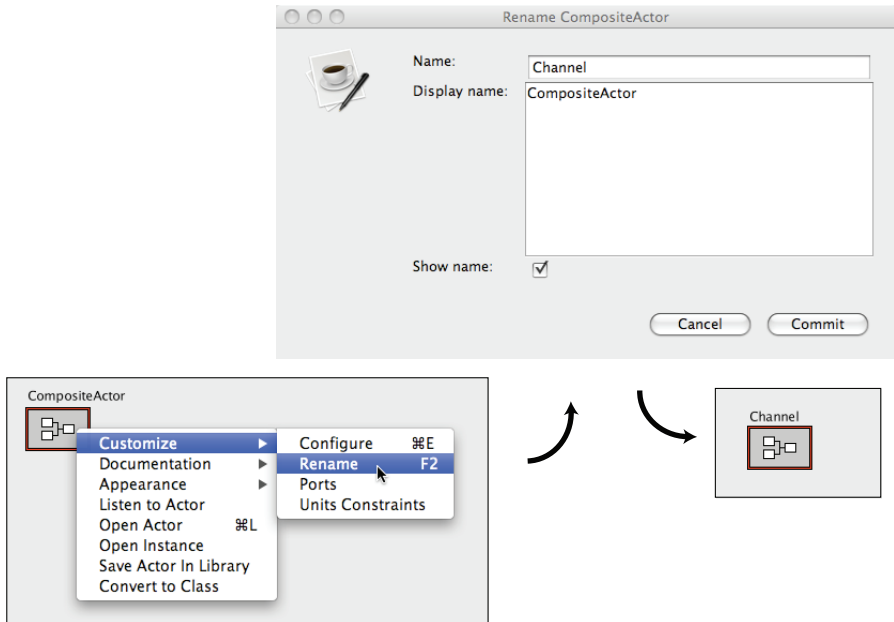


Figure 2.17: Changing the name of an actor.

Actor. You should get a blank graph editor, as shown in Figure 2.18. The original graph editor is still open. To see it, move the new graph editor window by dragging the title bar of the window. Alternatively, you can click on the upward pointing triangle in the toolbar to navigate back to the container.

2.3.2 Adding Ports to a Composite Actor

First we have to add some ports to the composite actor. There are several ways to do this, but clicking on the port buttons in the toolbar is probably the easiest. You can explore the ports in the toolbar by lingering with the mouse over each button in the toolbar. A tool tip pops up that explains the button. The buttons are summarized in Figure 2.19.

Create an input port and an output port and rename them *input* and *output* by right clicking on the ports and selecting [Customize->Rename]. Note that, as shown in Figure

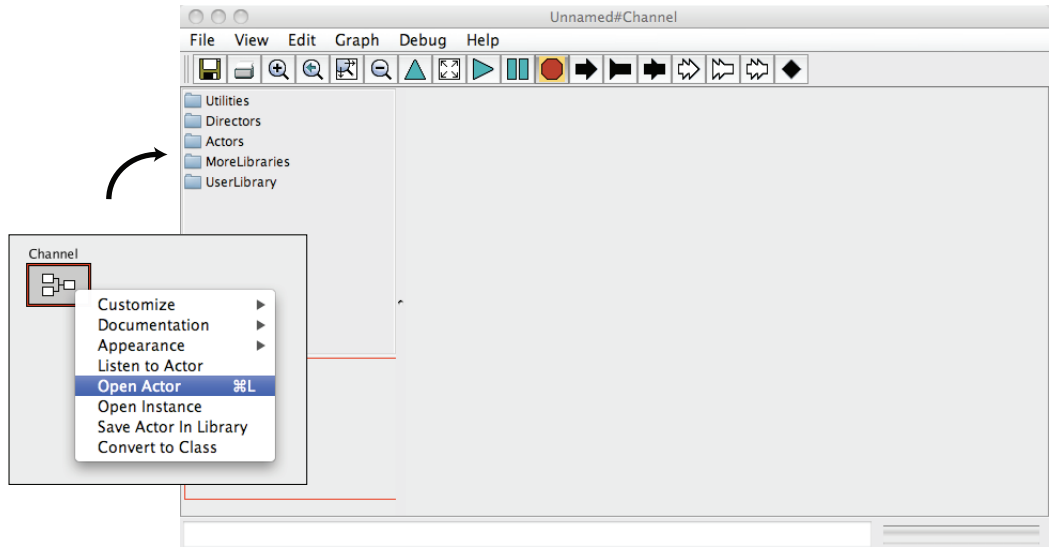


Figure 2.18: Opening a new composite actor, which shows the blank inner actor.

2.20, you can also right click⁷ on the background of the composite actor and select [Customize->Ports] to add ports, remove ports, or change whether a port is an input, an output, or a multiport. The resulting dialog also allows you to set the type of the port, although much of the time you will not need to do this, since the type inference mechanism in Ptolemy II will figure out the type from the connections. You can also specify the **direction of a port**.⁸ You can also control whether the name of the port is shown outside the icon (by default it is not), and even whether the port is shown at all.

Then, using these ports, create the diagram shown in Figure 2.21⁹. The **Gaussian** actor creates values from a Gaussian distributed random variable, and is found in the Random library. Now if you close this editor and return to the previous one, you should be able to easily create the model shown in Figure 2.22. The **Sinewave** actor is listed under the Sources->SequenceSources library, and the **SequencePlotter** actor is found under the Sinks->SequenceSinks library. Notice that the Sinewave actor is also a **com-**

⁷On a Macintosh, control-click.

⁸The direction of a port is where it appears on the icon; by default input ports appear on the left, output ports on the right, and ports that are both inputs and outputs appear on the bottom of the icon.

⁹Hint: to create a connection starting on one of the external ports, hold down the control key when dragging, or on a Macintosh, the command key.

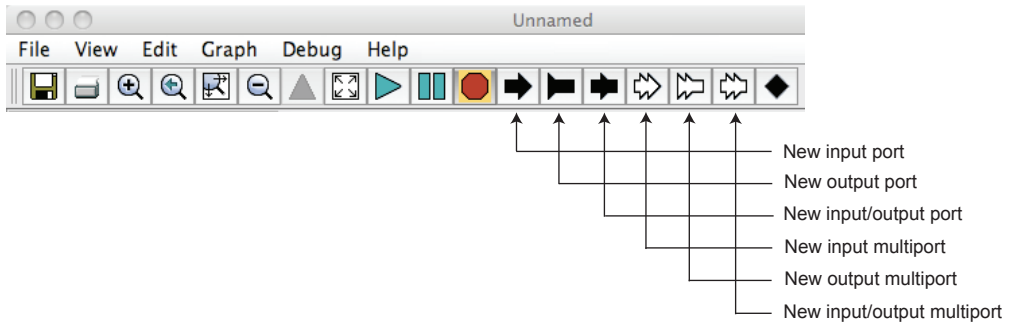


Figure 2.19: Summary of toolbar buttons for creating new ports.

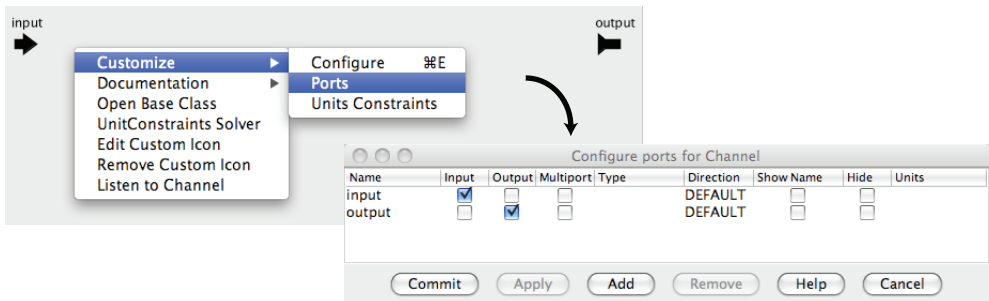


Figure 2.20: Right clicking on the background brings up a dialog that can be used to configure ports.

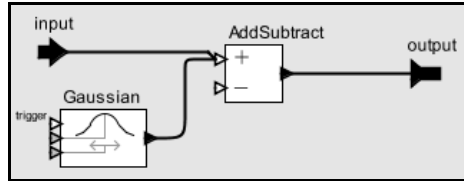


Figure 2.21: A simple channel model defined as a composite actor.

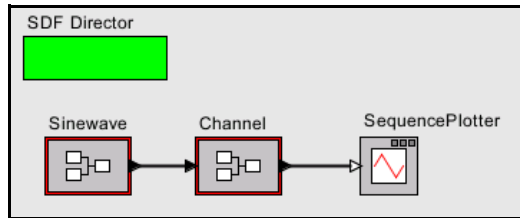


Figure 2.22: A simple signal processing example that adds noise to a sinusoidal signal.

[posite actor](#), as suggested by its red outline (try opening the actor). If you execute this model (you will probably want to set the *iterations* parameter of the director to something reasonable, like 100), you should see something like Figure 2.23.

2.3.3 Setting the Types of Ports

In the above example, we never needed to define the types of any ports. The types were inferred from the connections and the constants in the model. Indeed, this is usually the case in Ptolemy II, but occasionally, you will need to set the types of the ports. Notice in Figure 2.20 that there is a column in the dialog box that configures ports for specifying the type. Thus, to specify that a port has type *boolean*, you could enter *boolean* into the dialog box. There are other commonly used types: *complex*, *double*, *fixedpoint*, *float*, *general*, *int*, *long*, *matrix*, *object*, *scalar*, *short*, *string*, *unknown*, *unsignedByte*, *xmlToken*, *arrayType(int)*, *arrayType(int, 5)*, *[double]* and *{ x=double, y=double }*. To specify that the type of a port is a double matrix enter in the type column of for the corresponding port the following:

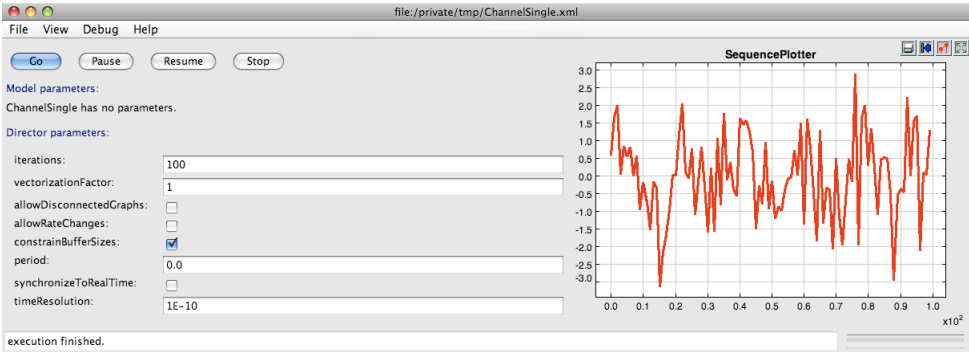


Figure 2.23: The output of the simple signal processing model in Figure 2.22.

```
[double]
```

This expression actually creates a 1 by 1 matrix containing a *double* (the value of which is irrelevant). It thus serves as a prototype to specify a double matrix type. Similarly, we can specify an array of complex numbers as

```
{complex}
```

In the Ptolemy II expression language, square braces are used for matrices, and curly braces are used for arrays. An **array** is an ordered list of tokens of arbitrary type. A **matrix** is one or two dimensional structure containing numeric types.

A **record** is a data structure consisting of an arbitrary number of elements, each of which has a name and a value, where the value can be of any type. To specify a record containing a string named “name” and an integer named “address” you can write:

```
{name=string, address=int}
```

Details about arrays, matrices, and records are given in Appendix ???. Details of the type system are given in Appendix ???.

2.4 Annotations and Parameterization

In this section, we will enhance the model in Figure 2.22 in a number of ways. We will add parameters, insert decorative and documentary annotations, and customize the icons for actors.

2.4.1 Parameters in Hierarchical Models

First, notice from Figure 2.23 that the noise overwhelms the sinusoid, making it barely visible. A useful channel model would have a **parameter** that sets the level of the noise. Open the channel model by right clicking on the channel model, and select `Open Actor`. In the channel model, add a parameter by dragging one in from the `Utilities->Parameters` sublibrary, as shown in Figure 2.24. Right click¹⁰ on the parameter to change its name to `noisePower`. (In order to be able to use this parameter in expressions, the name cannot have any spaces in it.) Also, right click or double click on the parameter to change its default value to 0.1.

Now we can use this parameter. First, let's use it to set the amount of noise. The `Gaussian` actor has a parameter called `standardDeviation`. In this case, the power of the noise is equal to the variance of the Gaussian, not the standard deviation. If you recall from basic statistics, the standard deviation is equal to the square root of the variance. Change the `standardDeviation` parameter of the Gaussian actor so its value is `sqrt(noisePower)`, as shown in Figure 2.25. This is an expression that references the `noisePower` parameter. (See Appendix ?? for details about the the expression language.)

We can check our improved model by returning to the top-level model and editing the parameters of the Channel actor (by either double clicking or right clicking and selecting [`Customize->Configure`]). Change the `noisePower` from the default 0.1 to 0.01. Run the model. You should now get a relatively clean sinusoid like that shown in Figure 2.26.

Note that you can also add parameters to a composite actor without dragging from the `Utilities` library by clicking on the “Add” button in the edit parameters dialog for the Channel composite. This dialog is accessed by either double clicking on the Channel icon, or by right clicking and selecting [`Customize->Configure`], or by right clicking on the background inside the composite and selecting [`Customize->Configure`]. Parameters that are added this way will not be visible in the diagram, so this mechanism

¹⁰On a Macintosh, control-click.

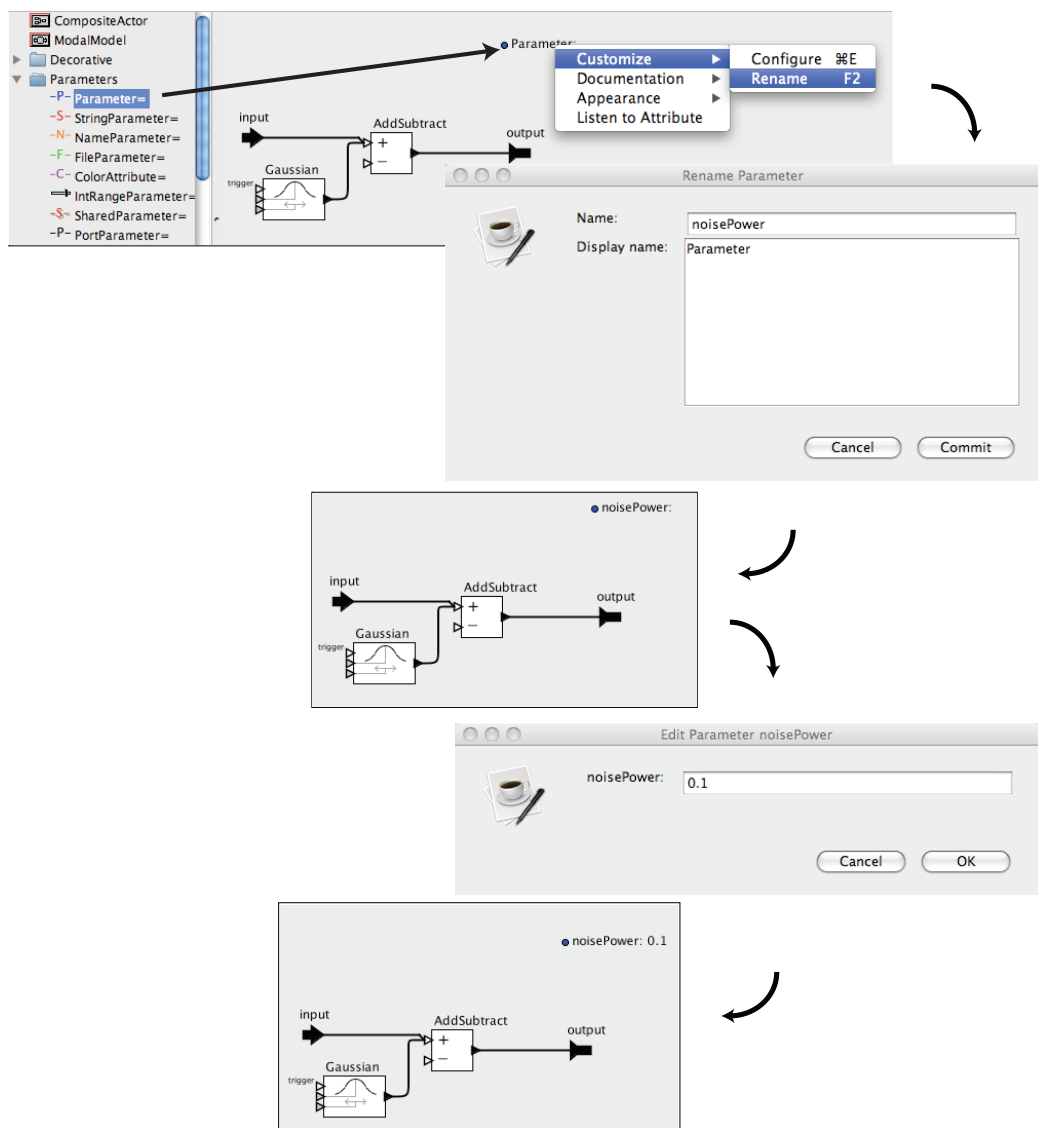


Figure 2.24: Adding a parameter to the channel model.

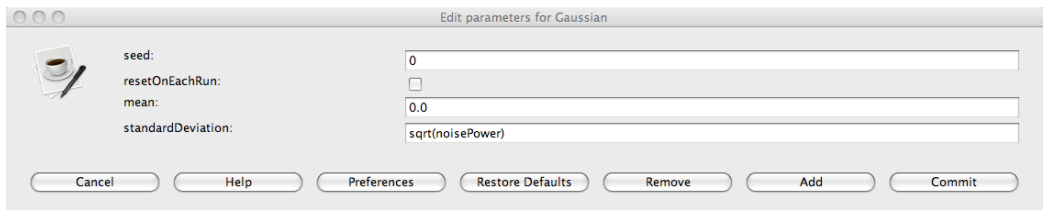


Figure 2.25: The standard deviation of the Gaussian actor is set to the square root of the noise power.

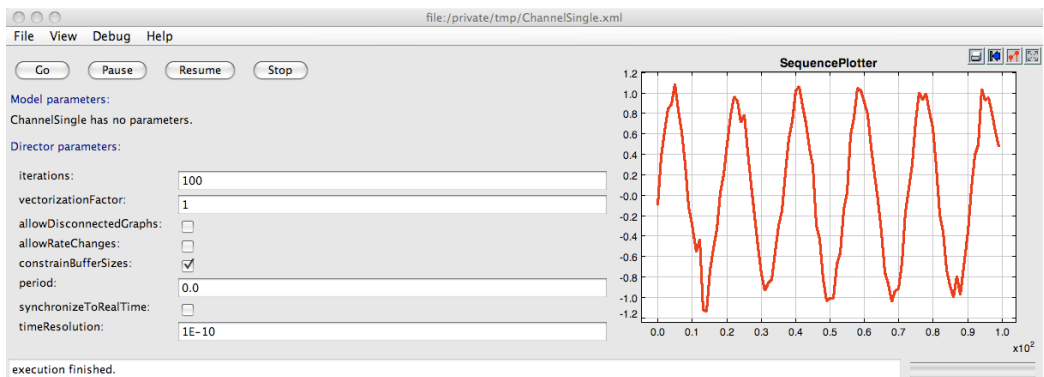


Figure 2.26: The output of the simple signal processing model in Figure 2.22 with noise power = 0.01.

should be used sparingly and only when there is a good reason to hide parameters from someone browsing the model.

2.4.2 Decorative Elements

There are several other useful enhancements you could make to this model. Try dragging an *Annotation* from the Utilities->Decorative sublibrary, and creating a title on the diagram. Such annotations are highly recommended. They correspond to comments in programs and can greatly improve the readability of a model. A limited number of other decorative elements like geometric shapes can also be added to the diagram from this same library.

2.4.3 Creating Custom Icons

A (rather primitive) icon editor is also provided with Vergil. To create a custom icon, right click on the icon and select [Appearance->Edit Custom Icon], as shown in Figure 2.27. The box in the middle of the icon editor displays the size of the default icon, for reference. Try creating an icon like the one shown in Figure 2.28. Hint: The fill color of the rectangle is set to `none` and the fill color of the trapezoid is first selected using the color selector, then modified to have an *alpha* (transparency) of 0.5. Finally, since the icon itself has the actor name in it, the [Customize->Rename] dialog is used to deselect *show name*.

2.5 Navigating Larger Models

Sometimes, a model gets large enough that it is not convenient to view it all at once. There are four toolbar buttons, shown in Figure 2.29 that help. These buttons permit zooming in and out. The “Zoom reset” button restores the zoom factor to the “normal” one, and the “Zoom fit” calculates the zoom factor so that the entire model is visible in the editor window.

In addition, it is possible to pan over a model. Consider the window shown in Figure 2.30. Here, we have zoomed in so that icons are larger than the default. The **pan window** at the lower left shows the entire model, with a red box showing the visible portion of the model. By clicking and dragging in the pan window, it is easy to navigate around the

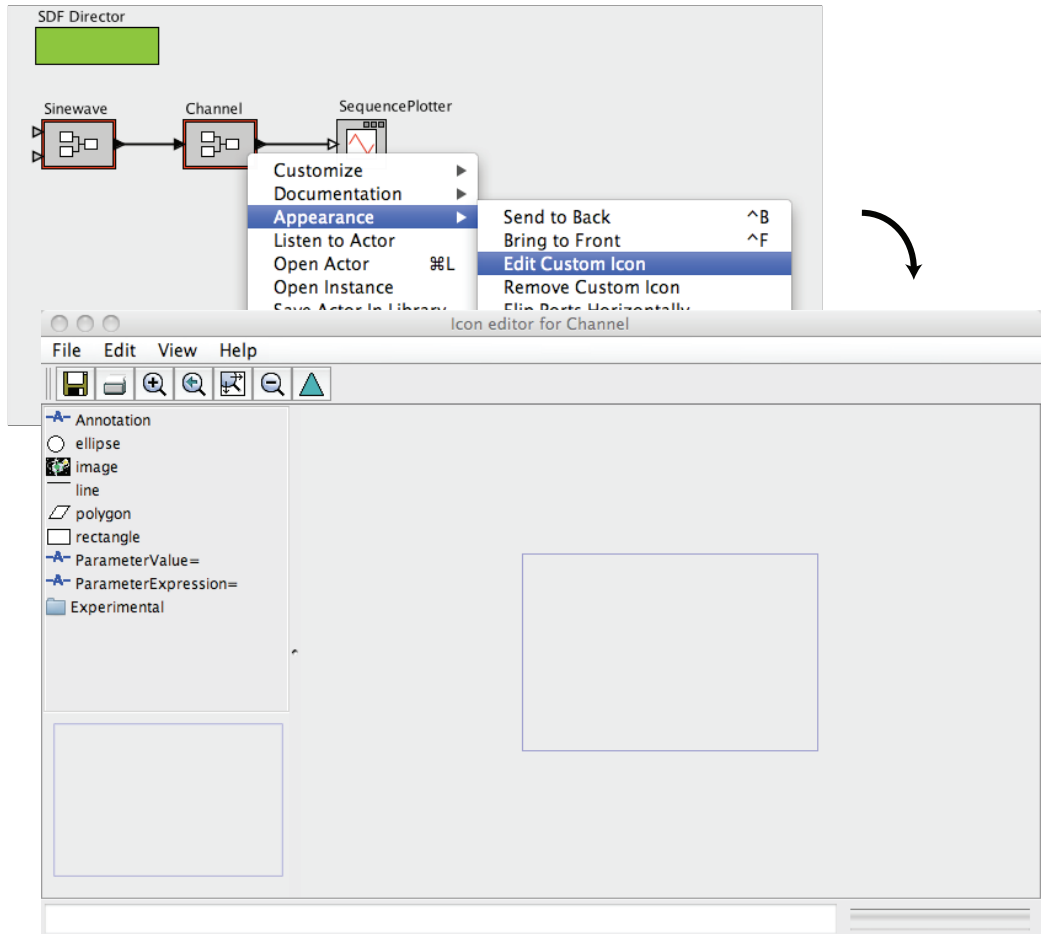


Figure 2.27: Custom icon editor for the Channel actor.

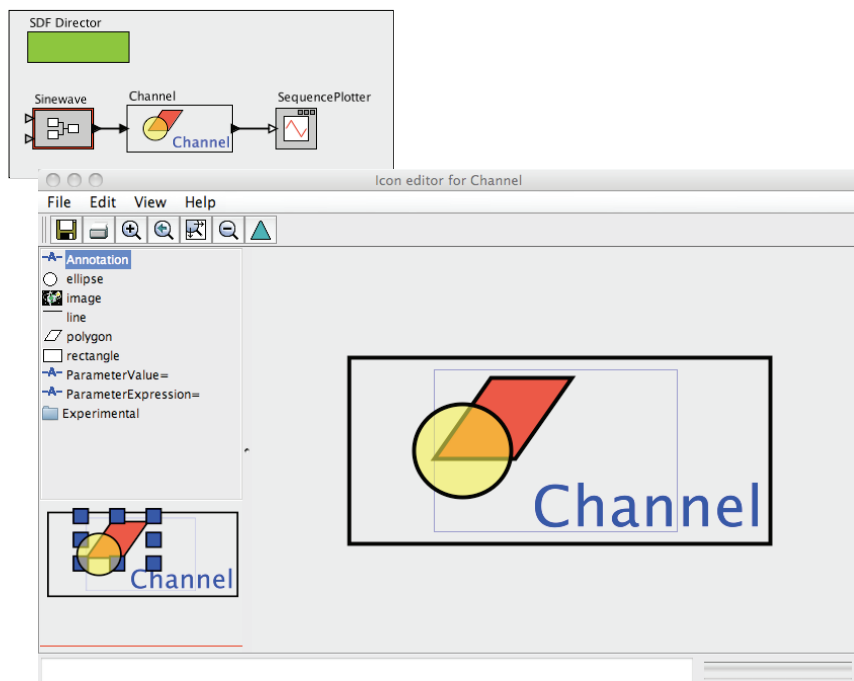


Figure 2.28: Custom icon for the Channel actor.

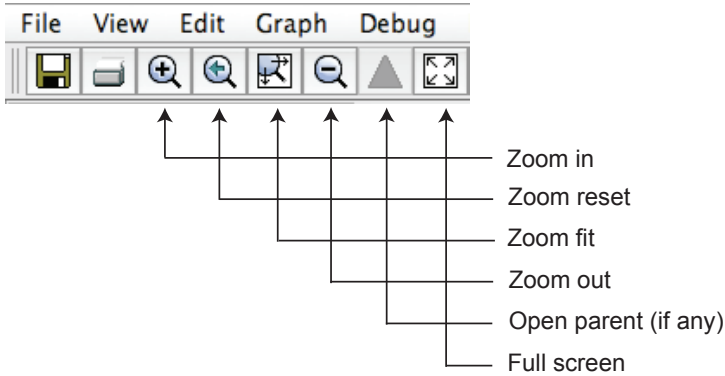


Figure 2.29: Summary of toolbar buttons for zooming and fitting.

entire model. Clicking on the “Zoom fit” button in the toolbar results in the editor area showing the entire model, just as the pan window does.

2.6 Classes and Inheritance

Ptolemy II includes the ability to define **actor-oriented classes** (Lee et al., 2009) with **instances** and **subclasses** with **inheritance**. The key idea is that you can specify that a component definition is a **class**, in which case all instances and subclasses inherit its structure. This improves modularity in designs. We will illustrate this capability with an example.

2.6.1 Creating and Using Actor-Oriented Classes

Consider the model that we developed in section 2.3, shown for reference in Figure 2.31. Suppose that we wish to create multiple instances of the channel, as shown in Figure 2.32. In that figure, the sinewave signal passes through five distinct channels (note the use of a relation to broadcast the same signal to each of the five channels). The outputs of the channels are added together and plotted. The result is a significantly cleaner sine

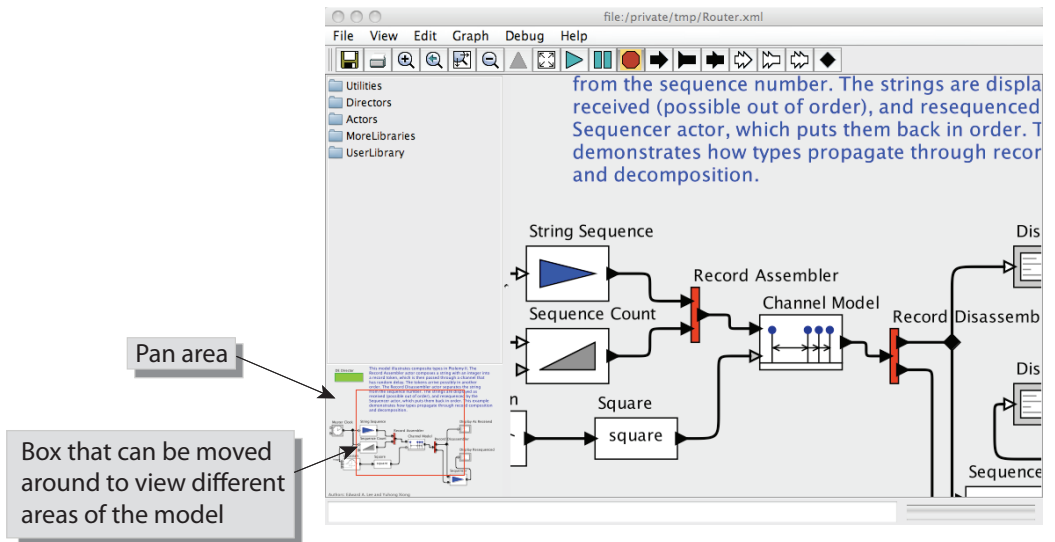


Figure 2.30: The pan window at the lower left has a red box representing the visible area of the model in the main editor window. This red box can be moved around to view different parts of the model.

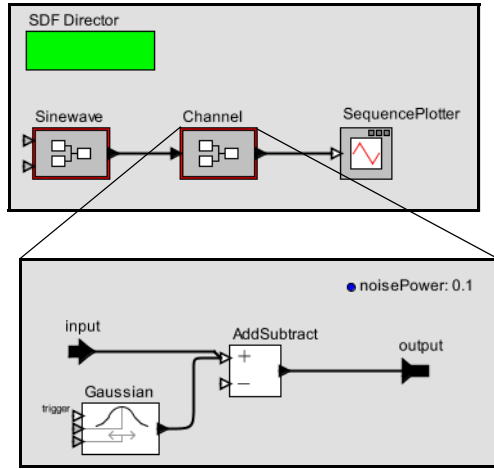


Figure 2.31: Hierarchical model that we will modify to use classes.

wave than the one that results from one channel alone¹¹. However, this is a poor design, for two reasons. First, the number of channels is hardwired into the diagram. We will deal with that problem in the next section. Second, each of the channels is a *copy* of the composite actor in Figure 2.31. This results in a far less maintainable or scalable model than we would like. Consider, for example, what it would take to change the design of the channel. Each of the five copies would have to be changed individually.

A better solution is to define a Channel class. To do this, begin with the design in Figure 2.31, and remove the connections to the channel, as shown in Figure 2.33. Then right click and select `Convert to Class`. (Note that if you fail to first remove the connections, you will get an error message when you try to convert to class. A class is not permitted to have connections.) The actor icon acquires a blue halo, which serves as a visual indication that it is a class, rather than an ordinary actor (which is an instance). Classes play no role in the execution of the model, and merely serve as definitions of components that must then be instantiated. By convention, we put classes at the top of the model, near the director, since they function as declarations.

¹¹In communication systems, this technique is known as diversity, where multiple channels with independent noise are used to achieve more reliable communication.

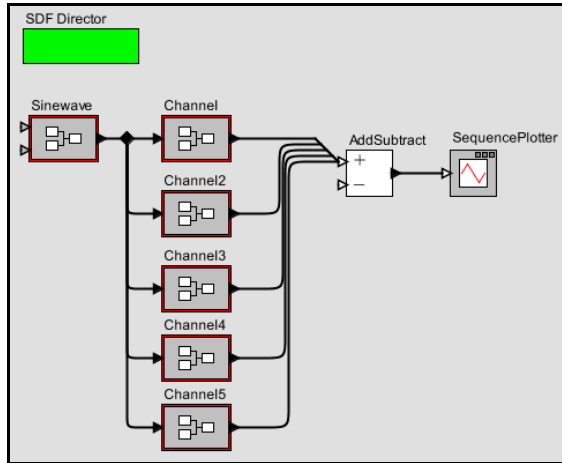


Figure 2.32: A poor design of a diversity communication system, which has multiple copies of the channel as defined in Figure 2.31.

Once you have a class, you can create an instance by right clicking and selecting `Create instance` or typing `Control-N`. Do this five times to create five instances of the class, as shown in Figure 2.33. Although this looks similar to the design in Figure 2.32, it is, in fact, a much better design. To verify this, try making a change to the class, for example by creating a custom icon for it, as shown in Figure 2.34. Note that the changes propagate to each of the instances of the class. A more subtle advantage is that the XML file representation of the model is much smaller, since the design of the class is given only once rather than five times.

If you invoke `Open Actor` any of the instances (or the class) in Figure 2.34, you will see the same channel model. In fact, you will see the class definition. Any change you make inside this hierarchical model will be automatically propagated to all the instances. Try changing the value of the *noisePower* parameter, for example.

If you wish to view the instance rather than the class definition, you can select `Open Instance` on one of the instances. The window that opens shows only that instance. Each of the subcomponents that it inherits from the class definition is highlighted with a pink halo, indicating that this is an inherited component.

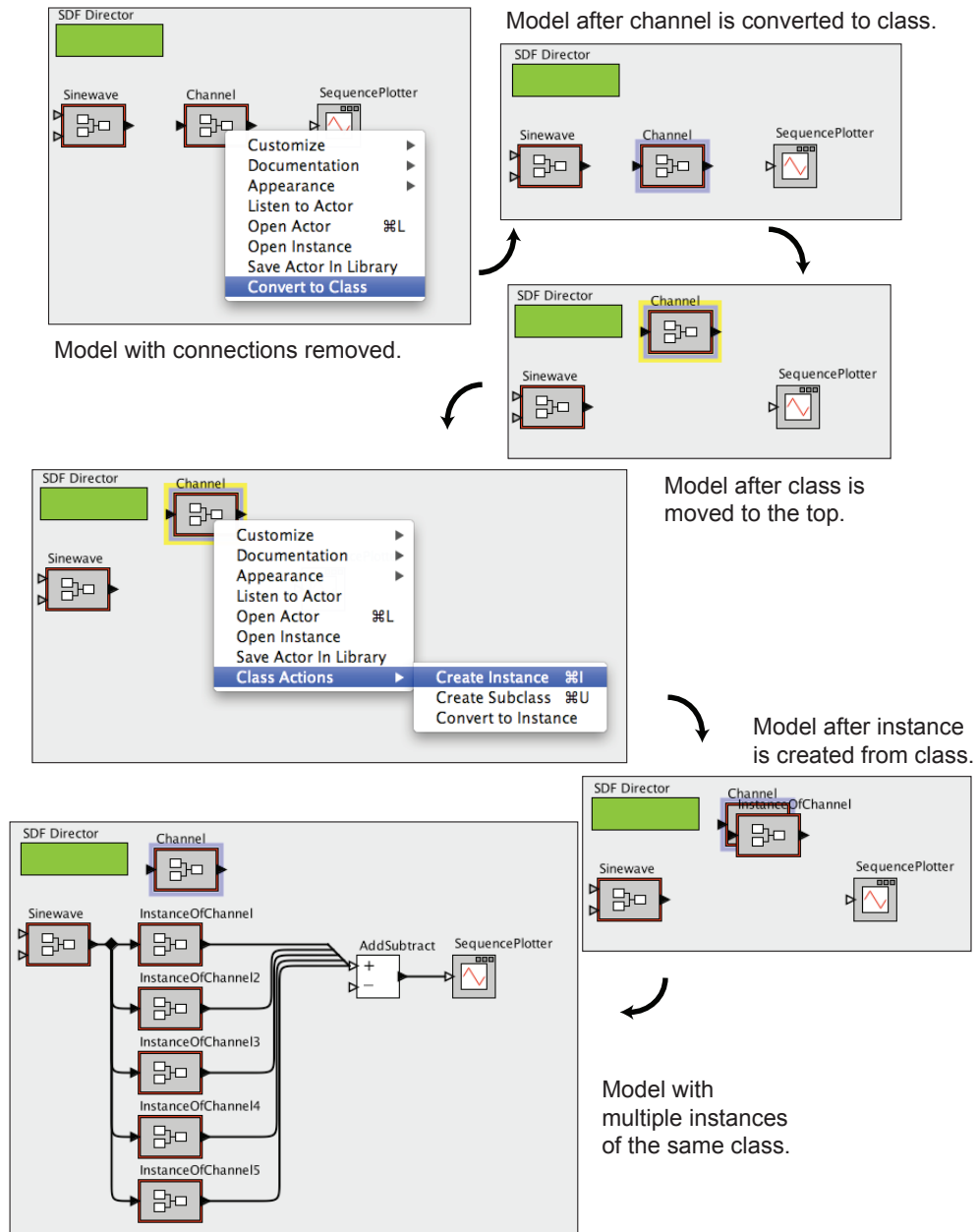


Figure 2.33: Creating and using a channel class.

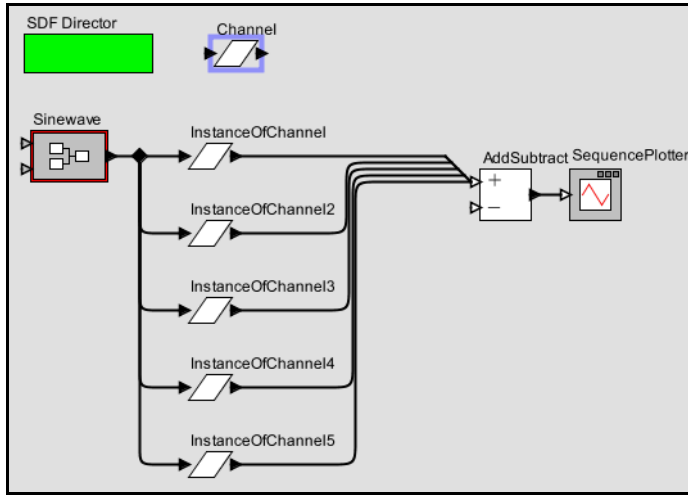


Figure 2.34: The model from Figure 2.33 with the icon changed for the class. Note that changes to the base class propagate to the instances.

2.6.2 Overriding Parameter Values in Instances

By default, all instances of Channel in Figure 2.34 have the same icon and the same parameter values. However, each instance can be customized by overriding these values. In Figure 2.35, for example, we have modified the custom icons so that each has a different color, and the fifth one has an extra graphical element. To do this, just right click on the icon of the instance and select `Edit Custom Icon`.

2.6.3 Subclassing and Inheritance

Suppose now that we wish to modify some of the channels to add interference in the form of another sinewave. A good way to do this is to create a subclass of the Channel class, as shown in Figure 2.36. A subclass is created by right clicking on the class icon and selecting `Create Subclass`. The resulting icon for the subclass appears right on top of the icon for the class, so it needs to be moved over, as shown in the figure.

Looking inside the subclass reveals that it contains all the elements of the class, but with their icons now surrounded by a pink halo. These elements are *inherited*. They cannot

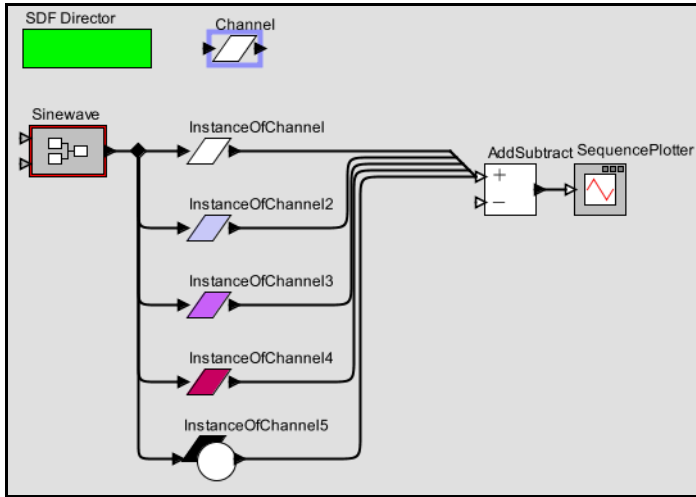


Figure 2.35: The model from Figure 2.34 with the icons of the instance changed to override parameter values in the class.

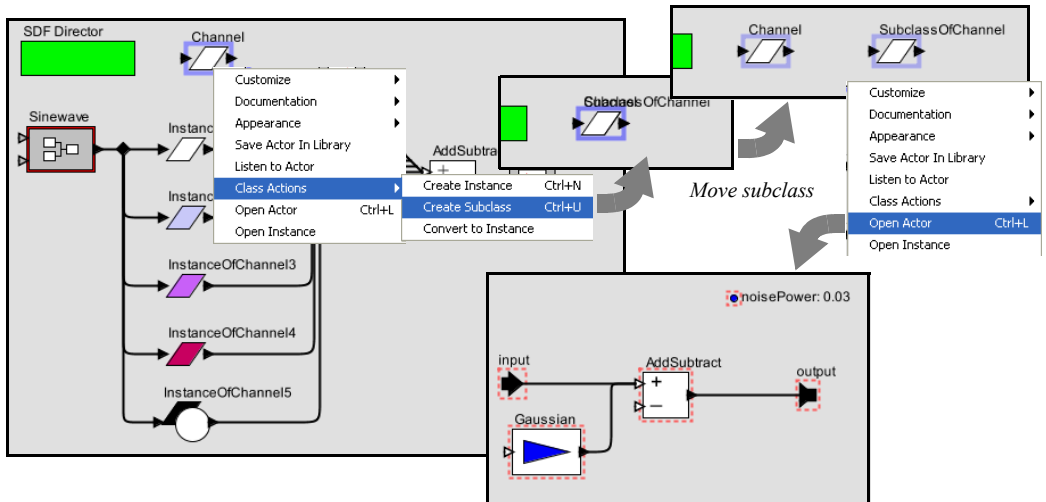


Figure 2.36: The model from figure 2.35 with a subclass of the Channel with no overrides (yet).

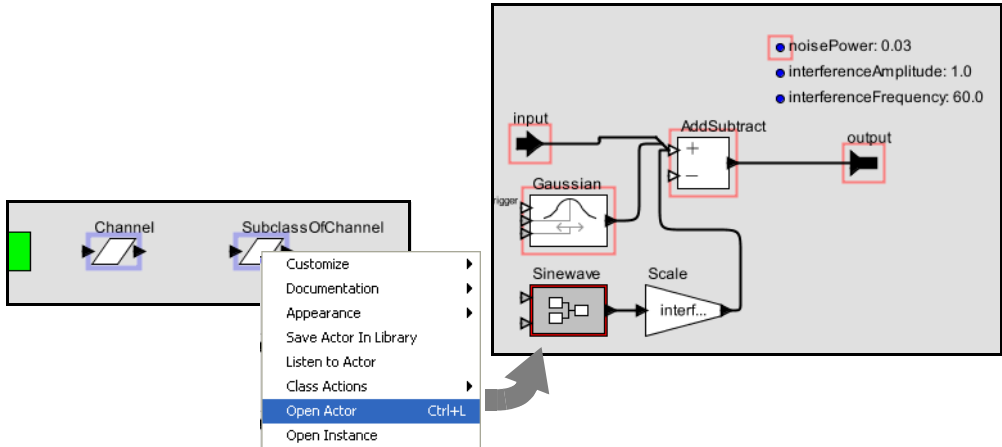


Figure 2.37: The subclass from Figure 2.36 with overrides that add sinusoidal interference.

be removed from the subclass (try to do so, and you will get an error message). You can, however, change their parameter values and add additional elements. Consider the design shown in Figure 2.37, which adds an additional pair of parameters named *interferenceAmplitude* and *interferenceFrequency* and an additional pair of actors implementing the interference. A model that replaces the last channel with an instance of the subclass is shown in Figure 2.38, along with a plot where you can see the sinusoidal interference.

An instance of a class may be created anywhere in a hierarchical model that is either in the same composite as the class or in a composite contained by that composite. To put an instance into a submodel, simply copy (or cut) an instance from the composite where the class is, and then paste that instance into the composite.

2.6.4 Sharing Classes Across Models

A class may be shared across multiple models by saving the class definition in its own file. We will illustrate how to do that with the *Channel* class. First, right click and invoke *Open Actor* on the *Channel* class, and then select *Save As* from the *File* menu. The dialog that appears is shown in Figure 2.39. The checkbox at the right, labeled *Save submodel only* is by default unchecked, and if left unchecked, what will be saved will be the entire

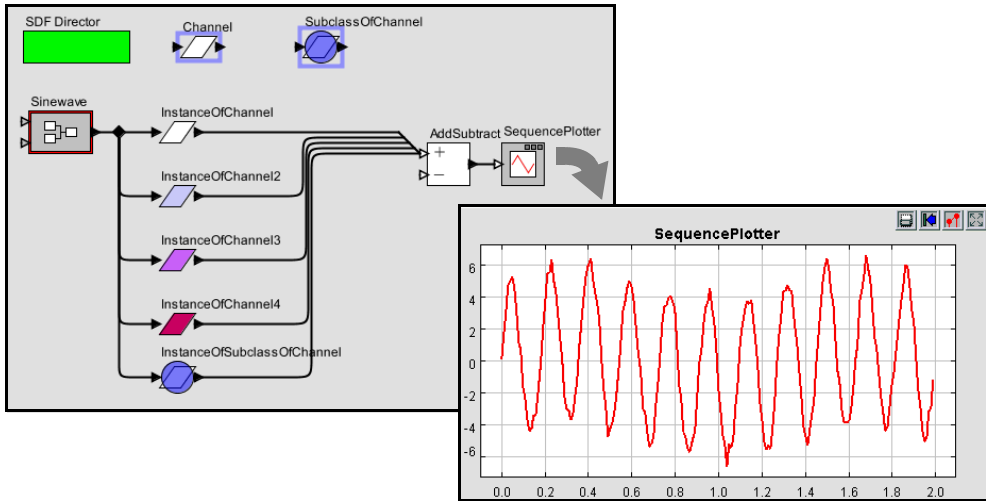


Figure 2.38: A model using the subclass from Figure 2.37 and a plot of an execution.

model.¹² In our case, we wish to save the Channel submodel only, so we must check the box.

A key issue is to decide where to save the file. As always with files, there is an issue that models that use a class defined in an external file have to be able to find that file. In general Ptolemy II searches for class definitions relative to the **classpath**, which is given by an environment variable called CLASSPATH. In principle, you can set this environment variable to include any particular directory that you would like searched. In practice, changing the CLASSPATH variable often causes problems with programs, so we recommend, when possible, simply storing the file in a directory within the Ptolemy II installation directory.¹³ In Figure 2.39, the Channel class is saved to a file called `Channel.xml` in the directory `$PTII/myActors`, where `$PTII` is the location of the Ptolemy II installation. This class definition can now be used in any model as follows. Open the model, and select `Instantiate Entity` in the Graph menu, as shown in Figure 2.40. Simply

¹²On some platforms, a separate dialog will appear with the `Save submodel only` check box.

¹³If you don't know where Ptolemy II is installed on your system, you can find out by invoking [File->New->Expression Evaluator] and typing `PTII` followed by Enter. Or, in a Graph editor, select [View->JVM Properties] and look for the `ptolemy.ptII.dir` property.

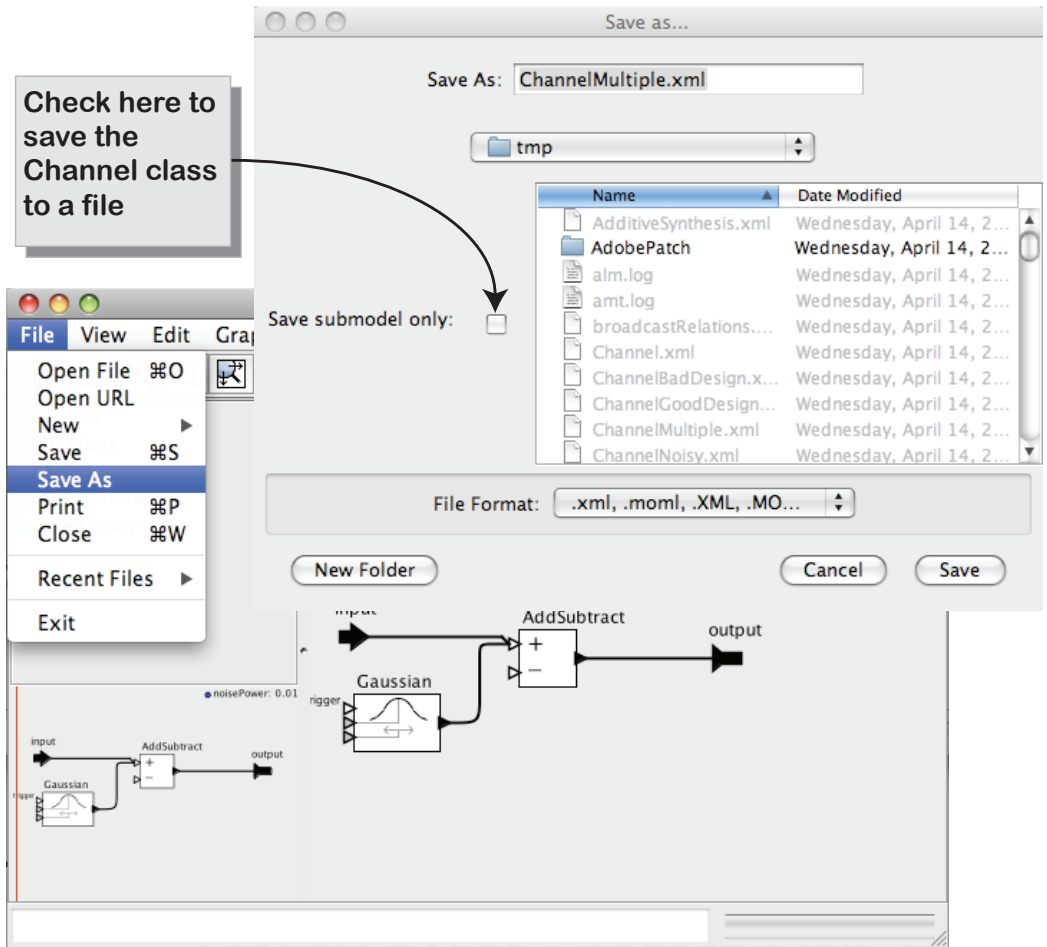


Figure 2.39: A class can be saved in a separate file to then be shared among multiple models. On some platforms, a separate dialog will appear with the `Save submodel only` check box.

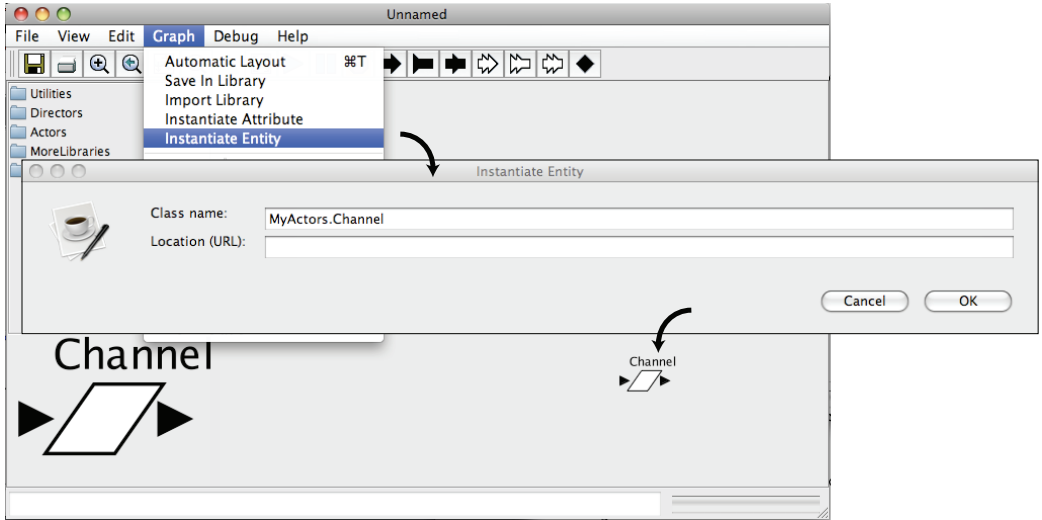


Figure 2.40: An instance of a class defined in a file can be created using `Instantiate Entity` in the `Graph` menu.

enter the fully qualified class name relative to the `$PTII` entry in the classpath, which in this case is “`myActors.Channel`”. Once you have an instance of the `Channel` class that is defined in its own file, you can add it to the **UserLibrary** that appears in the library browser to the left in Vergil windows, as shown in Figure 2.41. To do this, right click on the instance and select `Save Actor in Library`. As shown in the figure, this causes another window to open, which is actually the user library. The user library is a Ptolemy II model like any other, stored in an XML file. If you now save that library model, then the class instance will be available in the `UserLibrary` henceforth in any Vergil window.

One subtle point is that it would not accomplish the same objective if the class definition itself (vs. an instance of the class) were to be saved in the user library. If you were to do that, then the user library would provide a new class definition rather than an instance of the class when you drag from it.

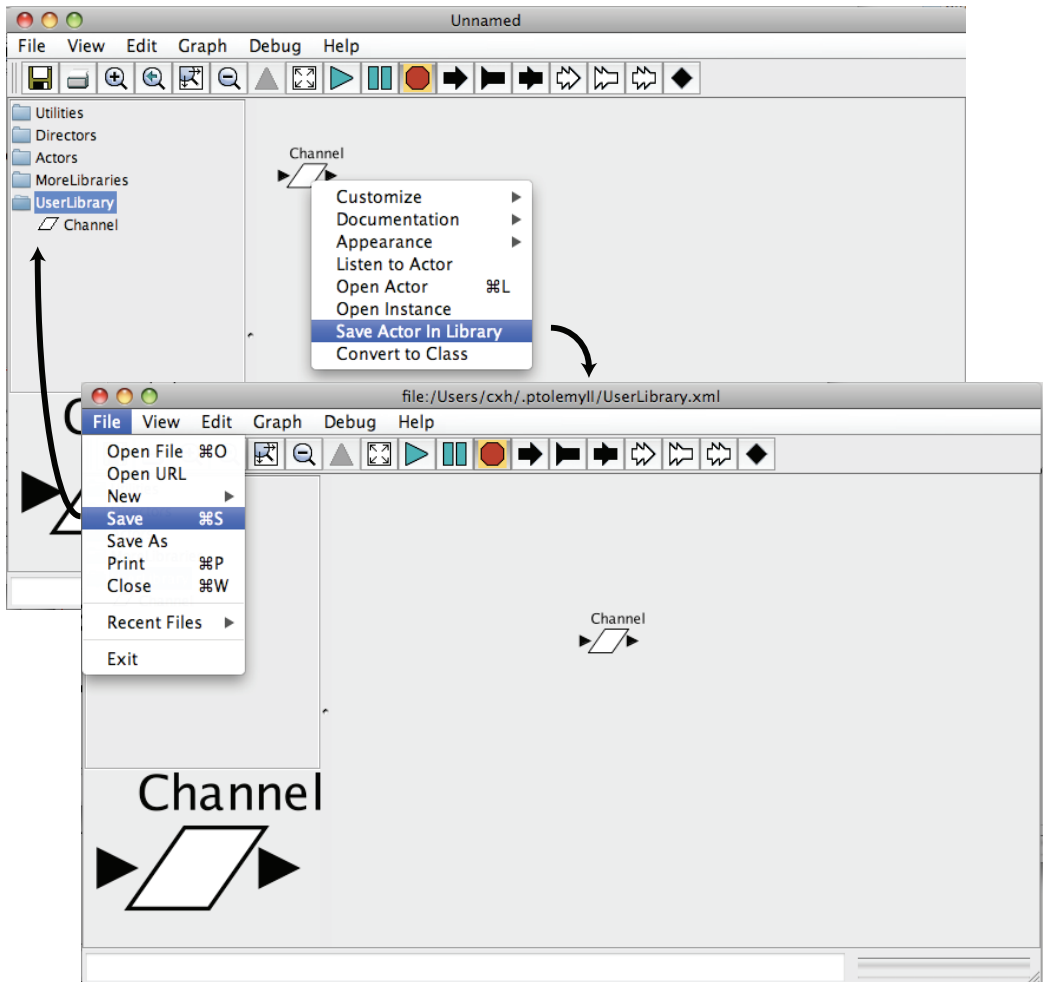


Figure 2.41: Instances of a class that is defined in its own file can be made available in the UserLibrary.

2.7 Higher-Order Components

Ptolemy II includes a number of **higher-order components**, which are actors that operate on the structure of the model rather than on their input data. This notion of higher-order components appeared in Ptolemy Classic and is described in [Lee and Parks \(1995\)](#), but the realization in Ptolemy II is more flexible than that in Ptolemy Classic. These higher-order components help significantly in building large designs where the model structure does not depend on the scale of the problem. In this section, we describe a few of these components, all of which are found in the `HigherOrderActors` library. The `ModalModel` actor is described in Chapter 4.

2.7.1 MultInstance Composite

Consider the model in Figure 2.34, which has five instances of the `Channel` class wired in parallel. This model has the unfortunate feature that the number of instances is hard-wired into the diagram. It is awkward, therefore, to change this number, and particularly awkward to create a larger number of instances. This problem is solved by the **MultInstanceComposite** actor¹⁴. A model equivalent to that of Figure 2.34 but using the `MultInstanceComposite` actor is shown in Figure 2.42. The `MultInstanceComposite` is a composite actor into which we have inserted a single instance of the `Channel` (this is inserted by creating an instance of the `Channel`, then copying and pasting it into the composite). `MultInstanceComposite` must be *opaque* (which means it has a director), so that its Actor interface methods (`preinitialize()`, ..., `wrapup()`) are invoked during model initialization.

The `MultInstanceComposite` actor has three parameters, *nInstances*, *instance* and *showClones* shown in Figure 2.43. The first of these specifies the number of instances to create. At run time, this actor replicates itself this number of times, connecting the inputs and outputs to the same sources and destinations as the first (prototype) instance. In Figure 2.42, notice that the input of the `MultInstanceComposite` is connected to a relation (the black diamond), and the output is connected directly to a multiport input of the `AddSubtract` actor. As a consequence, the multiple instances will be wired in a manner similar to Figure 2.34, where the same input value is broadcast to all instances, but distinct output values are supplied to the `AddSubtract` actor.

¹⁴The `MultInstanceComposite` actor was contributed to the Ptolemy II code base by Zoltan Kenczy and Sean Simmons, of Research In Motion Limited.

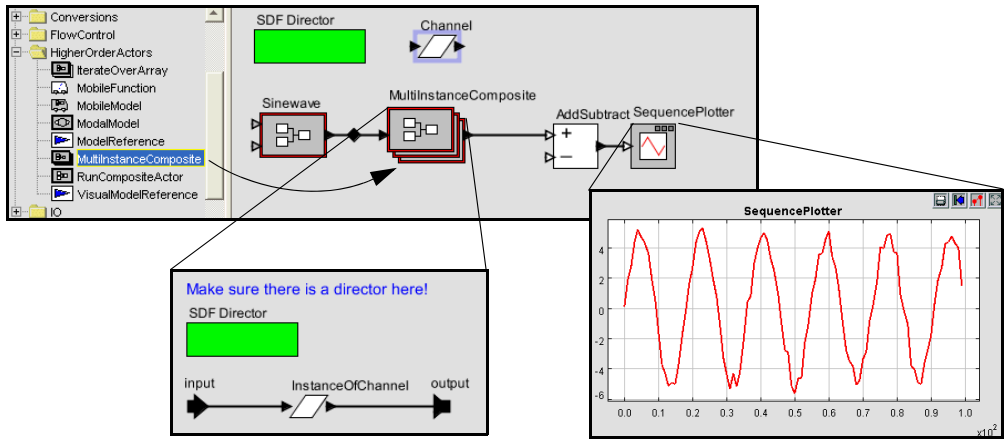


Figure 2.42: A model that is equivalent to that of Figure 2.34, but using a `MultInstanceComposite`, which permits the number of instances of the channel to change by simply changing one parameter value.

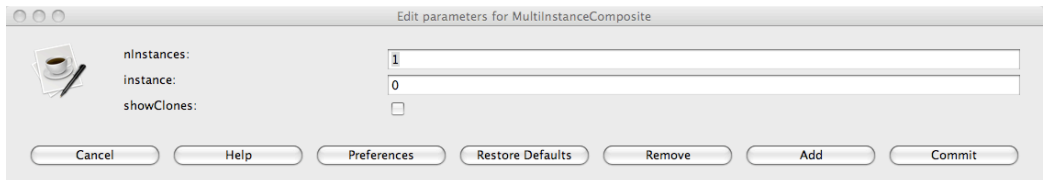


Figure 2.43: The first parameter of the `MultInstanceComposite` specifies the number of instances. The second parameter is available to the model builder to identify individual instances. The third parameter controls whether the instances are rendered on the screen.

The model of Figure 2.42 is better than that of Figure 2.34 because now we can change the number of instances by changing one parameter value. The instances can also be customized on a per-instance basis by expressing their parameter values in terms of the *instance* parameter of the [MultiInstanceComposite](#). Try, for example, making the *noise-Power* parameter of the `InstanceOfChannel` actor in Figure 2.42 depend on *instance*. E.g., set it to `instance * 0.1` and then set *nInstances* to 1. You will see a clean sine wave when you run the model.

2.7.2 IterateOverArray

The implementation of the `Channel` class, which is shown in Figure 2.39, happens to not have any state, meaning that an invocation of the `Channel` model does not depend on data calculated in a previous invocation. As a consequence, it is not really necessary to use *n* distinct instances of the `Channel` class to realize a diversity communication system. A single instance could be invoked *n* times on *n* copies of the data. We can do this using the **IterateOverArray** higher-order actor.

The `IterateOverArray` actor can be used in a manner similar to how we used the `MultiInstanceComposite` in the previous section. That is, we can populate it with an instance of the `Channel` class, similar to Figure 2.42. Just like the `MultiInstanceComposite`, the `IterateOverArray` actor requires a director inside. An example is shown in Figure 2.44. Notice that in the top-level model, instead of using a relation to broadcast the input to multiple instances of the channel, we create an array with multiple copies of the channel input. This is done using a combination of the **Repeat** actor (found in the `FlowControl->SequenceControl` sublibrary) and the **SequenceToArray** actor (found in the `Array` library). The **Repeat** actor has a single parameter, *numberOfTimes*, which in Figure 2.44 we have set equal to the value of the *diversity* parameter that we have added to the model. The **SequenceToArray** actor has a parameter *arrayLength* that we have also set equal to *diversity* (this parameter, interestingly, can also be set via the *arrayLength* port, which is filled in gray to indicate that it is both parameter and a port). The output is sent to an **ArrayAverage** actor, also found in the `Array` library.

The execution of the model in Figure 2.44 is similar to that of the model in Figure 2.42, except that the scale of the output is different, reflecting the fact that the output is an average rather than a sum.

The `IterateOverArray` actor also supports dropping into it an actor by dropping the actor onto the `IterateOverArray` icon. The actor can be either an atomic library actor or a

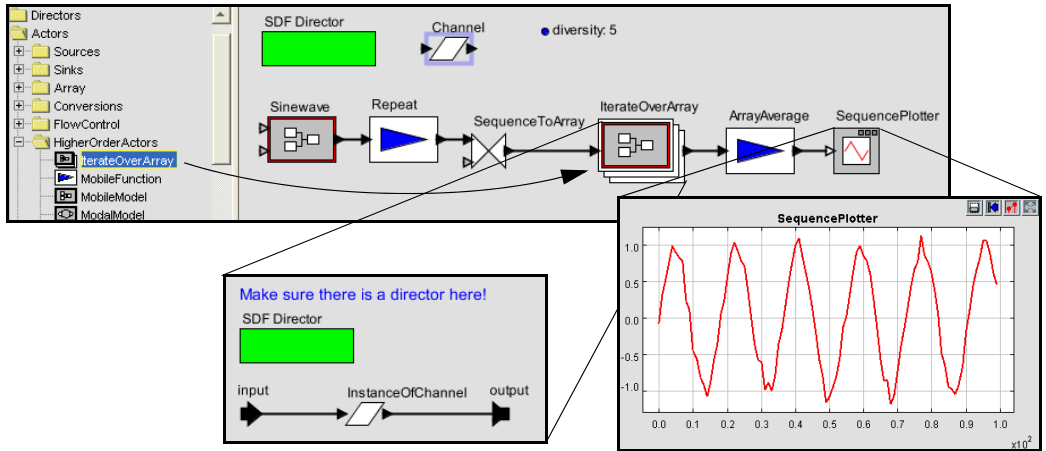


Figure 2.44: The `IterateOverArray` actor can be used to accomplish the same diversity channel model as in Figure 2.42, but without creating multiple instances of the channel model. This works because the channel model has no state.

composite actor (although if it is composite actor, it is required to have a director). This mechanism is illustrated in Figure 2.45. When an actor is dragged from the library, and then dragged over the `IterateOverArray` actor, the icon acquires a white halo, suggesting that if the actor is dropped, it will be dropped into the actor under the cursor, rather than onto the model containing that actor. When you look inside the `IterateOverArray` actor after doing this, you will see the class definition. Add an `SDFDirector` to it before executing it.

2.7.3 Mobile Code

A pair of (still experimental) actors in Ptolemy II support mobile code in two forms. The **MobileFunction** actor accepts a function in the expression language (see Appendix ??) at one input port and applies that function to data that arrives at the other input port. The **MobileModel** actor accepts a MoML description of a Ptolemy II model at an input port and then executes that model, streaming data from the other input port through it.

A use of the **MobileFunction** actor is shown in Figure 2.46. In that model, two functions are provided to the **MobileFunction** in an alternating fashion, one that computes x^2 and

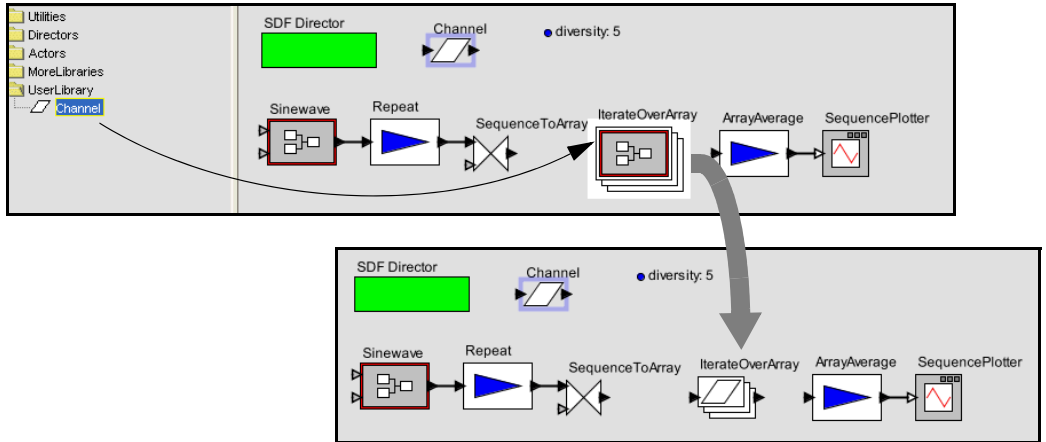


Figure 2.45: The `IterateOverArray` actor supports dropping an actor onto it. When you do this, it transforms to mimic the icon of the actor you dropped onto it, as shown. Here we are using the `Channel` class that we saved to the `UserLibrary` as shown in Figure 2.41.

the other that computes 2^x . These two functions are provided by two instances of the `Const` actor, found in the `Sources->GenericSources` sublibrary. The functions are interleaved by the `Commutator` actor, from `FlowControl->Aggregators` sublibrary.

2.7.4 Lifecycle Management Actors

A few actors in the `HigherOrderActors` library provide in a single firing the entire execution of another Ptolemy II model. The **`RunCompositeActor`** actor executes the contained model. The **`ModelReference`** actor executes a model that is defined elsewhere in its own file or URL. The **`VisualModelReference`** actor opens a Vergil view of a referenced model when it executes a referenced model. These actors generally associate ports (that the user of the actor creates) with parameters of the referenced or contained model. They can be used, for example, to create models that repeatedly run other models with varying parameter values. See the documentation of the actors and the demonstrations in the tour for more details.

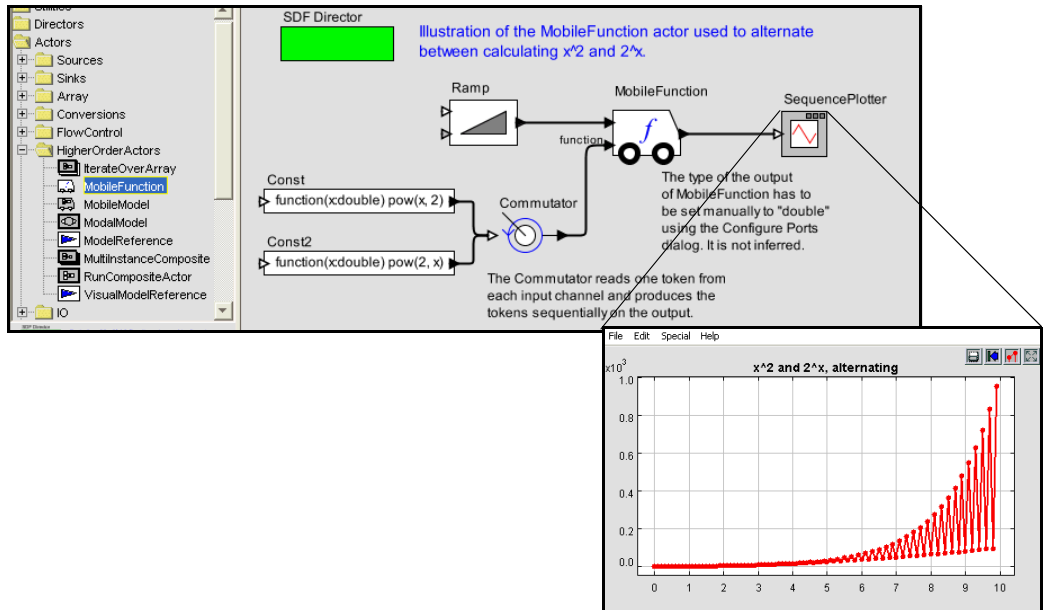


Figure 2.46: The MobileFunction actor accepts a function definition at one port and applies it to data that arrives at the other port.

2.8 Conclusion

This chapter has introduced the visual interface to Ptolemy II called Vergil, which support graphical construction of models, and along the way, has introduced a number of capabilities of the underlying system.

Discrete Event Models

Edward A. Lee, Jie Liu, Lukito Muliadi, and Haiyang Zheng

Contents

	<i>Sidebar: Clock Actors</i>	74
	<i>Sidebar: Time Measurement</i>	75
3.1	Time in DE Models	76
3.1.1	Model Time vs. Real Time	76
3.1.2	Simultaneous Events	77
3.1.3	Synchronizing Events	79
3.2	Scheduling	80
	<i>Sidebar: Time Delays</i>	80
	<i>Sidebar: Samplers and Synchronizers</i>	82
	<i>Probing Further: DE Semantics</i>	83
3.2.1	Priorities	84
3.2.2	Feedback Loops	86
3.2.3	Zeno Models	88
3.2.4	Multithreaded Execution	90
3.3	Using DE with Dataflow	94
3.4	Wireless and Sensor Network Systems	95
	<i>Sidebar: Queue and Server Actors</i>	96
3.5	Summary	97
	Exercises	99

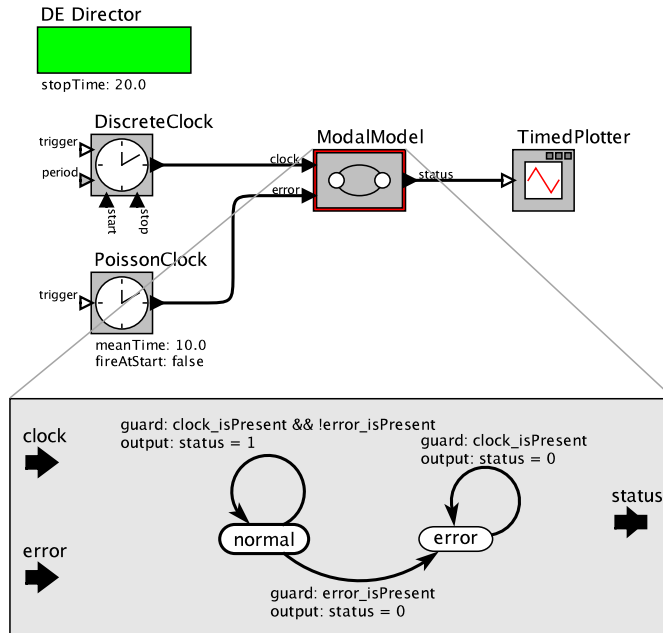


Figure 3.1: Simple example of a DE model.

The **discrete event (DE)** domain models timed, discrete interactions between concurrent actors. Each interaction is called an **event**, and is conceptually understood to be an instantaneous message sent from one actor to another. The key idea in DE is that each actor reacts to its input events in temporal order. That is, each time an actor fires, it will see input events that occur later in time than those it saw in previous firings. The model of time is a key part of this [model of computation](#).

Example 3.1: A simple example is shown in Figure 3.1. This example illustrates a common application of DE, modeling faults or error conditions that occur at random times. In this example, the [DiscreteClock](#) actor produces events that are regularly spaced in time (representing “normal” behavior), and the [PoissonClock](#) produces events that are irregularly spaced in time (representing the occurrence of an error). See the sidebar on page 74 for a description of these clock actors.

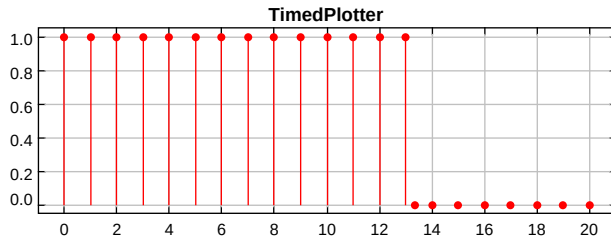


Figure 3.2: Sample output from the model in Figure 3.1.

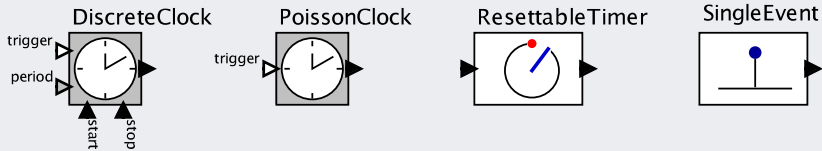
The `ModalModel` actor has behavior given by the `FSM` shown in the figure. The initial state is `normal`. While in this state, whenever an input event is present on the `error` input port, the `ModalModel` outputs a token with value 0 and transitions to state `error`. Otherwise, if an input is present on the `clock` input port, then it outputs a token with value 1. This represents a system that produces events with value 1 at regular time intervals, until an error occurs, after which it produces events with value 0. It is essential that the `ModalModel` actor react to inputs in temporal order, because once it has made the transition to state `error`, it can never go back.

The output of a typical execution, plotted by the `TimedPlotter` actor, is shown in Figure 3.2 (see Appendix A for information on the `TimedPlotter`). In the plot, we can see that the `PoissonClock` produced an error event at approximately time 13.3. The *meanTime* of the `PoissonClock` actor, which specifies the expected time between events, is set to 10.0 in the model, so the actual time of the error in this run, 13.3, was reasonably close to the expected time.

In this chapter, we examine the mechanics and the subtleties of such DE models. We begin with a discussion of the model of time, and then show how DE provides a *determinate MoC*. We discuss the subtleties that arise when events are simultaneous and when models include feedback loops.

Sidebar: Clock Actors

Clock actors generate timed events. Four such actors are shown below:



These actors are found in the `Sources->TimedSources` library, except for **ResettableTimer**, which is in `DomainSpecific->DiscreteEvent`.

The **DiscreteClock** is a very versatile event generator. Its simplest use is to generate a sequence of events that are regularly spaced in time. Its default parameters have it produce tokens with value 1 (an *int*) spaced one time unit apart starting at the start time of the execution (typically time zero). This default setting is used to produce the plot in Figure 3.2. But **DiscreteClock** can generate much more complex event patterns that cycle through a finite sequence of values and have periodically repeating time offsets from the start of each period. It can also generate a one-time, finite sequence of events, instead of a periodic pattern, by setting the *period* to *Infinity*. These events can be arbitrarily placed in time. See the actor documentation for details.

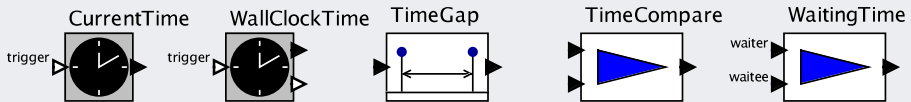
The **PoissonClock**, in contrast, produces events at random times. The time between events is given by independent and identically distributed (**IID**) exponential random variables. An output signal like this is called a **Poisson process**. Like the **DiscreteClock** actor, the **PoissonClock** actor can cycle through a finite sequence of values, or it can produce events with the same value each time.

The **ResettableTimer** actor produces an output after the time specified on the input event has elapsed (in *model time*, not *real time*). That is, an output event is produced at model time equal to the model time of the input plus the value of the input. If the input value is zero, then the output will be produced at the next *microstep*. This actor has features enabling a pending event to be canceled, and also enabling a new input event to preempt a previously scheduled output. See the actor documentation for details.

The **SingleEvent** actor is not really needed, since **DiscreteClock** is capable of producing single events. Nonetheless, it is sometimes useful because it visually emphasizes in a model that only a single event is produced.

Sidebar: Time Measurement

The following actors provide access to the [model time](#) of events:



The **CurrentTime** actor, which is found in `Sources->TimedSources`, observes the model time of events. It produces an output event whenever an input event arrives, where the value of the output event is the model time of the input event.^a The output event has the same [time stamp](#) as the input event, so the actor's reaction is conceptually instantaneous.

The **WallClockTime** actor, which is found in `RealTime`, observes the real time of events. Specifically, when it fires in reaction to an input event, it outputs a *double* representing the amount of [real time](#) that has passed (in seconds) since the actor was initialized. Since this value depends on arbitrary scheduling decisions that the DE director makes, it is [nondeterminate](#). Nonetheless, it can be useful for making measurements of performance. The inputs that arrive on the *trigger* input are also passed through to an output port, a feature that makes it somewhat easier to control the scheduling in some domains.

The other actors are in the `DomainSpecific->DiscreteEvent` library. These are used to measure model-time differences between events. **TimeGap** measures the difference between successive events in a signal. **TimeCompare** measures the time gap between events in two signals. It is triggered by the arrival of an event at either input, and sends to its output the difference between the model of time of the newly arrived event and the model time of the most recently received event on the other port. **WaitingTime** is an asymmetric version. When an event arrives on the *waiter* port, the actor begins waiting for an event to arrive on the *waitee* port. When the first such event arrives, the actor sends to the output the difference between the model times of the two events.

^aThe time value on the output of the **CurrentTime** actor is represented as a *double*, so it may not be an exact representation of the internal model time, which, as explained in the sidebar on page 17, has a fixed resolution that does not change as the value of time increases. The resolution of a *double*, by contrast, decreases as the magnitude increases.

3.1 Time in DE Models

In DE models, the connections between actors carry **signals** that consist of events placed on a time line. Each event has both a value and a **time stamp**, where the time stamp defines a global ordering between events. This is different from **dataflow**, where a signal consists of a sequence of tokens, and there is no time significance in the signal. A time stamp is a **superdense time** value, consisting of a **model time** and a **microstep**.

3.1.1 Model Time vs. Real Time

A DE model executes chronologically, processing the oldest events (those with earlier time stamps) first. Time advances as events are processed. There is potential confusion, however, between **model time**, the time that evolves in the model, and **real time**, the time that elapses in the real world while the model executes (also called **wall-clock time**). Model time may advance more rapidly than real time or more slowly. The DE director has a parameter, *synchronizeToRealTime*, that, when set to `true`, synchronizes the two notions of time, to the extent possible. It does this by delaying execution of the model, if necessary, allowing real time to catch up with model time. When this parameter is set to `true`, model-time values are interpreted as being in units of seconds, but otherwise, no specific units are imposed in model time. The units are arbitrary.

Example 3.2: Consider the DE model shown in Figure 3.3. This model includes a **PoissonClock** actor, a **CurrentTime** actor, and a **WallClockTime** actor (see sidebars on pages 74 and 75). The plot shows that wall-clock time and model time are indeed quite different; in this case, wall-clock time barely advances during execution, whereas model time advances quite far (to about 9 seconds). Since the horizontal axis in this plot is model time, the model time plot increases linearly. If you set the *synchronizeToRealTime* parameter of the director to `true`, then you will find that the two plots coincide almost perfectly.

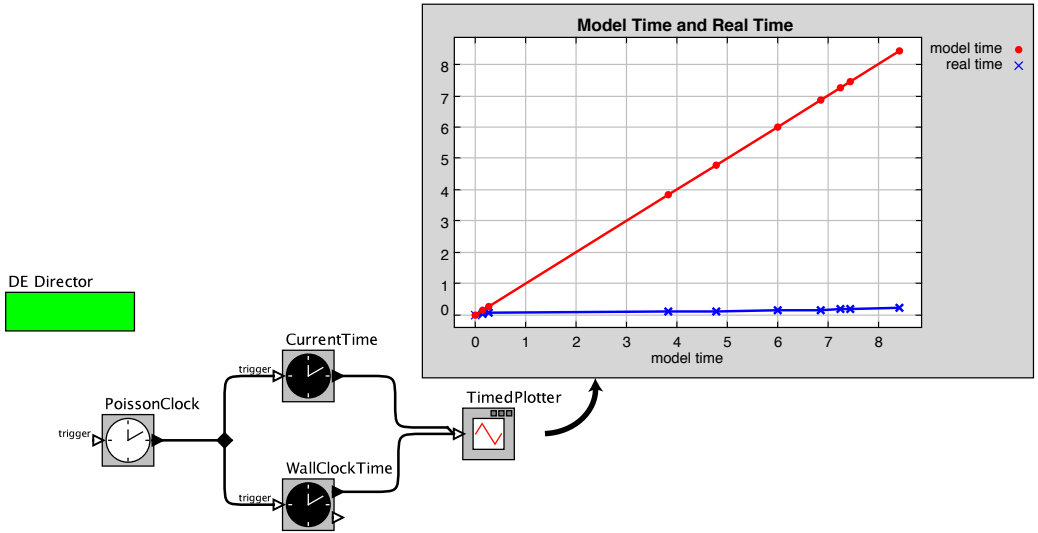


Figure 3.3: Model time vs. real time (wall clock time).

3.1.2 Simultaneous Events

A significant subtlety in using the DE domain is in how **simultaneous** events are handled. Strongly simultaneous events have the same time stamp (model time and microstep), whereas weakly simultaneous events have the same model time, but not necessarily the same microstep. We have stated that events are processed in chronological order, but if two events have the same time stamp, then there is some ambiguity. Which one should be processed first?

Example 3.3: Consider the model shown in Figure 3.4, which produces a histogram of the interarrival times of events from a *PoissonClock* actor. In this model, we calculate the difference between the current event time and the previous event time, resulting in the plot that is shown in the figure. The *Previous* actor is a **zero-delay actor**, meaning that it produces an output with the same time stamp as the input (except on the first firing, where in this case it produces no output, see sidebar on 80). Thus, when the *PoissonClock* actor produces an output, there will be two (strongly) simultaneous events, one at the input to the *plus* port of the *AddSub*-

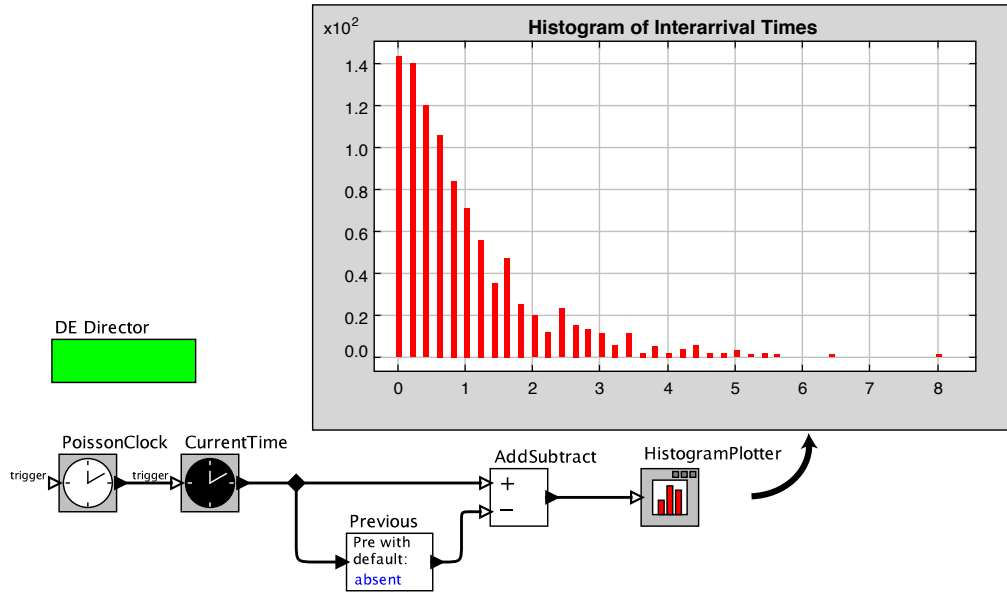


Figure 3.4: Histogram of interarrival times, illustrating handling of simultaneous events. There is a subtle bug in this model, corrected in Figures 3.5 and 3.6.

tract actor, and one at the input of the Previous actor. Should the director fire the AddSubtract actor or the Previous actor first? Either seems OK if it is to respect chronological order, but it seems intuitive that the Previous actor should be fired first.

It is helpful to know how the AddSubtract actor works. When it fires, it will add all (strongly simultaneous) available tokens on the *plus* port, and subtract all (strongly simultaneous) available tokens on the *minus* port. If there is a token at the *plus* port, but none at the *minus* port, then the output will equal the token at the *plus* port. Conversely, if there is a token at the *minus* port, but none at the *plus* port, then the output will equal the *negative* of the token at the *minus* port.

This example shows that to ensure **determinism**, it is essential that the order of firing of actors be well defined. In the example, it is essential that the order of firing of AddSubtract

and Previous be well defined. Indeed, in the DE domain, it is. Given strongly simultaneous events at the inputs of AddSubtract and Previous, the director will always fire the Previous actor first. The order is governed by a **topological sort** of the actors in the model. A topological sort is simply a list of the actors in data-precedence order.

Example 3.4: For the model in Figure 3.4, there is only one valid topological sort:

PoissonClock, CurrentTime, Previous, AddSubtract, HistogramPlotter

In this list, AddSubtract is after Previous. So the when they have strongly simultaneous events, the DE director fires Previous first.

Thus, the DE director, by analyzing the structure of the model, delivers determinate behavior, where actors that produce data are fired before actors that consume their results, even in the presence of simultaneous events.

3.1.3 Synchronizing Events

There is a subtle bug in the model of Figure 3.4. Specifically, the very first event produced by the CurrentTime actor has value equal to the time stamp of the first event produced by the PoissonClock (which in this case is 0.0 because the PoissonClock by default produces an event at the start time of an execution). The Previous actor, however, will not produce any output at this time, because, by default, its output is absent upon arrival of its first input (see sidebar on page 80). Hence, the first output of the AddSubtract will have value 0.0, which is not, in fact, an inter-arrival time! Thus, the plotted histogram includes a spurious value 0.0.

To fix this problem, we would like to ensure that the AddSubtract actor receives exactly one event on each input port, and only receives events when there is one available for each port. We can accomplish this using the **Sampler** actor, as shown in Figure 3.5 (see sidebar on page 82). This actor produces an output event only if there is an input event on its *trigger* input (the bottom port). Upon receiving a trigger input, it passes to its output whatever (strongly simultaneous) events are available on its input port. So in this example, the first output from the CurrentTime actor will be discarded, because, at that time, there is no event on the *trigger* input.

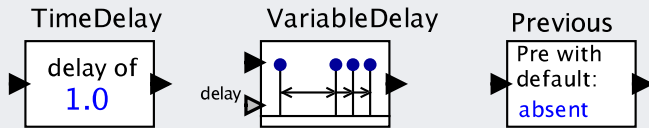
There are a number of other ways to accomplish the same goal. For example, the [Synchronizer](#) actor described in the sidebar on page 82 can also be used in place of the [Sampler](#). More directly, the [TimeGap](#) actor can also be used to solve this problem (see sidebar on page 75), as shown in Figure 3.6. It combines the functionality of the [Previous](#), [Sampler](#), and [AddSubtract](#) actors.

3.2 Scheduling

The main task of the DE director is to fire actors in time-stamp order. The DE director maintains an event queue that is sorted by time stamp, primarily by [model time](#) and second-

Sidebar: Time Delays

The following actors provide mechanisms for changing the [time stamps](#) of events:



The most useful of these is **TimeDelay**, which increments the [model time](#) of the input event by a fixed amount. The amount of the increment is displayed in the icon and defaults to 1.0. This actor, therefore, delays an event (in model time, not in [real time](#)). The **VariableDelay** actor is similar, except that the amount of delay can be optionally provided as an event on the input port labeled *delay*. Any input events that arrive simultaneously with or after an event on the *delay* port will be delayed (in model time) by the amount given by the value of the *delay* input.

Occasionally, it is useful to set a model-time delay to zero. This has the effect of incrementing the [microstep](#) of an event rather than its model time. This can be useful particularly in feedback loops, as discussed in the main text.

The **Previous** actor, upon receiving an input event, produces an output event with the same time stamp as the input event, but with the value of the previously received input event. When it receives the first event, there was no previous event, so it outputs instead a default value, if one is given, or outputs nothing (absent output), if no default value is given. This actor, therefore, delays each input event until the arrival of the next input event.

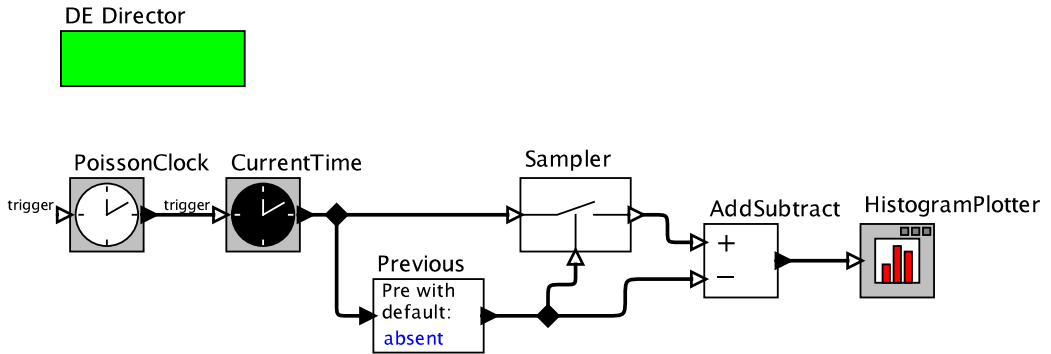


Figure 3.5: Histogram of interarrival times, correcting the subtle bug in Figure 3.4 using the Sampler actor.

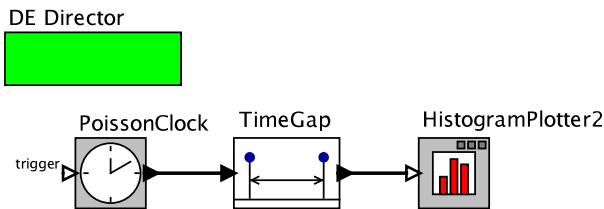


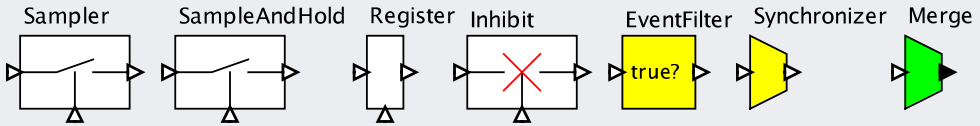
Figure 3.6: Histogram of interarrival times, correcting the subtle bug in Figure 3.4 using the TimeGap actor.

daily by [microstep](#). The DE director governs the execution by selecting the earliest event in the event queue and making its time stamp the **current model time**. It then determines what actor that event is destined for, and finds all other events in the event queue with the same time stamp that are destined to the same actor. It makes those events available to the actor on its input ports and then fires the actor.

The DE director provides a strong guarantee that ensures determinism; specifically, no two firings of a given actor occur at the same current model time. This means that the director has to guarantee that it has made available *all* events with time stamp equal to the current time before it fires the actor. It cannot later provide an additional event with the same time stamp.

Sidebar: Samplers and Synchronizers

The following actors provide mechanisms for synchronizing events:



The **Sampler** and **Synchronizer** are in `FlowControl->Aggregators`, whereas the rest are in `DomainSpecific->DiscreteEvent`. When it receives an event on the *trigger* port (the one at the bottom of the icon), the **Sampler** actor copies whatever (strongly simultaneous) input events are available on the input port to its output port. The input is a [multiport](#), and hence can receive any number of input signals. The input events are copied to the corresponding output [channel](#).

The **SampleAndHold** actor is similar, except that upon receiving a trigger, it copies the *most recent* input event (which may or may not be simultaneous with the trigger event) to the corresponding output channel. It provides an *initialValue* parameter, which optionally provides the output value to produce if no input has arrived upon triggering.

The **Register** actor is similar to **SampleAndHold**, except that upon receiving a trigger, it copies the most recent *strictly earlier* input event to the corresponding output channel. This actor, unlike **Sampler** or **SampleAndHold**, always introduces delay, and hence is also similar to the delay actors described in the sidebar on page 80. The delay can be as small as one [microstep](#), in which case the input and output will be weakly [simultaneous](#). Because it always introduces delay, this actor is useful in [feedback](#) loops (see Section 3.2.2).

The **Inhibit** actor is the converse of the **Sampler**. It copies all inputs the output *unless* it receives a *trigger* input. The **EventFilter**, which accepts only *boolean* inputs, copies only true-valued inputs to the output. The **Synchronizer** actor copies inputs to outputs only if every input channel has a strongly simultaneous event.

The **Merge** actor merges events on any number of input channels into a single signal in time-stamp order. If it receives strongly simultaneous events, then it either discards all but the first one (if the *discard* parameter is `true`), or it outputs the events with increasing microsteps (if the *discard* parameter is `false`).

As discussed above, the DE director performs a **topological sort** of the actors. Once this sort has been performed, each actor can be assigned a **level**, which is the length of the longest upstream path of actors that may produce events on which it depends that have the same time stamp.

Example 3.5: For example, the actors in Figure 3.5 have the following levels:

- PoissonClock: 0
- CurrentTime: 1

Probing Further: DE Semantics

Discrete-event modeling based on time stamps has been around for a long time. One of the earliest complete discrete-event formalisms is called DEVS, for Discrete Event System Specification (Zeigler, 1976). Many subtly different variants have appeared over time (see for example Ramadge and Wonham (1989), Cassandras (1993), Baccelli et al. (1992), and Zeigler et al. (2000)). Variants of DE also form the foundation for the widely used hardware description languages VHDL, Verilog, and SystemC, and a number of network simulation tools such as OPNET Modeler (from OPNET Technologies, Inc.), ns-2 (a collaborative open-source effort led by the Virtual Internetwork Testbed Project VINT), and ns-3 (<http://www.nsnam.org/>). There has also been extensive work on simulation strategies for DE models. A particularly interesting challenge is exploiting parallel hardware (because of the strong ordering imposed by time stamps) (Chandy and Misra, 1979; Jefferson, 1985; Fujimoto, 2000). The variant of DE described in this chapter, and specifically with its use of **superdense time** appears to be unique.

The formal **semantics** of DE is a fascinating and deep topic. The oldest approaches describe execution of DE models in terms of state machines (Zeigler, 1976). More recent approaches define a metric space (Bryant, 1985) in which actors become contraction maps and the meaning of a model becomes a fixed point of these maps (Reed and Roscoe, 1988; Yates, 1993; Lee, 1999; Liu et al., 2006). DE semantics have also been given as a generalization of the semantics of **synchronous/reactive** languages (Lee and Zheng, 2007), and as fixed points of monotonic functions over a complete partial order (CPO) (Broy, 1983; Liu and Lee, 2008). These latter approaches are denotational (Baier and Majster-Cederbaum, 1994), whereas the state machine approaches have a more operational flavor.

- Previous: 2
- Sampler: 3
- AddSubtract: 4
- HistogramPlotter: 5

When two events with the same time stamp are inserted into the event queue, if one is destined to an actor with a lower level, then that event will appear earlier in the event queue. This ensures, for example, that in Figure 3.5, Previous will fire before Sampler, and Sampler will fire before AddSubtract. Thus, when AddSubtract fires, it is assured of seeing all input events at the current model time.

3.2.1 Priorities

The DE director sorts events primarily by model time, secondarily by microstep, and, as we have just seen, next by level. But it is still possible to have events that have the same time stamp and level.

Example 3.6: Consider the model shown in Figure 3.7. Here, the two Ramp actors have the same level (1) and the two FileWriter actors have the same level (2). When the clock produces an event, there will be two events in the event queue with the same time stamp and level, one destined to Ramp and one destined to Ramp2. Since these two actors do not communicate, it does not matter in which order they are fired. However, the two FileWriter actors might be writing to the same file (or to standard out). In this case, it does matter in which order the actors fire, but the order cannot be determined by time stamp and level alone.

The previous example illustrates a rather peculiar case. In effect, there are two actors that affect each other even though there is no communication between them in the model. They are communicating **under the table**,¹ in a manner invisible to the DE director.

¹The term “under the table” comes from sneaky demonstrations of “wireless” communication where in fact communication is being accomplished using a wire that is passed under the table, hidden from view.

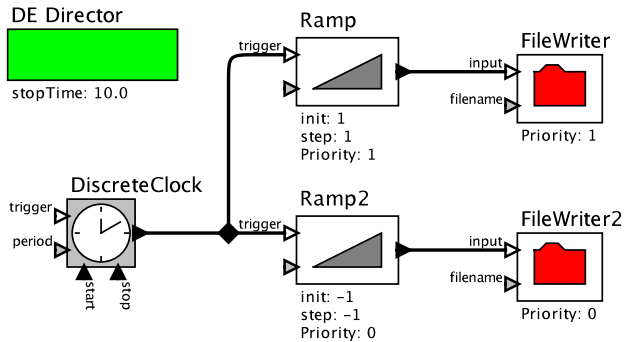


Figure 3.7: Although it is rarely necessary, occasionally it is useful to set priorities of actors in DE. This only has an effect when the firing order is otherwise not determined by time stamps or by data precedences.

When there is such communication, the model builder may wish to exercise some control over the order of execution.

In the `Utilities->Parameters` library you can find a *Priority* parameter that can be dragged and dropped onto actors. The value can then be set for each actor independently by double clicking on the actor. A lower value is interpreted as a higher priority. When events have the same time stamp and the same level, the DE director then consults the priority of the destination actors. If the actors have priorities, then it will put those events destined to actors whose priorities are higher (*Priority* has a lower value) earlier in the event queue.

Example 3.7: For the example in Figure 3.7, Ramp2 and FileWriter2 have *Priority* zero, so they will fire before Ramp and FileWriter, which have *Priority* one. Without these priorities, the order would be *nondeterminate*.

It is rare in DE modeling to need to use priorities. This is fortunate because they are really rather awkward. Their value has global meaning, throughout the model, and hence this mechanism is not very modular. It is only needed when actors communicate under the table.

3.2.2 Feedback Loops

There remains one key subtlety. If the model has a directed loop (called a **feedback loop**), then a topological sort is not possible. In the DE domain, every feedback loop is required to have at least one actor in it that introduces a time delay, such as [TimeDelay](#), [VariableDelay](#), or [Register](#) (see sidebars on pages 80 and 82).

Example 3.8: Consider for example the model shown in Figure 3.8. That model has a [DiscreteClock](#) actor, which is set to produce events every 1.0 time units. Those events trigger the [Ramp](#) actor, which produces outputs that start at 0 and increase by 1 on each firing. In this model, the output of the Ramp goes into an [AddSubtract](#) actor, which subtracts from the Ramp output its own prior output delayed by one time unit. The result is shown in the plot in the figure.

So that models with feedback are determinate, the DE director assigns delay actors level zero, but the delay actors are required to not read their inputs when they fire. Instead, they must defer reading the input until the [postfire](#) stage, which occurs after all actors that are going to fire at the current time have fired.

Example 3.9: In the example in Figure 3.8, [TimeDelay](#) has level 0, whereas [AddSubtract](#) has level 1, so [TimeDelay](#) will fire before [AddSubtract](#) at any given time stamp when it is scheduled to produce an output. However, it does not read its input until after the [AddSubtract](#) has fired in response to its own output event. It reads its input in the [postfire](#) phase, at which time it simply records input and requests a new firing at the current time plus its time delay (1.0, in this case).

Occasionally, you will need to put a [TimeDelay](#) actor in a feedback loop with a delay of 0.0. This has the effect of incrementing the microstep without incrementing the model time, which allows iteration without time advancing.

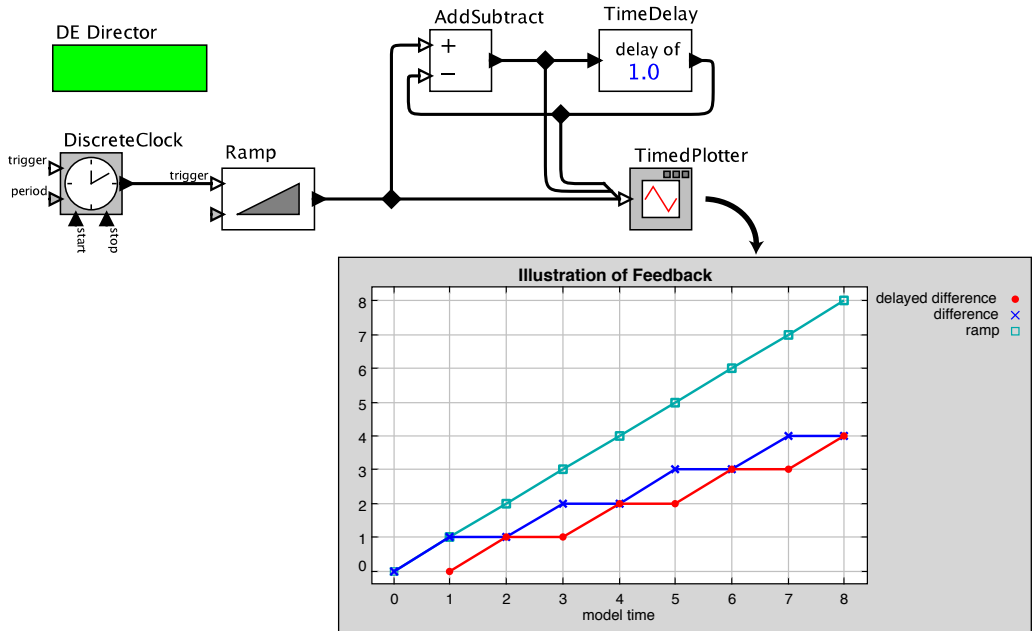


Figure 3.8: Discrete-event model with feedback, which requires a delay actor such as **TimeDelay**.

Example 3.10: Consider the model in Figure 3.9, which produces the plot in Figure 3.10. This model produces a variable number of events at each integer model time. It accomplishes this with a feedback loop that has a **TimeDelay** actor with the delay set to 0.0. This causes the events that are fed back to have incremented microsteps. This model uses a **BooleanSwitch** (see sidebar on page ??) to feed back a token only if its value is non-negative.

The previous example illustrates iteration in DE using a construct similar to the feedback iteration in dataflow illustrated in Example ??. In DE, we can use a **Merge** actor rather than a **BooleanSelect** because the use of time stamps makes the **Merge** determinate.

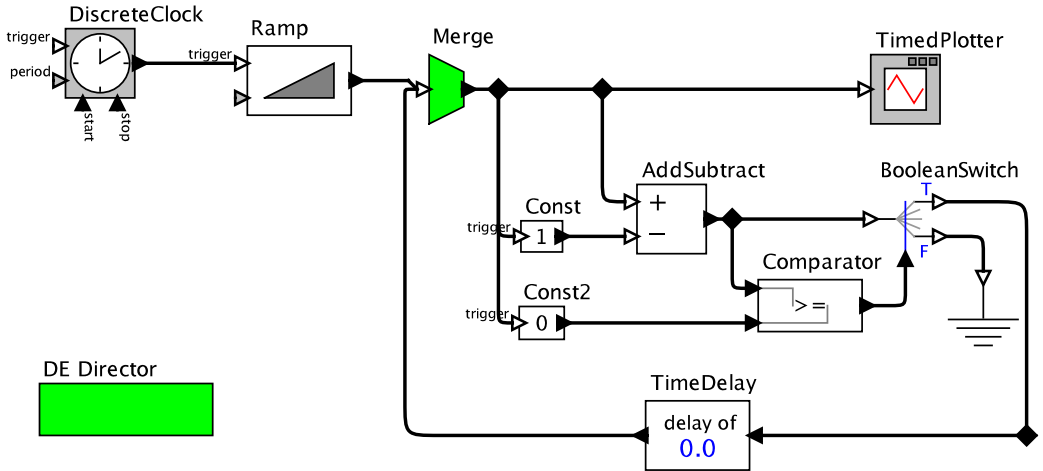


Figure 3.9: Illustration of TimeDelay with delay value of zero.

3.2.3 Zeno Models

It is possible to create DE models where time fails to advance.



Figure 3.10: Result of executing the model in Figure 3.9.

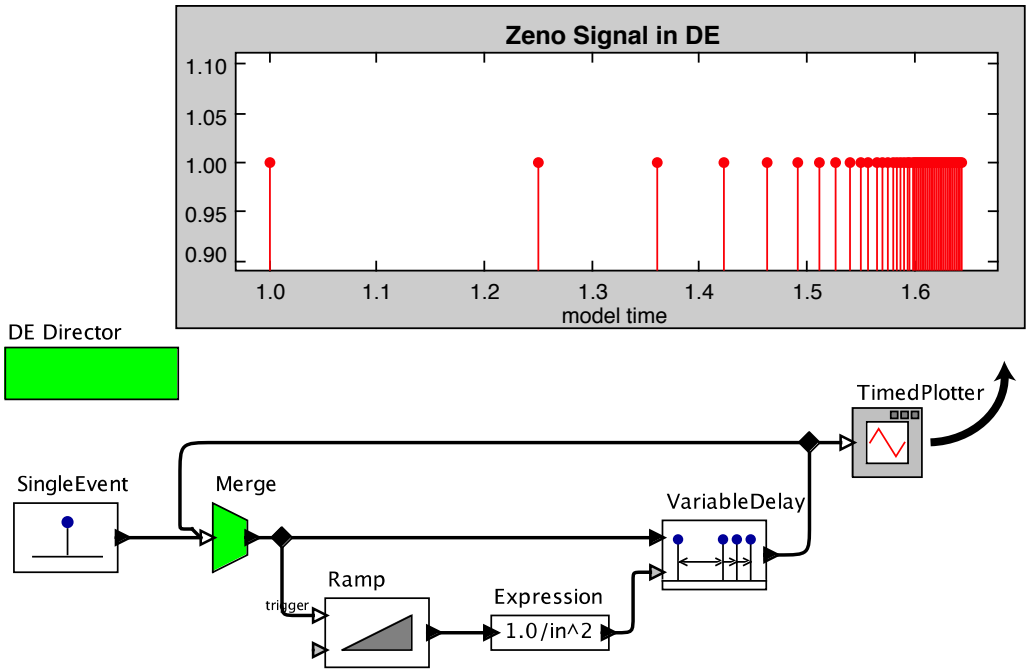


Figure 3.11: Example of a Zeno model, where time fails to advance.

Example 3.11: Suppose that if in Figure 3.9 we were to omit the BooleanSwitch and unconditionally feed back the tokens. Then time would never advance. Only the microsteps would advance.

A model where model time stops advancing and only the microsteps advance is called a **stuttering Zeno** model. Since microsteps are implemented by the DE director as a Java *int*, the increment of the microstep will eventually overflow, causing the director to throw an exception.

It is also possible to have models known simply as **Zeno** models where time does continue to advance, but it will not advance past some finite value.

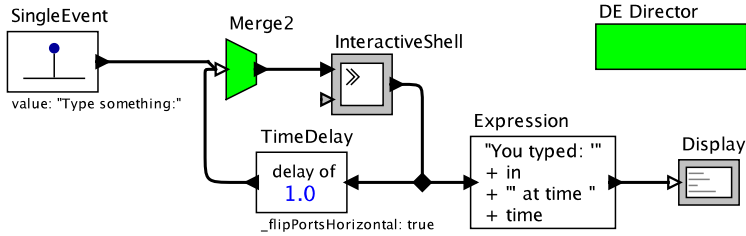


Figure 3.12: A DE model using the InteractiveShell actor, whose execution blocks until a user types something.

Example 3.12: Consider the model in Figure 3.11. This model triggers a feedback loop using a [SingleEvent](#) actor (see sidebar on page 74). In this feedback loop is a [VariableDelay](#) actor (see sidebar on page 80) whose delay value is given by $1/n^2$, where n begins at $n = 1$ and is incremented each time a token cycles around the loop. The delay, therefore, approaches zero.

In DE, because time has finite precision (see sidebar on page 17), eventually a Zeno model becomes a stuttering Zeno model. The particular model in the previous example, however, fails sooner than that because when n becomes large enough, floating point errors in calculating $1/n^2$ (oddly) yield a negative number, causing the VariableDelay actor to throw an exception.

3.2.4 Multithreaded Execution

The DE director fires one actor at a time and does not fire the next actor until the previous one finishes firing. This creates two potential problems. First, if an actor does not return from its `fire()` method, then the entire model will be blocked. This can be a particular issue for actors that perform I/O. Second, the execution of the model is unable to exploit multicore architectures. Both of these problems can be solved using the **ThreadedComposite** actor, found in the `HigherOrderActors` library. The following example illustrates the first problem.

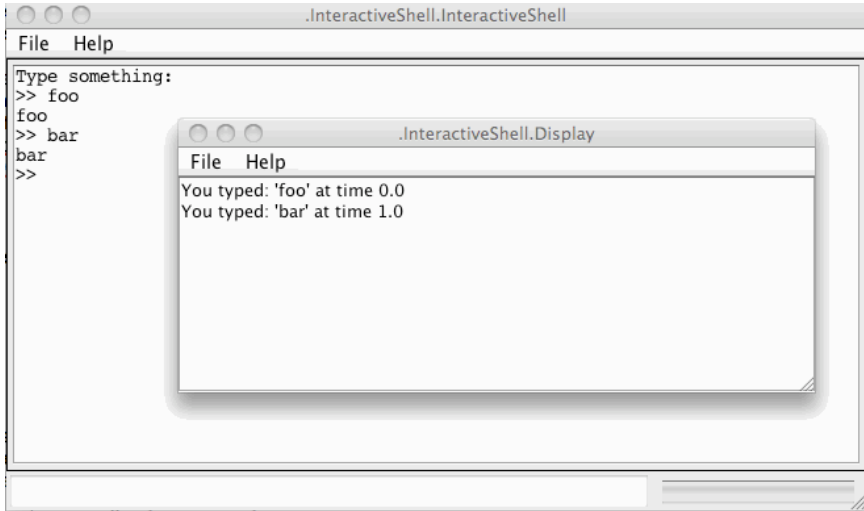


Figure 3.13: An execution of the model in Figure 3.12.

Example 3.13: Consider the example in Figure 3.12, which uses the **InteractiveShell** actor, found in `Sources->SequenceSources`. This actor opens a window like the background window in Figure 3.13 into which a user can type. When the actor fires, it displays in the window whatever input it has received, followed by a prompt (which defaults to “>>”), and then waits for the user to type something. When the user types something and hits return, it produces on its output an event containing whatever the user typed as its value. The **Expression** actor is used to format a string to display (see Section ?? in Appendix ??).

Note that the time stamps in this model are not particularly meaningful. They certainly do not accurately reflect the time at which the user types, but rather only represent the order of what the user typed. In addition, this model cannot do anything while the **InteractiveShell** actor is waiting for the user to type something.

The **ThreadedComposite** actor is an example of a **higher-order actor**, which is an actor that has another actor as a parameter or an input. **ThreadedComposite** is parameterized by another actor, and when it fires, instead of performing any functionality itself, it supervises

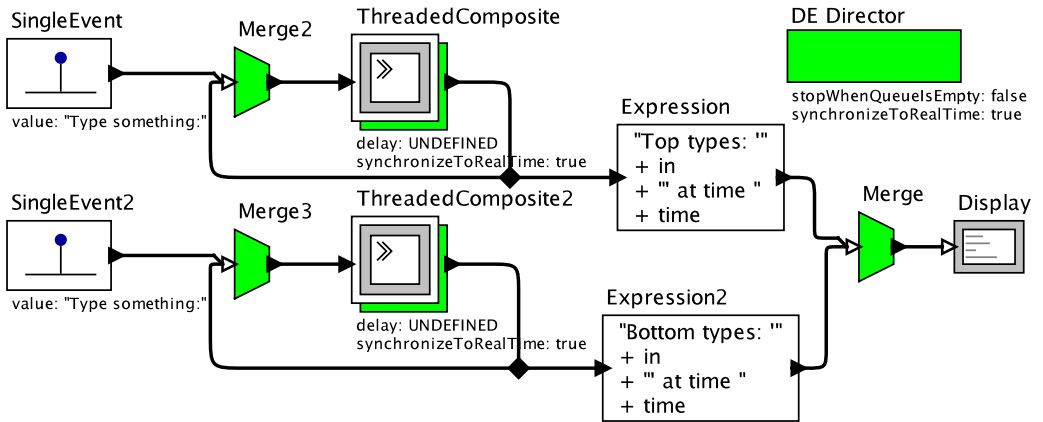


Figure 3.14: A DE model where two instances of *InteractiveShell* are executed in separate threads by the *ThreadedComposite* actor.

the execution of the other actor in another thread. Since the actor executes in another thread, the model is not blocked. This capability can be used to create a much more useful interactive model, as shown in the next example.

Example 3.14: Consider the model in Figure 3.14. This model opens two interactive shell windows into which users can type, and shown in Figure 3.15. The typing can be done in any order, and the model will not block if the user does not type something.

To create this model, we dragged in two instances of *ThreadedComposite*, and then dropped onto those instances two instances of *InteractiveShell*. The icons of the *ThreadedComposite* actors changed to look like those of *InteractiveShell*, decorated with small green shadow.

In this model, the two instances of *InteractiveShell* execute asynchronously in separate threads from that of the DE director. So when the model is running, there are three threads executing. The threads running the *InteractiveShell* actors block waiting for user input. When the user types something and hits return, the *InteractiveShell* actor produces an output, causing its containing *ThreadedComposite* to produce an output.

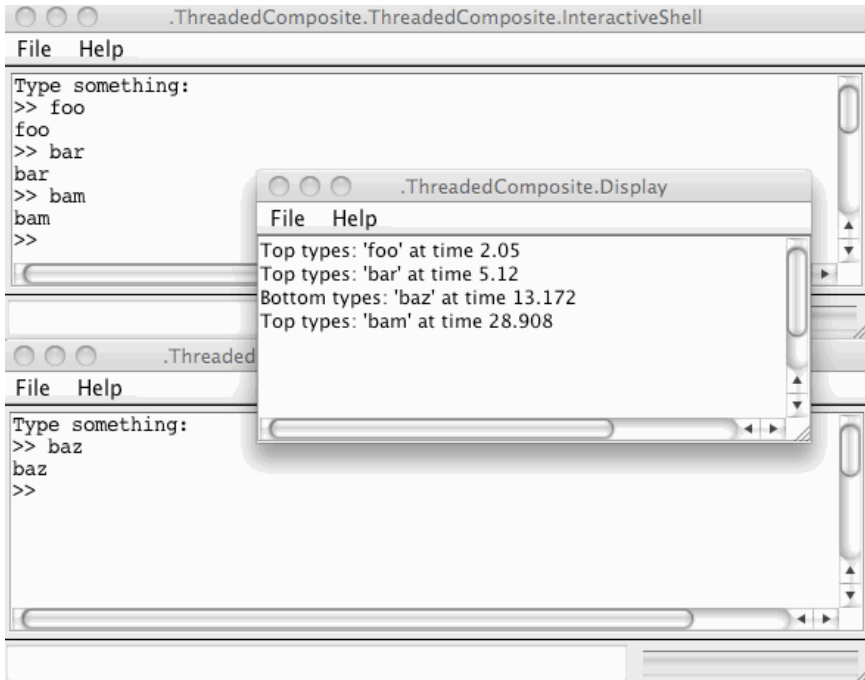


Figure 3.15: An execution of the model in Figure 3.14.

There are a number of subtle points about this model. The *delay* parameter of the ThreadedComposite actors is set to `UNDEFINED`, which instructs the ThreadedComposite actors to assign the current model time, whatever that may be, to their output event time stamps. The time stamps of the outputs are therefore *nondeterminate*.

If instead we had set this parameter to some number τ , then the output events would be assigned model time $t + \tau$, where t is the model time of the triggering input event. This makes the output time stamps determinate. However, it also limits the amount concurrency. When *delay* is set to a number τ , the model will block when current model time reaches $t + \tau$. It has to do this because otherwise the ThreadedComposite would attempt to produce output events with time stamps in the past.

Another subtle issue is that the *synchronizeToRealTime* parameter of the director is set to `true`. This ensures that “current model time” advances no faster than real time. Thus, in the output traces shown in Figure 3.15, we can interpret the reported times as a measure of the elapsed time in seconds from the start of execution until the user typed something. This gives the time stamps a physical meaning. It also justifies the nondeterminacy in the model, since the time at which a user types something is certainly nondeterminate (it is not specified by the model).

A third subtlety is that the *stopWhenQueueIsEmpty* parameter of the director is set to `false`. By default, the DE director will stop executing when there are no more events to process. But in this model, events can later appear because the user types something. Thus, we do not want the model to stop when the queue becomes empty.

The `ThreadedComposite` offers a mechanism for executing models concurrently in multiple threads, but the mechanism is much more determinate and controllable than using threads directly, which is fraught with difficulties (Lee, 2006). The actor contained by a `ThreadedComposite` need not be an atomic actor; it can be an arbitrarily complex *composite actor*. The fact that the contained actor executes in a separate thread enables use of actors that block on I/O, and it also enables parallel execution on multicore machines, for improved performance. The subtleties of this actor and further details on its usage are described by Lee (2008).

3.3 Using DE with Dataflow

The *SDF* director can be quite useful within DE models. If a composite actor in a DE model contains an SDF director, then the contained dataflow model will execute one *complete iteration* each time an event is received on an input port. This is useful when, within a DE model, you want to perform some complex computation that does not involve timed events and that is conveniently described by a dataflow model.

The *DDF* director can also be used within DE models, although as discussed in Section ??, controlling what constitutes an iteration in DDF is more difficult than with SDF. Using the *PN* director within DE rarely makes sense, because, as discussed in Section ??, it is quite difficult to bound an iteration in a deterministic way.

The dataflow directors described in Chapter ?? are untimed, except that the SDF director has a *period* parameter. This parameter is sometimes conveniently used within a DE model. If the parameter is set to something other than zero, then the SDF submodel will execute periodically within the DE model, regardless of whether any input events are provided. This can be used to design [clock](#) actors with much more complicated output patterns than are easily specified using [DiscreteClock](#). Note however that the submodel will execute *only* at multiples of the *period*, and not whenever an input event is provided. If at some multiple of the period there are insufficient inputs to execute a complete iteration, then the SDF model will not fire at that time. Moreover, if inputs are provided faster than the SDF submodel consumes them, then they will queue up, possibly eventually exhausting available memory. See Exercise 1 for an example.

It is rarely useful to put a DE model inside an SDF model (or any other dataflow model). The DE submodel will want to be fired at model times determined by its internal actors, but the SDF model will only fire at multiples of the *period*. Hence, this combination will typically trigger an exception something like

```
IllegalActionException: SDF Director is unable to fire
CompositeActor at the requested time: ... . It responds it
will fire it at: ...
    in .DEwithinSDF.CompositeActor.DE Director
```

It will work, however, to put DE within SDF if the SDF model is in turn within a DE model, and the *period* parameter of the SDF model is set to zero. This is because the SDF director will delegate firing requests up the hierarchy, ignoring time advancements itself.

3.4 Wireless and Sensor Network Systems

The **wireless domain** builds on the DE domain to support modeling of wireless networks. In the wireless domain, channel models mediate communication between actors, and the [visual syntax](#) does not require wiring between components. The visual representation of models in the wireless domain is more important than in other Ptolemy II domains in that the location of icons forms a two-dimensional map of the wireless system being modeled.

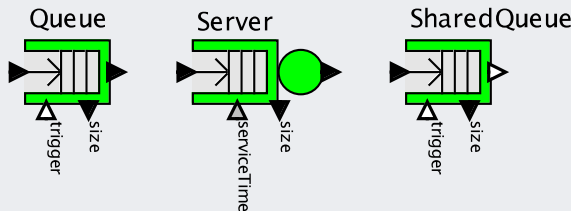
Example 3.15: The top level of the model in Figure 3.16 contains a **WirelessDirector**, two instances of **WirelessComposite**, and a **DelayChannel**. **WirelessCom-**

posite1 has an output port, and WirelessComposite2 has an input port. Although these two ports are not directly connected, they communicate nonetheless. Each port has a parameter *outsideChannel* that specifies by name the wireless channel through which they communicate, which in this case is DelayChannel.

The DelayChannel component models a wireless communication channel that, in this simple case, delays the message by an amount of time that is proportional to the distance between the two WirelessComposite icons in the model. The propor-

Sidebar: Queue and Server Actors

The following actors provide queueing and are particularly useful for modeling behavior in communication networks:



These actors are found in the `DomainSpecific->DiscreteEvent`.

When a token is received on the input port of the **Queue** actor, it is stored in the queue. When the *trigger* port receives a token, the oldest element in the queue is produced on the output. If there is no element in the queue when a token is received on the trigger port, then no output is produced. In this circumstance, if the *persistentTrigger* parameter is true, then the next time an input is received, it is sent immediately to the output. A *capacity* parameter allows limiting the capacity of the queue. Inputs received when the queue is full are discarded. **SharedQueue** supports multiple outputs, each which draws tokens from the same queue.

Server models a server with a fixed or variable service time. A server is either busy (serving a customer) or not busy at any given time. If an input arrives when the server is not busy, then the input token is produced on the output with a delay given by the *serviceTime* parameter. If an input arrives while the server is busy, then that input is queued until the server becomes free, at which point it is produced on the output with an additional delay given by the *serviceTime* parameter. If several inputs arrive while the server is busy, then they are served on a first-come, first-served basis.

tionality constant is given by the *propagationSpeed* parameter (in arbitrary units of distance/time). In this case, the icons are about 175 units apart, so a propagation speed of 100 translates into a delay of approximately 1.75 time units.

In this model, *WirelessComposite1* is a **sporadic source** of events. A sporadic source is a random source where there is a fixed lower bound on the time between events. In this case, the lower bound is enforced by the *Server* actor (see sidebar on page 96) and the randomness is provided by the *PoissonClock* actor.

WirelessComposite2 in this model simply plots the received events as a function of time. The first event is sent by *WirelessComposite1* at time 1.0 and received by *WirelessComposite2* at approximately 2.75. The plot shows that no two events are more closely spaced than one time unit apart.

In addition to modeling channel delay, the wireless domain includes capabilities for modeling power loss, interference, noise, and occlusion. It can also model directional antenna gain and mobile transmitters and receivers. See Baldwin et al. (2004) and Baldwin et al. (2005) for details, and see the demos included in the Ptolemy II package for examples.

3.5 Summary

The DE domain provides a solid foundation for modeling timed behavior. Mastering its use requires an understanding of the model of time, the notion of simultaneity, and feedback, each of which is covered in this chapter. Key to DE is that execution is determinate, even when events are simultaneous. As a consequence, a system designer can have confidence in the behaviors exhibited by an execution of the model.

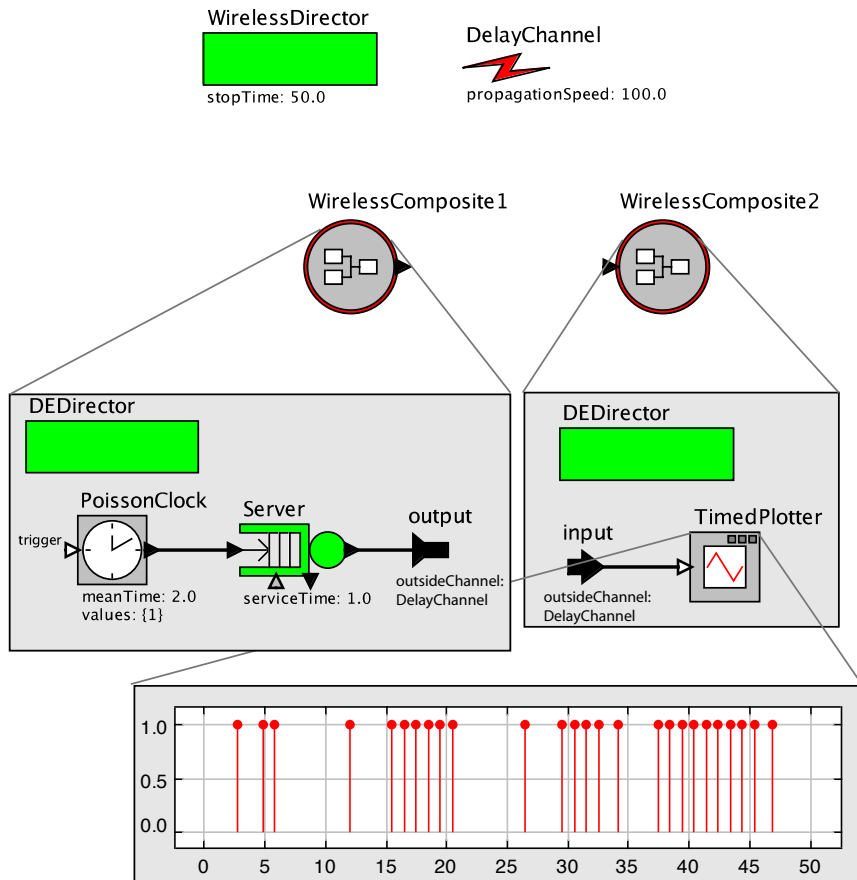


Figure 3.16: A model of a wireless system.

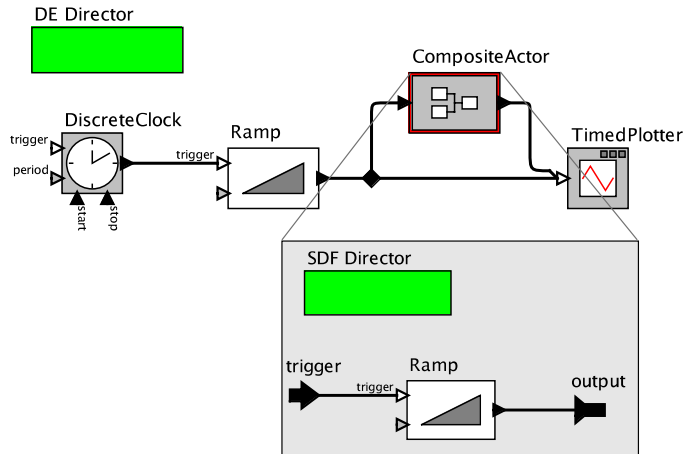


Figure 3.17: Simple example of an SDF submodel within a DE model.

Exercises

1. Consider the model in Figure 3.17. It has an SDF submodel within a DE model.
 - (a) Suppose the *period* parameter of the SDF director is 1.5 and the *period* of the [DiscreteClock](#) is 1.0. What output do you expect from the CompositeActor? Does this model execute with bounded memory?
 - (b) Find values of the *period* parameter of the SDF director and of the [DiscreteClock](#) actor that will generate the plot in Figure 3.18. Explain why this output makes sense.



Figure 3.18: A plot that can be generated by the model in Figure 3.17.

FSMs and Modal Models

Thomas Huining Feng, Edward A. Lee, Xiaojun Liu, and Haiyang Zheng

Contents

4.1	Finite State Machines	103
	<i>Sidebar: Foundations: Model of State Machines</i>	104
4.1.1	Execution Policy for an FSMActor	105
4.1.2	Referencing Input Values of an FSMActor	108
	<i>Probing Further: Hysteresis</i>	108
	<i>Probing Further: Internal Structure of FSMActor</i>	109
4.1.3	Output Actions	111
	<i>Sidebar: Mechanics: Creating FSMs in Vergil</i>	111
4.1.4	Set Actions and Extended Finite State Machines	113
4.1.5	Final States	113
4.1.6	Default Transitions	114
4.1.7	Nondeterministic State Machines	115
4.1.8	Fixed-Point Iterations	117
4.2	Modal Models	121
4.2.1	The Structure of Modal Models	123
4.2.2	Constructing Modal Models	126
	<i>Sidebar: Mechanics: Creating Modal Models</i>	127
4.2.3	Execution Semantics	132
4.2.4	Hierarchical FSMs	133
4.2.5	Preemptive Transitions	133

<i>Probing Further: Concurrent Composition of State Machines</i>	<i>134</i>
<i>Probing Further: Internal Structure of a Modal Model</i>	<i>136</i>
4.2.6 Reset Transitions	137
4.2.7 Transition Refinements	138
4.2.8 Execution Policy for Modal Models	140
4.2.9 Time and Modal Models	141
4.2.10 Time Delays in Modal Models	145
4.2.11 Time in Transition Refinements	147
4.2.12 Time and FSMs	148
4.2.13 Modal Model Principles	149
<i>Probing Further: Implementation of Transient States</i>	<i>150</i>
4.3 Conclusion	152
Exercises	153

The behavior of actors in Ptolemy II can be defined in a number of ways. It can be given in Java by defining a Java class, or in C using the **EmbeddedCActor**, or in Python using the **PythonScript** actor. Alternatively, an **opaque composite actor** can be created to describe the behavior of an actor as a Ptolemy II submodel. In this chapter, we explain how to give the behavior of an actor as a finite-state machine or a modal model. Intuitively, the **state** of a system or subsystem is its condition at a particular point in time. In general, the state affects how the (sub)system reacts to inputs. Formally, we define the state to be an encoding of everything about the past that has an effect on the system’s reaction to current or future inputs.

For example, the **Ramp** actor, which produces a counting sequence, has state. Its reaction to a *trigger* input depends on how many times it has perviously fired. It uses a local variable to keep track of where in its counting sequence it is. This local variable is called a **state variable**, and in this case, the state variable has (typically) a numerical value.

In the case of the Ramp actor, the number of possible states depends on the data type of the counting sequence. If it is `int`, then there are 2^{32} possible states. If it is `double`, then there are 2^{64} . The number of possible states is very large. If the data type is `String`, then the number of possible states is infinite. Although the number of states is very large, the logic for changing from one state to the next is rather simple. So reasoning about the behavior of the actor is not difficult. The actor begins in a state given by its *init* parameter, and on each firing, increments its state by adding to it the value of the *step* parameter. (If the data type is `String`, then “adding” means concatenating.)

In contrast, it is common to have actors that have a relatively small number of possible states, but relatively complex logic for moving from one state to the next. The mechanisms

described here support design, visualization, and analysis of such actors. We first explain the Ptolemy II infrastructure supporting finite state machines (FSMs), and then explain the use of FSMs to construct modal models.

The next section below explains FSMs in Ptolemy II and gives some examples of their usage. The section after that explains modal models, which extend FSMs with the ability to have hierarchical refinements of the states.

4.1 Finite State Machines

A **state machine** is a system whose outputs depend not only on the current inputs, but also on the current state of the system. The state of a system is a summary of everything the system needs to know about previous inputs in order to produce outputs. The state of a system may be represented by a state variable $s \in \Sigma$, where Σ is the set of all possible states that the system can be in. A **finite state machine (FSM)** is a state machine where Σ is a finite set.

Example 4.1: Consider a thermostat controlling a heater that is a state machine with states $\Sigma = \{heating, cooling\}$. If the state is $s = heating$, then the heater is on. If $s = cooling$, then the heater is off. Suppose the target temperature is 20 degrees Celsius. If the heater is on, then the thermostat allows the temperature to rise past the target to, say, 22 degrees. If the heater is off, then it allows the temperature to drop past the target to, say, 18 degrees. Thus, the behavior depends on the state, which summarizes the history by remembering whether the heater is on off. This strategy avoids **chattering**, where the heater would turn on and off rapidly when the temperature is close to the target temperature.

FSMACTOR is a composite actor where refinement consists of states and transitions rather than other actors. Ptolemy II provides a visual notation for these states and transitions as shown in Figure 4.1. In that figure, the FSMACTOR has two input and two output ports, created by the model builder. In general, and FSMACTOR can have any number of input and output ports. The actor reacts to inputs and produces outputs as specified by an FSM, shown visually at the bottom of the figure. The FSM contains a finite number of states (three in the figure). One of these states is an **initial state** (labeled initialState in

the figure), which is the state of the actor at the beginning of execution of the model. The initial state is indicated by a bold outline. Some of the states may also be **final states**, indicated visually with a double outline (more about final states later). States are connected by **transitions**, the annotations on which determine what happens when the actor fires. Before diving into the details, however, the thermostat example may help make the notation intuitive.

Sidebar: Foundations: Model of State Machines

State machines are often described in the literature as a five tuple $(\Sigma, I, O, T, \sigma)$. Σ is the set of states, and σ is the initial state. Nondeterminate state machines may have more than one initial state, in which case $\sigma \subset \Sigma$ is itself a set, although this particular capability is not supported in Ptolemy II FSMs. I is a set of possible valuations of the inputs. In Ptolemy II FSMs, I is a set of functions of the form $i: P_i \rightarrow D \cup \text{absent}$, where P_i is the set of input ports (or input port names), D is the set of values that may be present on the input ports at a particular firing, and *absent* represents “absent” inputs (i.e., $i(p) = \text{absent}$ when $p.\text{isPresent}$ evaluates to false). O is similarly the set of all possible valuations for the output ports at a particular firing.

For a deterministic state machine, T is a function of the form $T: \Sigma \times I \rightarrow \Sigma \times O$, representing the transition relations in the FSM. The guards and output actions are, in fact, just encodings of this function. For a nondeterministic state machine (which is supported by Ptolemy II), the codomain of T is the powerset of $\Sigma \times O$, allowing there to be more than one destination state and output valuation.

The classical theory of state machines ([Hopcroft and Ullman, 1979](#)) makes a distinction between a **Mealy machine** and a **Moore machine**. A Mealy machine associates outputs with transitions. A Moore machine associates outputs with states. Ptolemy II supports both, using output actions for Mealy machines and state refinements in [modal models](#) for Moore machines.

Ptolemy II state machines are actually [extended FSMs](#), which require a richer model than that given above. Extended state machines add to the model above a set V of variable valuations, which are functions of the form $v: N \rightarrow D$, where N is a set of variable names and D is the set of values that variables can take on. An extended state machine is a six-tuple $(\Sigma, I, O, T, \sigma, V)$ where the transition function now has the form $T: \Sigma \times I \times V \rightarrow \Sigma \times O \times V$ (for deterministic state machines). This function is encoded by the transitions, guards, output actions, and set of actions of the FSM.

Example 4.2: A model of a thermostat and a heater is shown in Figure 4.2. The reader should be able to read that figure without much help. In that figure, the FSMActor has a *temperature* input and a *heat* output. Its behavior is given by the FSM shown in the grey box. There are two states, $\Sigma = \{heating, cooling\}$. There are four transitions. The guard on each transition gives the conditions under which the transition is taken. The output actions on each transition give the values produced on the output ports when the transition is taken. Reading the diagram, we see that while in the *heating* state, if the *temperature* input is less than *heatOffThreshold* (22.0), then the output value is *heatingRate* (0.1). When the *temperature* input becomes greater than or equal to *heatOffThreshold*, then the FSM changes to the *cooling* state and produces output value given by *coolingRate* (-0.05). An example execution is plotted in Figure 4.4.

4.1.1 Execution Policy for an FSMActor

An FSMActor contains a set of states and transitions. One of the states is an initial state, and any number of states may be final states. Each transition has a **guard expression**,

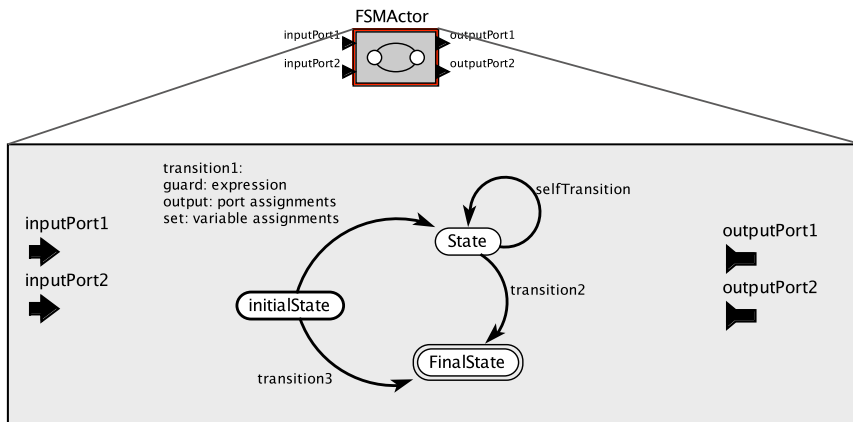


Figure 4.1: Visual notation for state machines in Ptolemy II.

any number of **output actions**, and any number of **set actions**. At the start of execution,

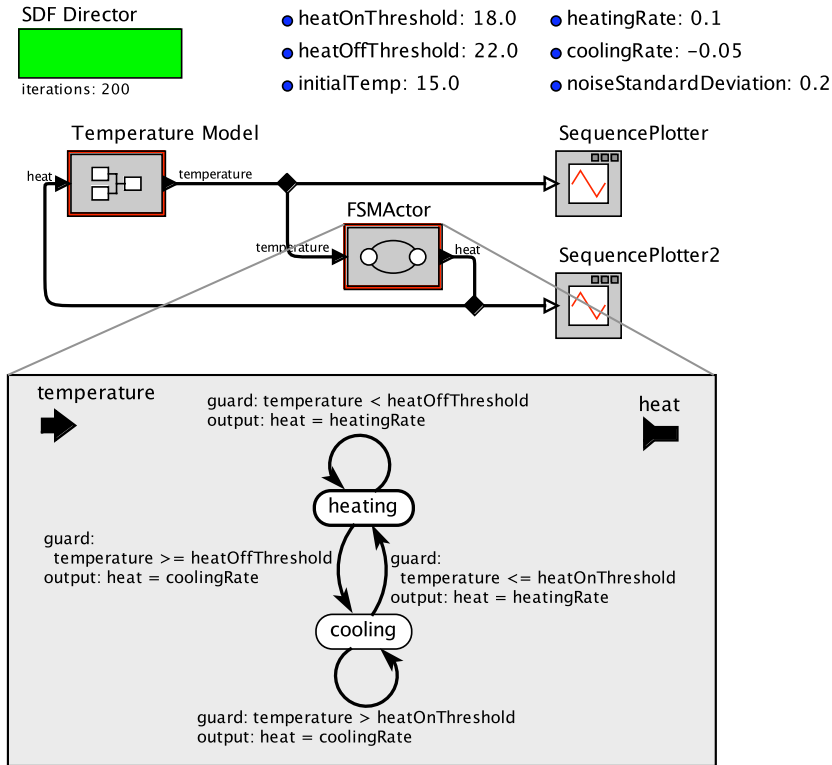


Figure 4.2: A model of a thermostat with hysteresis. The `Temperature Model` actor is shown in Figure 4.3.

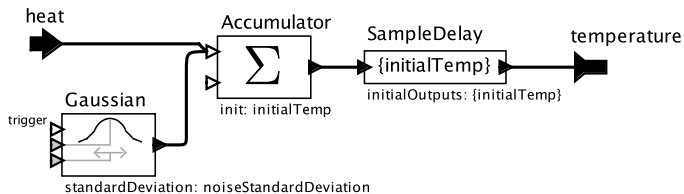


Figure 4.3: The `Temperature Model` composite actor of Figure 4.2.

the state of the actor is set to the initial state. Subsequently, each firing of the actor is a sequence of steps as follows. In the `fire()` method, the actor

1. reads inputs;
2. evaluates guards on outgoing transitions of the current state;
3. chooses a transition whose guard evaluates to true; and
4. executes the output actions on the chosen transition, if any.

In the `postfire()` method, the actor

5. executes the set actions of the chosen transition; and
6. changes the current state to the destination of the chosen transition.

These are separated into two distinct methods to support the use of this actor in domains that do a [fixed-point iteration](#) (such as SR and Continuous), as explained below in Section 4.1.8. In domains that do not do that (such as PN, SDF, and DDF), steps 1 through 6 can execute in sequence in each iteration, and the distinction between `fire()` and `postfire()` is not important.

After reading the inputs, this actor examines the outgoing transitions of the current state, evaluating their guard expressions. A transition is *enabled* if its guard expression evalu-

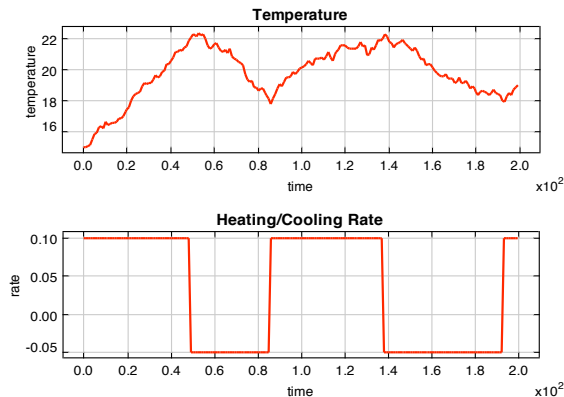


Figure 4.4: Two plots generated by Figure 4.2, showing the temperature (above) and whether the heater is on or off (below), both as a function of time.

ates to true. A blank guard expression is interpreted to be always true. The guard expression may refer to any input port and any variable in scope.

4.1.2 Referencing Input Values of an FSMActor

If an input port name *portName* is used in a guard expression, it refers to the current input on that port on channel zero. For example, in Figure 4.2, in the guard expression “`temperature < heatOffThreshold`” the variable `temperature` refers to the current value in the input port *temperature* input port, and `heatOffThreshold` refers to the parameter named *heatOffThreshold*.

In many models of computation, an input may be absent. If a port *p* is absent, then an expression like “`p < 10`” evaluates to false. However, an expression like “`p < 10 || true`” evaluates to true. A clearer technique that leads to more readable state ma-

Probing Further: Hysteresis

The thermostat in example 4.2 exhibits a particular form of state-dependent behavior called **hysteresis**. A system with hysteresis has the property that the absolute time scale is irrelevant. Suppose the input is a function of time, $x: \mathbb{R} \rightarrow \mathbb{R}$ (for the thermostat, $x(t)$ is the temperature at time t). Suppose that input x causes output $y: \mathbb{R} \rightarrow \mathbb{R}$, also a function of time. E.g., in Figure 4.4, x is upper signal and y is the lower one. For this system, if instead of x the input is x' given by

$$x'(t) = x(\alpha \cdot t)$$

for a non-negative constant α , then the output is y' given by

$$y'(t) = y(\alpha \cdot t) .$$

Scaling the time axis at the input results in scaling the time axis at the output, so the absolute time scale is irrelevant.

An alternative implementation of the thermostat would use a single temperature threshold, but instead would require that the heater remain on or off for at least a minimum amount of time, regardless of the temperature. This design would not have the hysteresis property. The consequences of this design choice are explored in exercise 1.

Probing Further: Internal Structure of FSMActor

FSMActor is a subclass of `CompositeEntity`, just like `CompositeActor`. Internally, it contains some number number of instances of `State` and `Transition`, which are subclasses of `Entity` and `Relation` respectively. The simple structure shown below:



is represented in **MoML** as follows:

```

1 <entity name="FSMActor" class="...FSMActor">
2   <entity name="State1" class="...State">
3     <property name="isInitialState" class="...Parameter"
4       value="true"/>
5   </entity>
6   <entity name="State2" class="...State"/>
7   <relation name="relation" class="...Transition"/>
8   <relation name="relation2" class="...Transition"/>
9   <link port="State1.incomingPort" relation="relation2"/>
10  <link port="State1.outgoingPort" relation="relation"/>
11  <link port="State2.incomingPort" relation="relation"/>
12  <link port="State2.outgoingPort" relation="relation2"/>
13 </entity>

```

The same structure can be specified in Java as follows:

```

1 import ptolemy.domains.modal.kernel.FSMActor;
2 import ptolemy.domains.modal.kernel.State;
3 import ptolemy.domains.modal.kernel.Transition;
4
5 FSMActor actor = new FSMActor();
6 State statel = new State(actor, "State1");
7 State state2 = new State(actor, "State2");
8 Transition relation = new Transition(actor, "relation");
9 Transition relation2 = new Transition(actor, "relation2");
10 statel.incomingPort.link(relation2);
11 statel.outgoingPort.link(relation);
12 state2.incomingPort.link(relation);
13 state2.outgoingPort.link(relation2);

```

Thus, above, we see three distinct concrete syntaxes for the same structure.

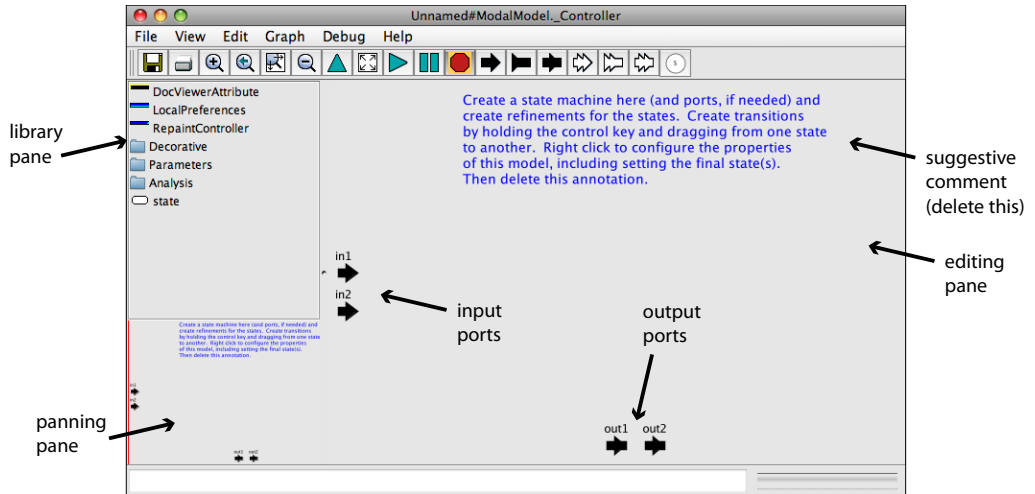


Figure 4.5: Editor for FSMs in Vergil, showing two input and two output ports, before being populated with an FSM.

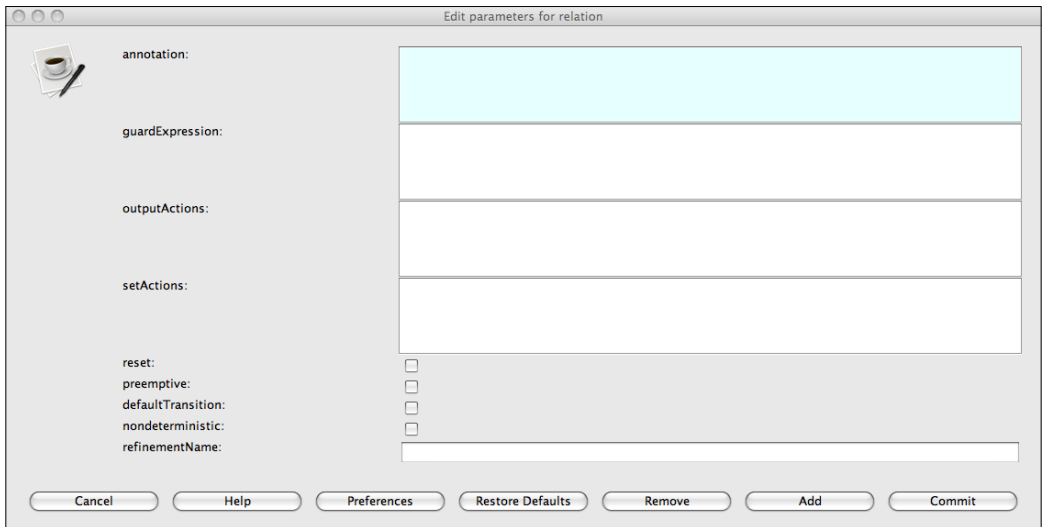


Figure 4.6: Dialog box for configuring a transition in an FSM.

chine models is to use the symbol `portName_isPresent` in guard expressions. This is a boolean that is true if an input on port *portName* is present.

Recall that a **multiport** may have multiple channels. To refer to a channel specifically, a guard expression may use the symbol `portName_channelIndex`, where *channelIndex* is an integer between 0 and $n - 1$, where n is the number of channels connected to the port. This symbol will evaluate to the value received on the port on the given channel. Similarly, a guard expression may refer to `portName_channelIndex_isPresent`.

4.1.3 Output Actions

Once a transition is chosen, its **output actions** are executed. The output action are specified by the *outputActions* parameter of the transition. The form of an output action is typically *portName = expression*, where the expression may refer to input values as above or any parameter in scope. For example, in Figure 4.2, the line

```
output: heat = coolingRate
```

Sidebar: Mechanics: Creating FSMs in Vergil

FSMACTOR in **Vergil** is in `MoreLibraries/Automata`. You can equivalently use **ModalModel** from the `Utilities` library. We recommend using **ModalModel** since it is a more general actor and can do everything **FSMACTOR** can do.

First, drag the actor into your model from the library. Populate the actor with input and output ports by right clicking (or control-clicking on a Mac) and selecting **Open Actor**. The resulting window is shown in Figure 4.5. It is similar to other Vergil windows, but has a customized library consisting primarily of a **State**, a library of parameters, and a library of decorative elements for annotating your design. It also includes a textual annotation with a suggestive comment that you will want to delete after reading. Drag in one or more states. To create transitions between states, **hold the control key** (or the **Command key** on a Mac) and click and drag from one state to the other. The grab handles on the transitions can be used to control the curvature and positioning of the transitions. Double click (or right click and select **Configure**) on the transition to set the guard, output actions, and set actions by entering text into the dialog box shown in Figure 4.6. You can also specify an annotation associated with the transition that has no effect on execution, and therefore functions like a comment.

specifies that the output port named *heat* should produce the value given by the parameter *coolingRate*. This gives the behavior of a **Mealy machine**, which is a style of state machine where outputs are produced by transitions rather than by states. **Moore machine** behavior is also achievable using state refinements that produce outputs, as explained below in Section 4.2.

Multiple output actions may be given by separating them with semicolons, as in *port1 = expression1; port1 = expression1*.

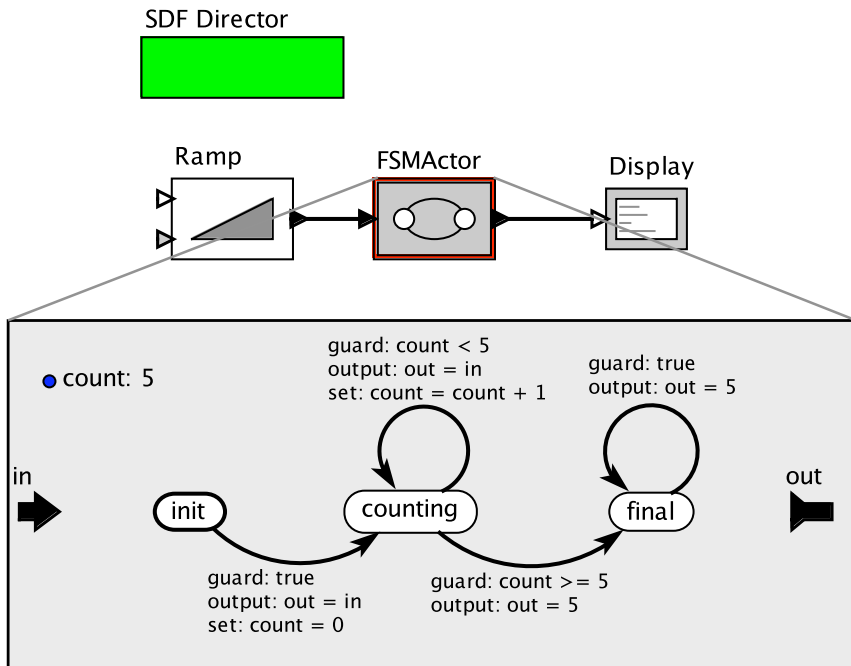


Figure 4.7: An extended FSM, where the *count* variable is part of the state of the system.

4.1.4 Set Actions and Extended Finite State Machines

The **set actions** on a transition can be used to set the values of parameters of the state machine. One practical use for this is to create an **extended FSM**, which is a finite state machine extended with a numerical state variable. It is called “extended” because the number of states now depends on the number of distinct values that the variable can take. It can even be infinite.

Example 4.3: A simple example of an extended FSM is shown in Figure 4.7, which again should be readable without much help. In this example, the **FSMActor** has parameter *count*, shown with the small blue bullet next to it. The transition from the initial state *init* to the counting state initializes *count* to 0 in its set action. The counting state has two outgoing transitions, one that is a self transition, and the other that goes to *final*. The self transition is taken as long as *count* is less than 5. That transition increments the value of *count* by one. When the value of *count* reaches 5, the transition to *final* is taken. Before taking that transition, the output is set equal to the input. Upon taking that transition, the output is henceforth constant. Therefore, the output of this model is the sequence 0, 1, 2, 3, 4, 5, 5, 5, ...

4.1.5 Final States

An FSM may have **final states**, which are states that, when entered, indicate the end of execution of the state machine.

Example 4.4: A variant of Example 4.3 is shown in Figure 4.8. The variant has the *isFinalState* parameter of the final state set to `true`. This is indicated by the double outline around the state. Upon entering that state, the **FSMActor** indicates to the enclosing director that it does not wish to execute any more (it does this by returning `false` from its `postfire()` method). As a result, the output sent to the **Display** actor is the finite sequence 0, 1, 2, 3, 4, 5, 5.

In the iteration in which an `FSMActor` enters a state that is marked final, the `postfire()` method of the `FSMActor` returns false. This indicates to the enclosing director that the `FSMActor` does not wish to be fired again. Most directors will simply avoid firing that actor again, but will continue executing the model. The `SDF` director, however, is different. Since it assumes regular consumption and production rates for all actors, and since it constructs its schedule statically, it cannot tolerate actors that refuse to fire. Hence, the `SDF` director will stop execution of the model altogether if *any* actor returns false from `postfire()`.

4.1.6 Default Transitions

An FSM may have **default transitions**, which are transitions that have the *default* parameter set to true (see Figure 4.6). These transitions become enabled if no other outgoing transitions of the current state are enabled. Default transitions are rendered as dashed arcs rather than solid arcs.

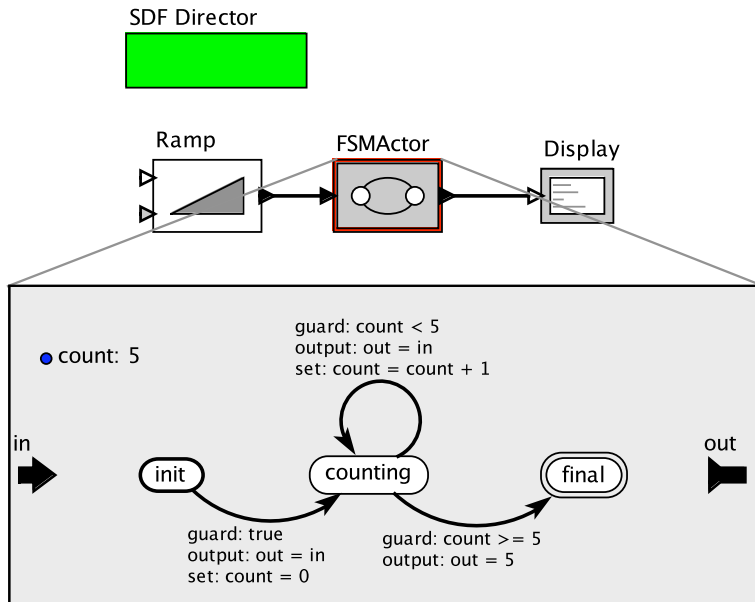


Figure 4.8: A state machine with a final state, which indicates the end of execution of the state machine.

Example 4.5: The FSM of Figure 4.2 can be equivalently implemented using default transitions as shown in Figure 4.9. Here, the default transitions simply specify that the outgoing transition going to the other state is not enabled, then the FSM should remain in the same state and produce an output.

If a default transition also has a guard expression, then that transition is enabled only if the guard evaluates to true *and* there is no other non-default transition enabled. Note that using default transitions with timed models of computation can be somewhat subtle. See Section 4.2.9 below.

4.1.7 Nondeterministic State Machines

If more than one guard evaluates to true at any time, then the FSM is a **nondeterministic FSM**. The transitions that have guards that simultaneously evaluate to true are called **non-**

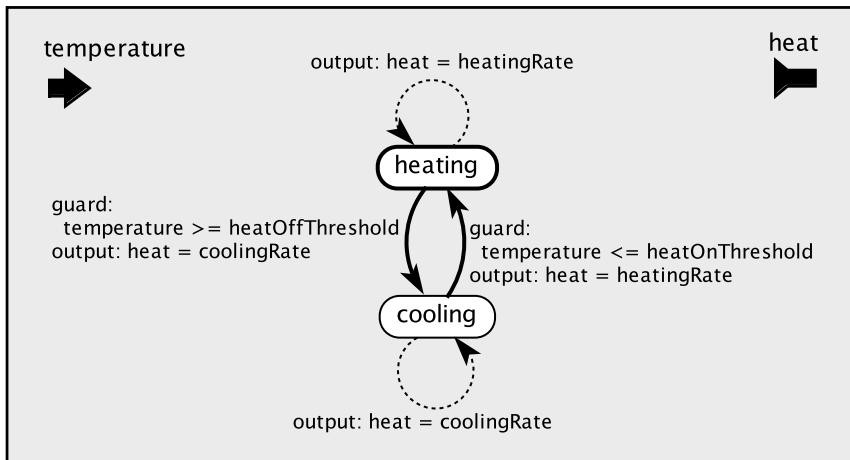


Figure 4.9: An FSM equivalent to the one in Figure 4.2, but using default transitions, indicated with dashed lines.

deterministic transitions. By default, transitions are not allowed to be nondeterministic, so if more than one guard evaluates to true, you will see an exception something like this:

```
Nondeterministic FSM error: Multiple enabled transitions found
but not all of them are marked nondeterministic.
in ... name of a transition not so marked ...
```

To permit them to be nondeterministic, set the *nondeterministic* parameter to `true` on every transition that can be enabled while another another transition is enabled.

Example 4.6: A model of a faulty thermostat is shown in Figure 4.10. When the FSM is in the heating state, both outgoing transitions are enabled (their guards are both `true`), so either one can be taken. Both transitions are marked nondeterministic, a fact that is indicated visually by rendering the transitions in red. A plot of an execution is shown in Figure 4.11. Note that the heater stays on now for rather short periods of time, causing the temperature to hover around 18 degrees, the threshold at which the heater is turned on.

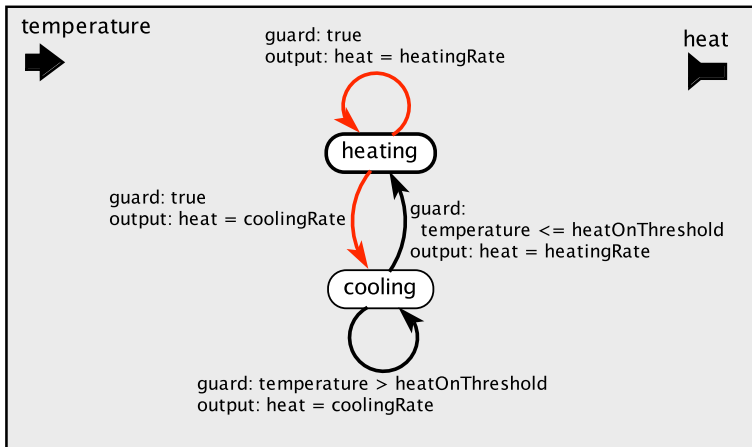


Figure 4.10: A model of a faulty thermostat that nondeterministically switches from heating to cooling.

In a nondeterministic FSM, if more than one transition is enabled and they are all marked nondeterministic, then one is chosen at random in the `fire()` method of the `FSMActor`. If the `fire()` method is invoked more than once in an iteration (see Section 4.1.8 below), then subsequent invocations in the same iteration will always choose the same transition.

If more than one default transition leaves a state, then these transitions must also be marked nondeterministic or an exception will result. Nondeterministic default transitions are rendered as red dashed arcs.

4.1.8 Fixed-Point Iterations¹

In Section 4.1.1 above, we explain that the execution of an `FSMActor` is divided into two segments, steps 1-4, which are performed in the `fire()` method, and steps 5-6, which

¹This section may be safely skipped on a first reading unless you are particularly focusing on fixed-point domains such as SR and Continuous.

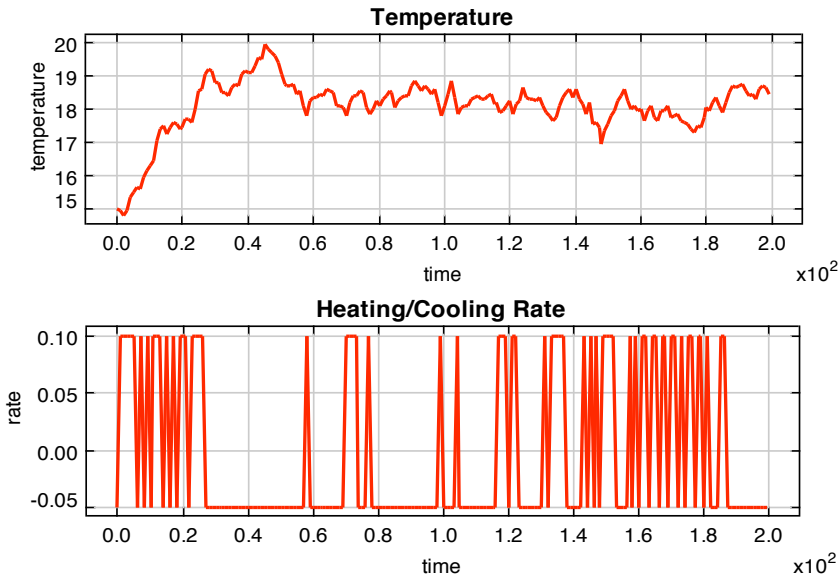


Figure 4.11: Plot of the thermostat FSM of Figure 4.10 replacing that in Figure 4.2.

are performed in the `postfire()` method. This separation is important in domains that perform a fixed-point iteration, such as SR and Continuous. In domains with a fixedpoint iteration, the `fire()` method may be invoked more than once in an iteration while the director searches for a solution. For such domains, it is important that the `fire()` method not include any persistent state changes. Steps 1-4 read inputs, evaluate guards, choose a transition, and produce outputs, but they do not commit to a state transition or change the value of any local variables.

The reason for this separation can be understood by studying the example in Figure 4.12. As explained in chapter ??, execution of an SR model involves finding a value for each

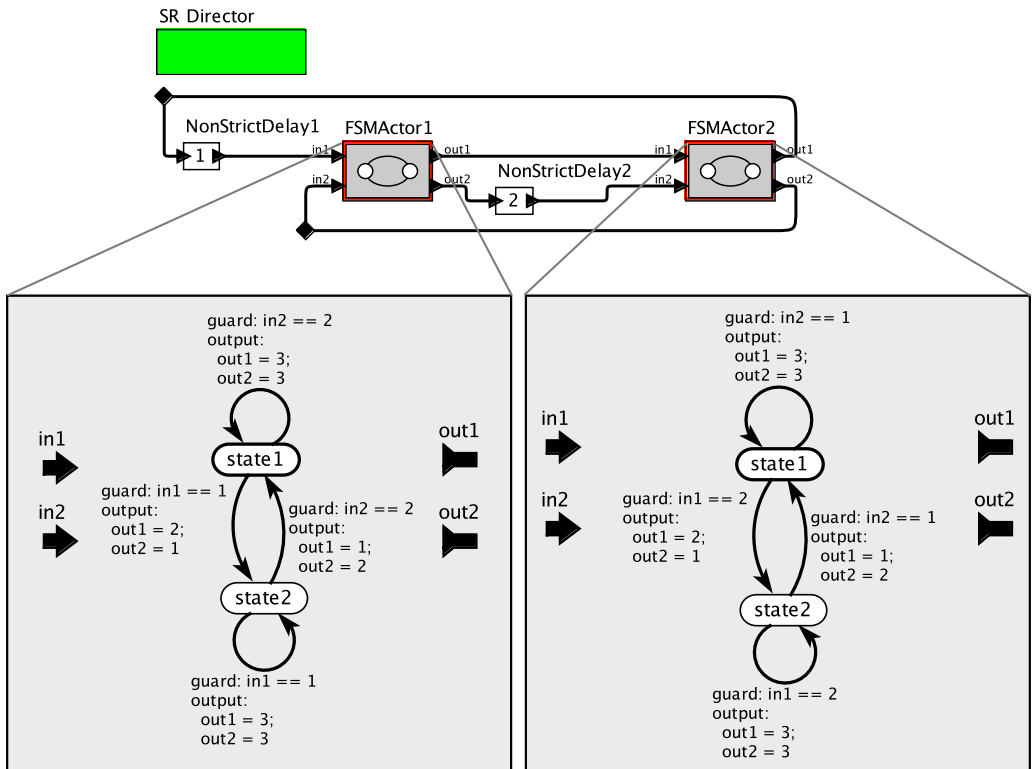


Figure 4.12: A model that requires separation of actions between the `fire()` method and the `postfire()` method in order to be able to converge to a fixed point.

signal at each tick of a global clock. On the first tick, each of the `NonStrictDelay` actors puts the value shown in its icon on its output port (the values are 1 and 2, respectively). This defines the *in1* value for `FSMActor1` and the *in2* value for `FSMActor2`. But the other input ports remain undefined. The value of *in2* of `FSMActor1` is specified by `FSMActor2`, and the value of *in1* of `FSMActor2` is specified by `FSMActor1`. This seems to create a causality loop, but close examination of the state machines shows that there is no causality loop.

In Figure 4.12, note for all states of the `FSMActors`, each input port has a guard that depends on its value. Thus, it would seem that both inputs need to be known before any output value can be asserted, which again suggests a causality loop. However, looking closely at the left FSM, we see that the transition from *state1* to *state2* will be enabled at the first tick of the clock because *in1* has value 1, given by `NonStrictDelay1`. If the state machine is determinate, then this must be the only enabled transition. Since there are no `nondeterministic transitions` in the state machine, we can assume this will be the chosen transition. Once we make that assumption, we can assert both output values as shown on the transition (*out1* is 2 and *out2* is 1).

Once we assert those output values, then both inputs of `FSMActor2` become known, and it can fire. Its inputs are *in1* = 2 and *in2* = 2, so in the right state machine the transition from *state1* to *state2* is enabled. This transition asserts that *out2* of `FSMActor2` has value 1, so now both inputs to `FSMActor1` are known to have value 1. This reaffirms that `FSMActor1` has exactly one enabled transition, the one from *state1* to *state2*.

It is easy to verify that at each tick of the clock, both inputs of each state machine have the same value, so no state ever has more than one enabled outgoing transition. Determinism is preserved. Moreover, the values of these inputs alternate between 1 and 2 in subsequent ticks. For `FSMActor1`, the inputs are 1, 2, 1, ... in ticks 1, 2, 3, ... For `FSMActor2`, the inputs are 2, 1, 2, ... in ticks 1, 2, 3, ...

As explained in Section 4.1.1 above, in the `fire()` method, the actor

1. reads inputs;
2. evaluates guards on outgoing transitions of the current state;
3. chooses a transitions whose guard evaluates to true; and
4. executes the output actions on the chosen transition, if any.

In a fixed-point iteration, this may happen multiple times, and there are subtleties associate with each step. Specifically:

1. *reads inputs*: Some inputs may not be known. Unknown inputs cannot be read, so the actor simply doesn't read them.
2. *evaluates guards on outgoing transitions of the current state*: Some of these guards may depend on unknown inputs. These guards may or may not be able to be evaluated. For example, if the guard expression is "`true || in1`" then it can be evaluated whether the input *in1* is known or not. If a guard cannot be evaluated, then it is not evaluated.
3. *chooses a transition whose guard evaluates to true*: If exactly one transition has a guard that evaluates to true, then choose that transition. If a transition has been chosen already in a previous invocation of the `fire()` method in the same iteration, then the actor checks that the *same* transition is chosen this time. If not, it throws an exception and execution is halted. The FSM is not permitted to change its mind about which transition to take partway through an iteration. If more than one transition has a guard that evaluates to true, then the actor checks that every such transition is identified as a [nondeterministic transition](#). If any such transition is not so marked, then the actor throws an exception. If all such transitions are marked nondeterministic, then it chooses one of the transitions. Subsequent invocations of the `fire()` method in the same iteration will choose the same transition.
4. *executes the output actions on the chosen transition, if any*: If a transition is chosen, then the output values can all be defined. Some of these may be specified on transition itself. If they are not specified, then they are asserted to be `absent` at this tick. If all transitions are disabled (all guards evaluate to false), then all outputs are set to `absent`. If no transition is chosen but at least one transition remains whose guard cannot be evaluated, then the outputs remain unknown.

In the `postfire()` method, the actor

5. executes the set actions of the chosen transition; and
6. changes the current state to the destination of the chosen transition.

These actions are performed exactly once after the fixed-point iteration has determined all signal values. If any signal values remain undefined at the end of the iteration, then an exception is thrown. The model is defective.

Nondeterministic FSMs in a fixed-point domain have some subtleties. It is possible to construct a model for which there is a fixed point that has two enabled transitions but where the selection between transitions is not actually random. It could be that only one of the transitions is ever chosen. This occurs when there are multiple invocations of the `fire()` method in the fixed-point iteration, and in the first of these invocations, one of the guards cannot be evaluated because it has a dependence on an input that is

not known. If the other guard can be evaluated in the first invocation of `fire()`, then the other transition will always be chosen. As a consequence, for nondeterministic state machines, the behavior may depend on the order of firings in a fixed-point iteration.

Note that **default transitions** may also be marked nondeterministic. However, a default transition will not be chosen unless all non-default transitions have guards that evaluate to false. In particular, it will not be chosen if any non-default transition has a guard that cannot yet be evaluated because of unknown inputs. If all non-default transitions have guards that evaluate to false and there is more than one default transition, all marked nondeterministic, then one is chosen at random.

4.2 Modal Models

Most interesting systems have behavior that changes over time. The changes in behavior may be triggered by user inputs, hardware failures, or sensor data, for example. A **modal model** is an explicit representation of a finite set of behaviors and the rules that govern transitions between behaviors. The switching between behaviors is governed by an FSM.

ModalModel is a hierarchical actor, like a **composite actor**, but with multiple refinements instead of just one. Each refinement gives one behavior. A **state machine** determines which refinement is active at any given time. The **ModalModel** actor is a more general form of the **FSMACTOR** described in the previous section. The **FSMACTOR** does not support state refinements. You can always use **ModalModel** instead of **FSMACTOR** and just not create state refinements. Hence, there is no real reason to use **FSMACTOR**.

Example 4.7: The model shown in Figure 4.13 has an actor labeled “Modal Model” that has two modes, clean and noisy. This models a communication channel with two modes of operation. In the clean mode, it passes inputs to the output unchanged. In the noisy mode, it adds a Gaussian random number to each input token. The top-level model provides an *event* signal that comes from the **PoissonClock** actor. That actor generates events at random times according to a Poisson process.² A sample execution of this model where the Signal Source actor provides a sine wave results in the plot shown in Figure 4.14.

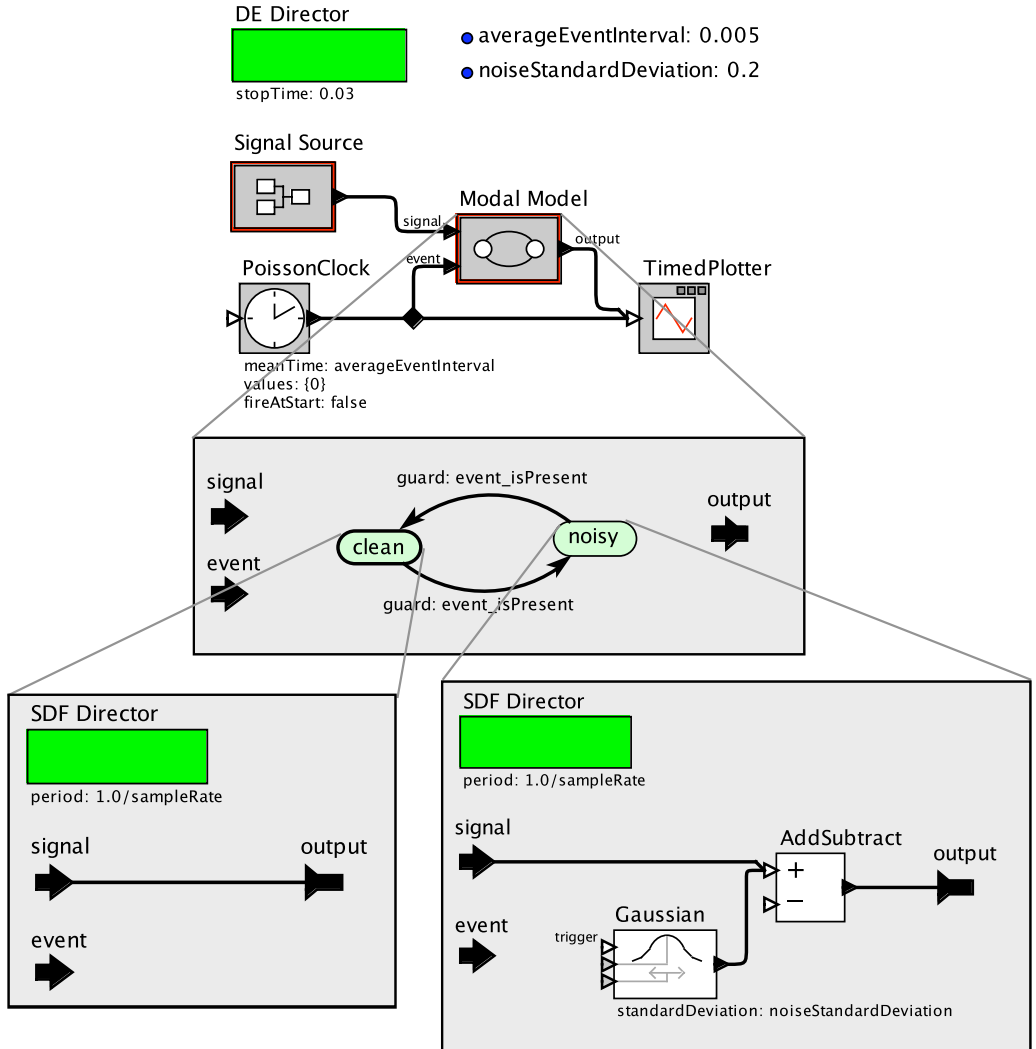


Figure 4.13: Simple modal model that has a normal (clean) operating mode, in which it passes inputs to the output unchanged, and a faulty mode, in which it adds Gaussian noise. It switches between these modes at random times determined by the PoissonClock actor.

4.2.1 The Structure of Modal Models

The general pattern of a modal model is shown in Figure 4.15. The behavior of a modal model is governed by a state machine, where each state is called a **mode**. In Figure 4.15, each mode is represented by a bubble, just like a state in a state machine, but filled in a light-blue color to suggest that it is a mode rather than an ordinary state. A mode, unlike an ordinary state, has a **mode refinement**, which is an **opaque composite actor** that defines the behavior when the mode is active. The example in Figure 4.13 shows two refinements, each of which is an **SDF model** that transforms input tokens to produce output tokens.

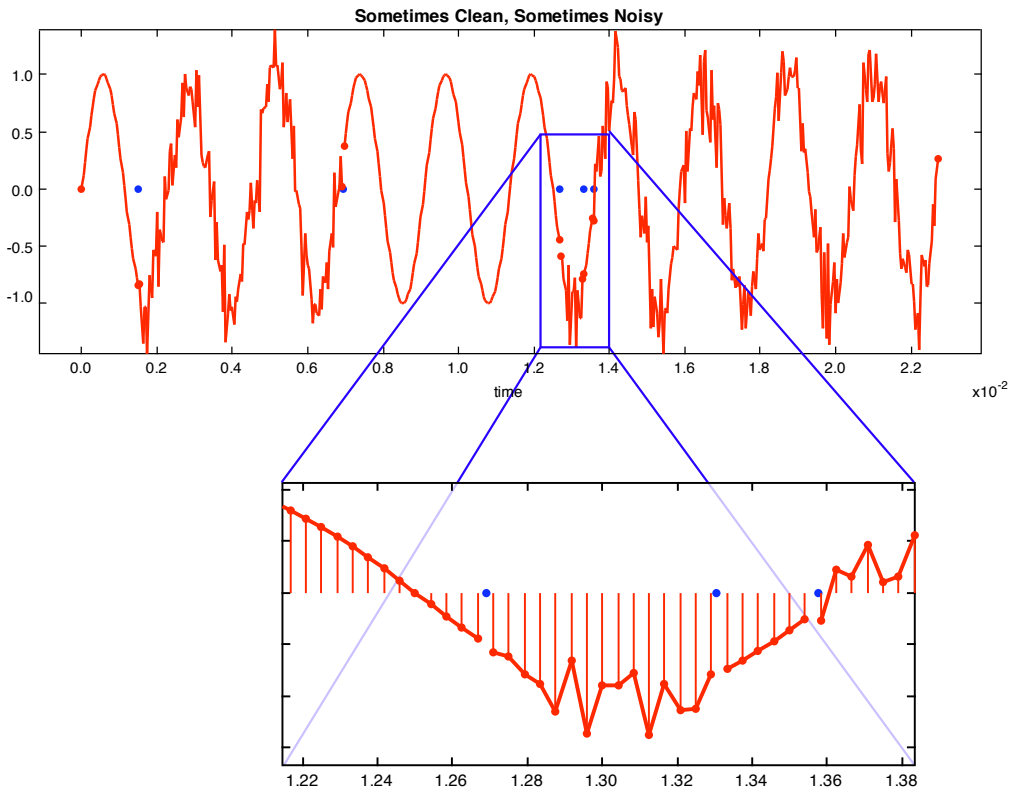


Figure 4.14: Plot generated by the model in Figure 4.13.

Note that it is essential that the refinement contain a director, and that the contained director be usable with the director that governs the execution of the modal model actor. The example in Figure 4.13 has an SDF director inside each of the modes and a DE director outside the modal model. SDF can generally be used inside DE, so this combination is valid.

Just like states in an ordinary state machine, modes are connected by arcs representing **transitions**. Each transition has a **guard**, which is a predicate (a boolean-valued expression) that specifies when the transition should be taken.

Example 4.8: In Figure 4.13, the transitions are guarded by the expression `event_isPresent`, which evaluates to true when the `event` input port has an event. Since that input port is connected to the `PoissonClock` actor, the transitions will be taken at random times, with an exponential random variable giving the time between transitions.

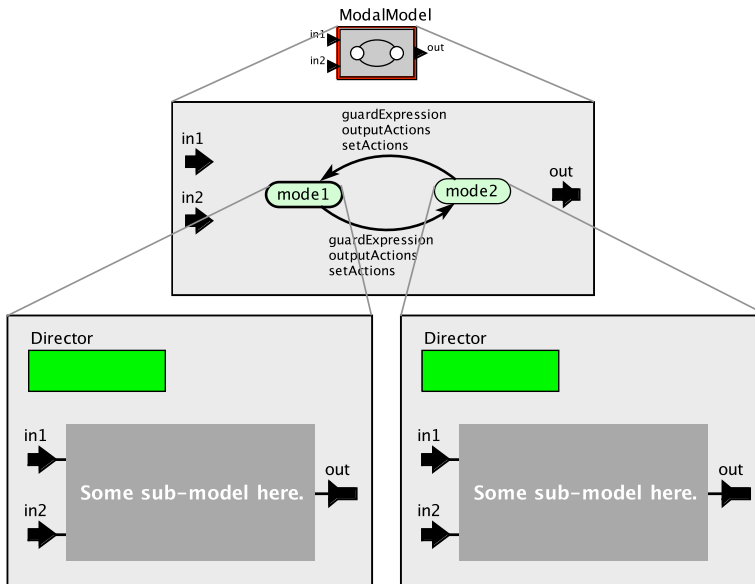


Figure 4.15: General pattern of a modal model with two modes, each with its own refinement.

A variant of the pattern in Figure 4.15 is shown in Figure 4.16, where two modes share the same refinement. This is useful when the behavior in different modes differs only by parameter values. For example, Exercise 3 constructs a variant of the example in Figure 4.13 where the clean refinement differs from the noisy refinement only by having a different parameter value for the Gaussian actor. To construct a model where multiple modes have the same refinement, add a refinement to one of the states, giving the refinement a name (by default, the suggested name for the refinement is the same as the name of the state, but the user can choose any name for the refinement). Then, for another state, instead of choosing `Add Refinement`, choose `Configure` (or simply double click on the state) and specify the refinement name as the value for the *refinementName* parameter. Both modes will now have the same refinement.

Another variant of the pattern is where a mode has multiple refinements. This can be accomplished by executing `Add Refinement` multiple times or by specifying a comma-

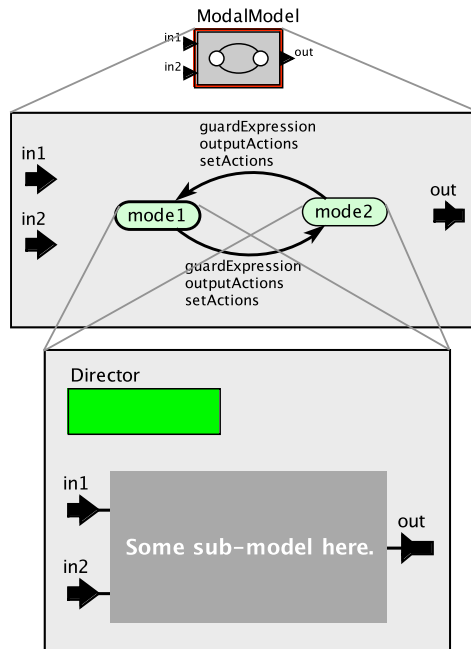


Figure 4.16: Variant of the pattern in Figure 4.15 where two modes share the same refinement.

separated list of refinement names for the *refinementName* parameter. These refinements will execute in the order that they are added.

4.2.2 Constructing Modal Models

A modal model is a component in a larger continuous-time (or other kind of) model. You can create a modal model by dragging one in from the HigherOrderActors library. By

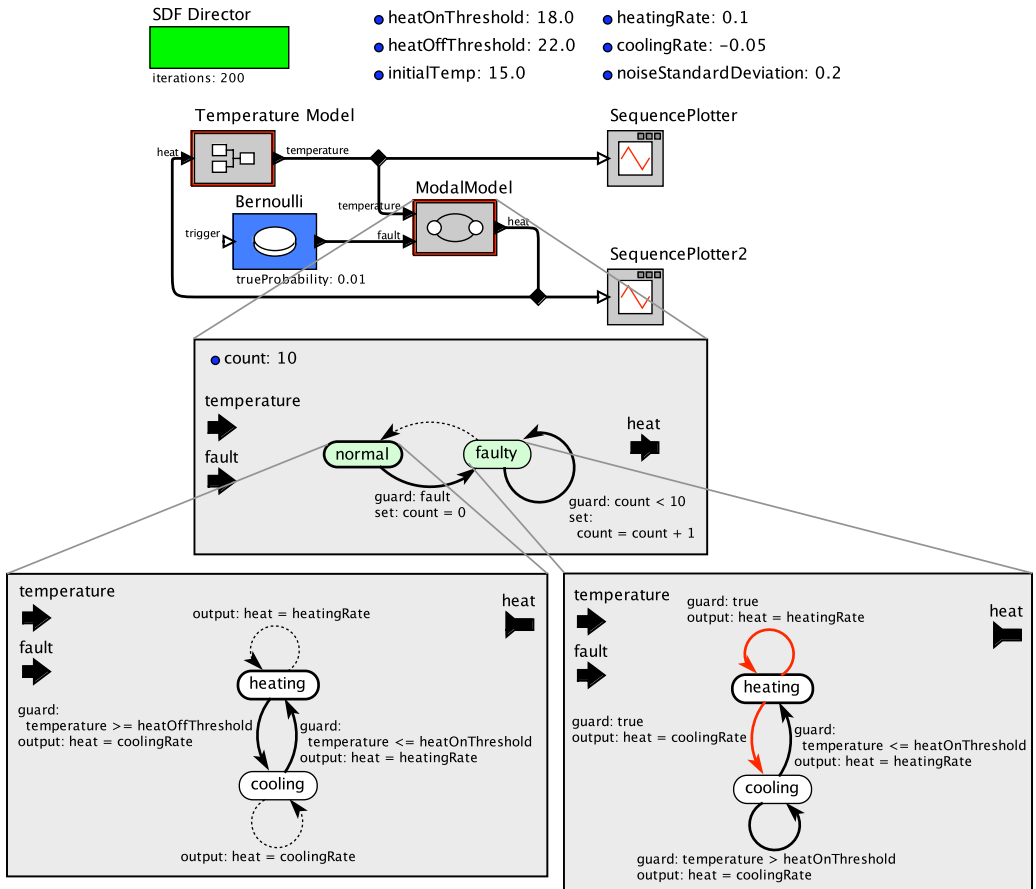


Figure 4.17: A hierarchical FSM that combines the normal and faulty thermostats of Examples 4.2 and 4.6.

default, it has no ports. To make it useful, you will need to add ports. The mechanism for doing that is identical to adding ports to a composite model, and is explained in section 2.3.2. Figure ?? shows a top-level continuous-time model with a single modal model that has been renamed [Ball Model](#). Three output ports have been added to that modal model, but only the top one is used. It gives the vertical distance of the ball from the surface on which it bounces.

If you create a new modal model by dragging in a [ModalModel](#) from the `HigherOrderActors` library, create an output port and name it *output*, and then look inside, you will get a Finite State Machine (FSM) editor like that shown in Figure 4.18. The annotation text suggests that you delete it once you no longer need it.

The output port that you created is in fact indicated in the state machine as being both an output and input port. The reason for this is that guards in the state machine can refer to output values that are produced on this port by refinements. In addition, the output actions of a transition can assign an output value to this port. Hence, the port is, in fact, both an output and input for the state machine.

To create a finite-state machine, drag in states (white circles), or click on the state icon in the toolbar. You can rename these states by right clicking on them and selecting [Customize->Rename]. Choose names that are pertinent to your application. You must specify the initial state of the FSM by right clicking on the background of the FSM Editor, selecting “Edit Parameters”, and specifying an initial state name.

Sidebar: Mechanics: Creating Modal Models

The creation of a modal model is illustrated in Figure 4.21. The process starts by dragging in a [ModalModel](#) from the `Utilities` library and populating it with ports. Then, open the modal model and populate it with one or more states and transitions. To create the transitions, hold the Control key (or the Command key on a Mac) and click and drag from one state to the other. To add a refinement, right click on a state and select Add Refinement. You can choose a `Default Refinement` or a `State Machine Refinement`. The former will require a director and actors that process input data to produce outputs. The latter will enable creation of a [hierarchical FSM](#).

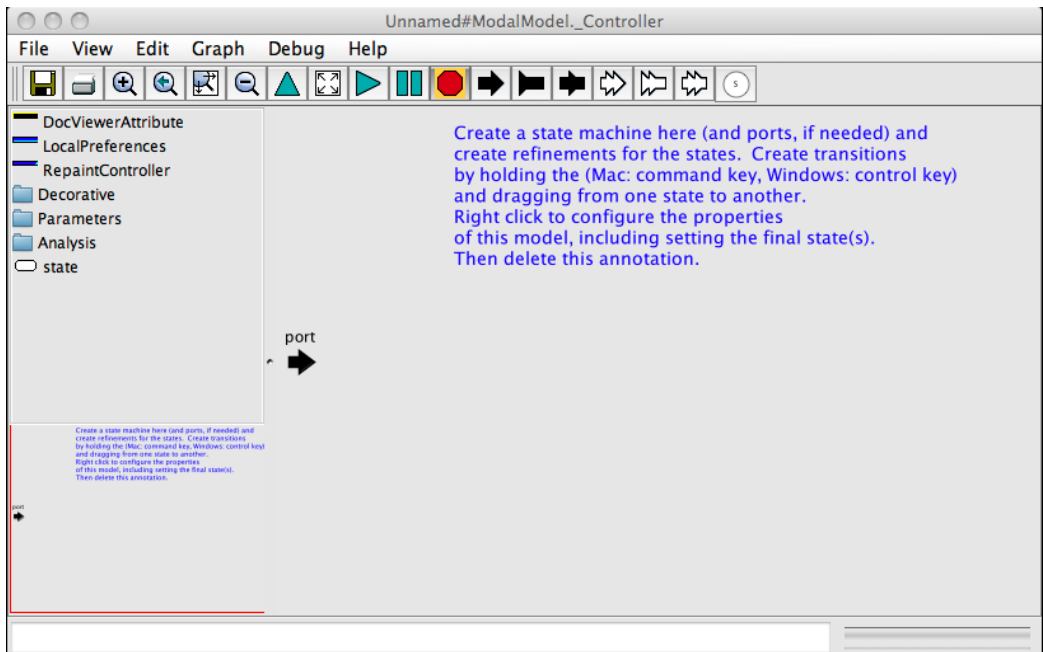


Figure 4.18: Inside of a new modal model that has had a single output port added.

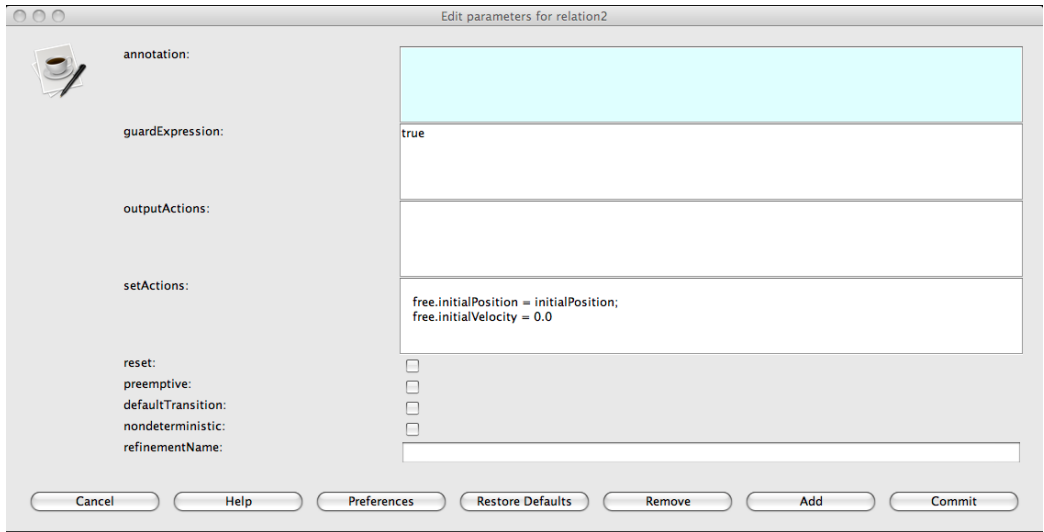


Figure 4.19: Transition dialog for the transition from init to free in Figure ??.

Creating Transitions

To create transitions, you must hold the control button³ on the keyboard while clicking and dragging from one state to the next (a transition can also go back to the same state). The handles on the transition can be used to customize its curvature and orientation. Double clicking on the transition (or right clicking and selecting “Customize” | “Configure”) allows you to configure the transition. The dialog for the transition from init to free is shown in Figure 4.19. In that dialog, we see the following:

- The *guardExpression* is true, so this transition is always enabled. The transition will be taken as soon as the model begins executing. A guard expression can be any boolean-valued expression that depends on the inputs, parameters, or even the outputs of any refinement of the current state (see below). Thus, this transition is used to initialize the model.
- The *outputActions* are empty, meaning that when this transition is taken, no output is specified. This parameter can have a list of assignments of values to output ports,

³Or the command button on a Macintosh computer.

separated by semicolons. Those values will be assigned to output ports when the transition is taken.

- The *setActions* field contains the following statements:

```
free.initialPosition = initialPosition;  
free.initialVelocity = 0.0
```

The “free” in these expressions refers to the mode refinement in the *free* state. Thus, *free.initialPosition* is a parameter of that mode refinement. Here, its value is assigned to the value of the parameter *initialPosition*. The parameter *free.initialVelocity* is set to zero.

- The *reset* parameter is set to *false*, meaning that the refinement of the destination mode is not initialized when the transition is taken.
- The *preemptive* parameter is set to *false*. In this case, it makes no difference, since the init state has no refinement. Normally, if a transition out of a state is enabled and *preemptive* is *true*, then the transition will be taken without first executing the refinement. Thus, the refinement will not affect the outputs of the modal model.
- The *defaultTransition* parameter is set to *false*. If this parameter was set to *true*, then this transition would be enabled if and only if no other non-default transition is enabled.
- The *nondeterministic* parameter is set to *false* meaning that if this transition is enabled, it must be the only enabled transition. This parameter specifies whether this transition is nondeterministic. Here nondeterministic means that this transition may not be the only enabled transition at a time.
- The *refinementName* parameter is empty, indicating that the state is not refined. This parameter specifies the names of refinements. The refinements must be instances of *TypedActor* and have the same container as the *FSMActor* containing this state, otherwise an exception will be thrown when *getRefinement()* is called. Usually, the refinement is the empty string or a single name. However, if a comma-separated list of names is provided, then all the specified refinements will be executed.

A state may have several outgoing transitions. However, it is up to the model builder to ensure that at no time does more than one guard on these transitions evaluate to true. In

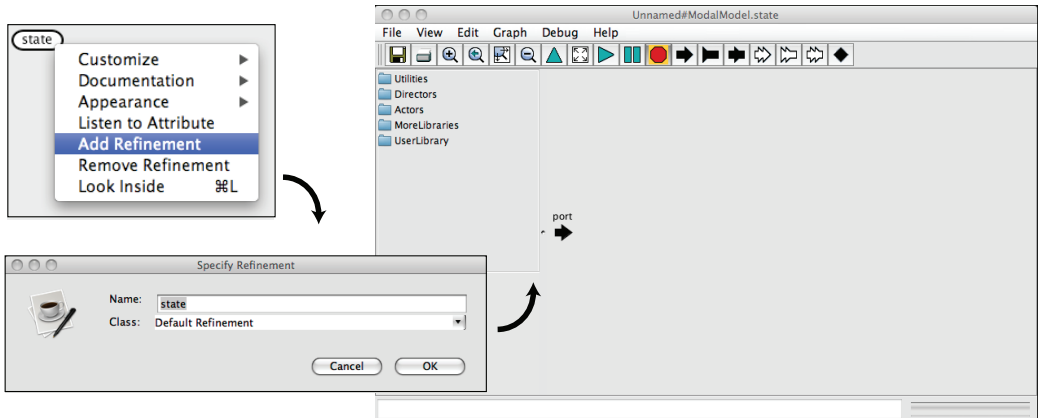


Figure 4.20: Adding a refinement to a state.

other words, Ptolemy II does not allow nondeterministic state machines, and will throw an exception if it encounters one.

Creating Refinements

Both states and transitions can have **refinements**. To create a refinement, right click⁴ on the state or transition, and select “Add Refinement.” You will see a dialog like that in Figure 4.20. As shown in the figure, you will be offered the alternatives of a “Default Refinement” or a “State Machine Refinement.” The first of these provides a block diagram model as the refinement. The second provides another finite state machine as the refinement. In the former case (the default), a blank refinement model will open, as shown in the figure. You will have to create a director in the refinement. The modal model will not operate without a director in the refinement.

You can also create refinements for transitions, but these have somewhat different behavior. They will execute exactly once when the transition is taken. For this reason, only certain directors make sense in such refinements. The most commonly useful is the SDF director. Such refinements are typically used to perform arithmetic computations that are too elaborate to be conveniently specified as an action on the transition.

⁴On a Macintosh, control-click.

Once you have created a refinement, you can look inside a state or transition. For the bouncing ball example, the refinement of the free state is shown in Figure ???. This model exhibits certain key properties of refinements:

- Refinements must contain directors. In this case, the `ContinuousDirector` is used.
- The refinement has the same ports as the modal model, and can read input values and specify output values. When the state machine is in the state of which this is the refinement, this model will be executed to read the inputs and produce the outputs.

4.2.3 Execution Semantics

The behavior of a refinement is simple. When the modal model is executed, the following sequence of events occurs:

- For any transitions out of the current state for which *preemptive* is `true`, the guard is evaluated. If exactly one such guard evaluates to true, then that transition is chosen. The *output actions* of the transition are executed, and the refinements of the transition (if any) are executed, followed by the *set actions*.
- If no preemptive transition evaluated to true, then the refinement of the current state, if there is one, is evaluated at the current time step.
- Once the refinement has been evaluated (and it has possibly updated its output values), the guard expressions on all the outgoing transitions of the current state are evaluated. If none is true, the execution is complete. If one is true, then that transition is taken. If more than one is true, then an exception is thrown (the state machine is nondeterministic). What it means for the transition to be “taken” is that its *output actions* are executed, its refinements (if any) are executed, and its set actions are executed.
- If *reset* is true on a transition that is taken, then the refinement of the destination mode (if there is one) is initialized.

There is a subtle distinction between the *output actions* and the *set actions*. The intent of these two fields on the transition is that *output actions* are used to define the values of output ports, while set actions are used to define state variables in the refinements of the

destination modes. The reason that these two actions are separated is that while solving a continuous-time system of equations, the solver may speculatively execute models at certain time steps before it is sure what the next time step will be. The *output actions* make no permanent changes to the state of the system, and hence can be executed during this speculative phase. The *set actions*, however, make permanent changes to the state variables of the destination refinements, and hence are not executed during the speculative phase.

4.2.4 Hierarchical FSMs

A particularly useful form of modal model is a **hierarchical FSM**. This is a modal model where the state refinements are themselves state machines.

Example 4.9: A hierarchical FSM that combines the normal and faulty thermostats of Examples 4.2 and 4.6 is shown in Figure 4.17. In this model, a **Bernoulli** actor is used to generate a *fault* signal (which will be `true` with probability 0.01). When the *fault* signal is `true`, the modal model will transition to the faulty state and remain there for 10 iterations before returning to the normal mode. The state refinements are the same as those in Figures 4.9 and 4.10, giving normal and faulty behavior of the thermostat.

To create a hierarchical FSM, when you select `Add Refinement` in the context menu of a state in your state machine, instead of choosing `Default Refinement`, you should choose `State Machine Refinement`. The inside state machine can reference the input ports and write to the output ports, and its states can themselves have refinements (either `Default Refinements` or `State Machine Refinements`).

Notice that the model in Figure 4.17 combines a **stochastic state machine** with a non-deterministic FSM. The stochastic state machine has random behavior, but an explicit probability model is provided in the form of the **Bernoulli** actor. The nondeterministic FSM also has random behavior, but no probability model is provided.

4.2.5 Preemptive Transitions

A **preemptive transition** is a transition that may prevent the execution of the current state refinement. A transition is preemptive if its *preemptive* parameter evaluates to `true`. The

Probing Further: Concurrent Composition of State Machines

State machines have a long history in the theory of computation (Hopcroft and Ullman, 1979). Concurrent composition of state machines is a more recent area of study. An early model for such concurrent composition is **Statecharts**, due to David Harel (Harel, 1987). With Statecharts, Harel introduced the notion of **and states**, where a state machine can be in both states *A* and *B* at the same time. On careful examination, the Statecharts model is concurrent composition of hierarchical FSMs under an **SR** model of computation. Statecharts are therefore (roughly) equivalent to modal models combining hierarchical FSMs and the SR director in Ptolemy II (Eker et al., 2003). Statecharts were realized in a software tool called **Statemate** (Harel et al., 1990).

Harel's work triggered a flurry of activity, resulting in many variants of the model (von der Beeck, 1994). Harel's visual syntax was subsequently adopted to become part of **UML**, the **unified modeling language** (Booch et al., 1998). A particularly elegant version is **SynchCharts** (André, 1996), which provides a visual syntax to the Esterel synchronous language (Berry and Gonthier, 1992).

One of the key properties of synchronous composition of state machines is that it becomes possible to model a composition of components as itself a state machine. A straightforward mechanism for doing this results in a state machine whose state space is the cross product of the individual ones. More sophisticated mechanisms have been developed, such as interface automata (de Alfaro and Henzinger, 2001).

Hybrid systems can also be viewed as modal models, where the concurrency model is a continuous time model (Maler et al., 1992; Henzinger, 2000; Lynch et al., 1996). In the usual formulation, hybrid systems couple FSMs with ordinary differential equations (ODEs), where each state of the FSMs is associated with a particular configuration of ODEs. A variety of software tools have been developed for specifying, simulating, and analyzing hybrid systems (Carloni et al., 2006).

Girault et al. (1999) showed that FSMs can be combined hierarchically with a rich variety of concurrent models of computation. They called such compositions ***charts** or **starCharts**, where the star represents a wildcard. Several active research projects continue to explore expressive variants of concurrent state machines. **BIP** (Basu et al., 2006), for example, composes state machines using rendezvous interactions. Alur et al. (1999) give a very nice study of semantic questions around concurrent FSMs, including various complexity questions. Prochnow and von Hanxleden (2007) describe sophisticated techniques for visual editing of concurrent state machines.

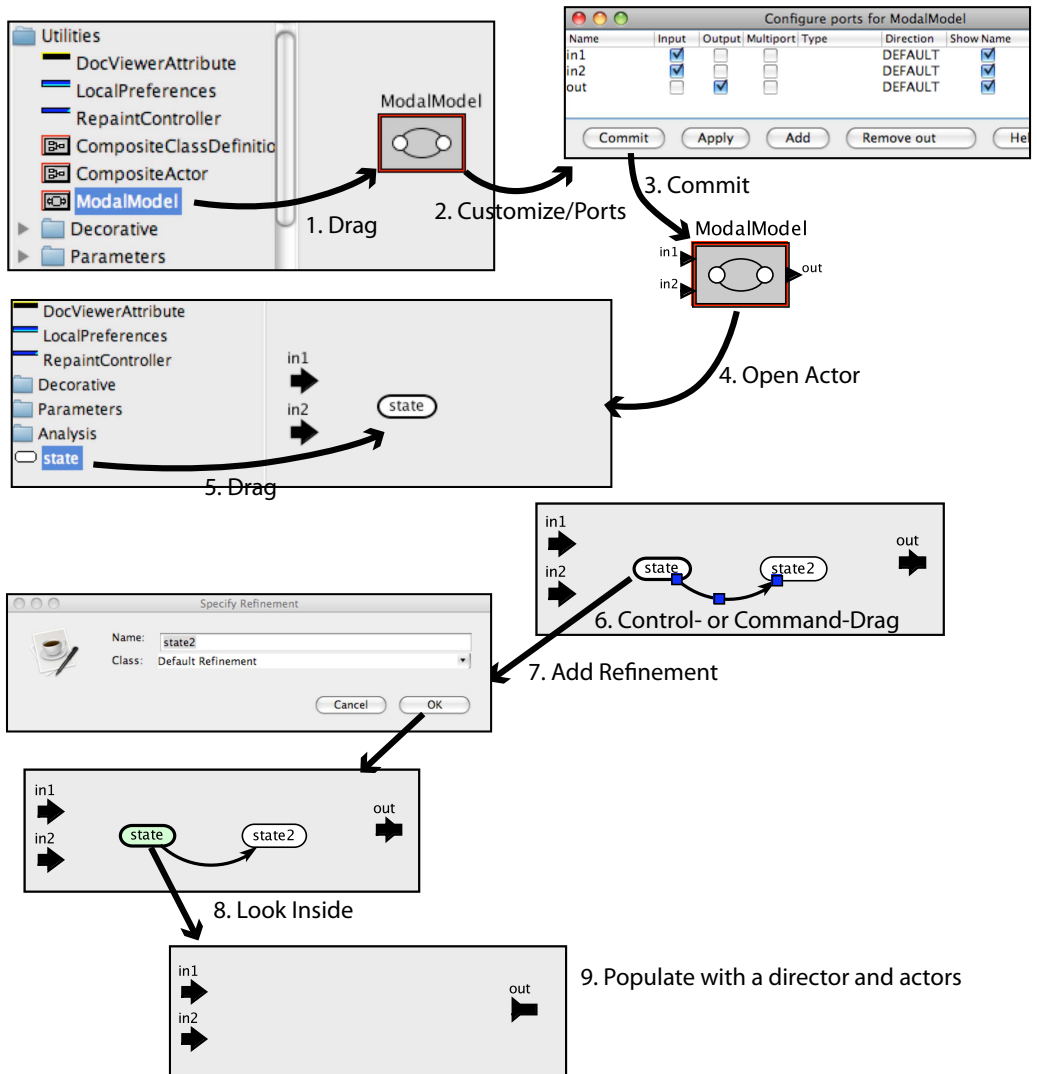


Figure 4.21: How to create modal models.

syntax in [Vergil](#) is as shown in Figure 4.22, which shows a red circle at the origin of the transition. In that figure, if the current state is `state1` and the expression `guard1` evaluates to `true`, then the refinement of `state1` will not be executed. Moreover, if `guard1` evaluates to `true`, then `guard2` will not be evaluated at all.

Probing Further: Internal Structure of a Modal Model

In Ptolemy II, every object (actor, state, transition, port, parameter, etc.) can have at most one container. In a modal model, two states can share the same refinement. This seems to violate the principle of “at most one container.”

The implementation in Ptolemy II, however, does not violate this principle. A [ModalModel](#) actor is actually a specialized composite actor that contains an instance `FSMDirector`, an [FSMACTOR](#), and any number of composite actors. Each composite actor is a candidate to be a refinement for any state of the [FSMACTOR](#). The [FSMACTOR](#) is the controller, in that it determines which mode is active at any time. The `FSMDirector` ensures that input data is delivered to the [FSMACTOR](#) and all active modes.

The [Vergil](#) user interface, however, hides this structure. When you execute an `Open Actor` command on a [ModalModel](#), the user interface actually skips a level of the hierarchy and takes you directly the [FSMACTOR](#) controller. It does not show the layer of the hierarchy that contains the [FSMACTOR](#), the [FSMDirector](#), and the refinements. Moreover, when you `Look Inside` a state, the user interface goes up one level of the hierarchy and opens *all* refinements of the selected state. This somewhat intricate architecture balances expressiveness with user convenience.

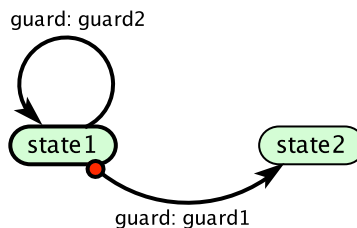


Figure 4.22: A preemptive transition, if chosen, results in the refinement not being executed. It is indicated by the red circle at the start of the transition.

4.2.6 Reset Transitions

A reset transition, if chosen, results in the refinement of the *destination* state being reset to its initial condition. It is indicated by the open arrowhead at the end of the transition, as shown in Figure 4.23. Specifically, after the transition has been taken, the `initialize()` method of the refinements of the destination state will be called. If, for example, the destination state has an FSM as its refinement, then the state of that refinement FSM will be set to the initial state.

A reset transition may be used to restart an FSM after it has reached a final state, as illustrated in the following example.

Example 4.10: We can use a final state to temporarily stop execution of a sub-model, and a reset transition to restart it. In Figure 4.24, the `ModalModel` has only a single state and a reset transition that is taken whenever the input is a multiple of 10. When this transition is taken, the refinement will be initialized. The refinement is an extended FSM that counts 5 inputs, copying them unchanged to the output each time. When 5 inputs have arrived, the refinement transitions to the final state and stops executing. Subsequent iterations will produce no output (the output of the `ModalModel` will be absent). When the refinement is re-initialized, then it will begin again, copying the next five inputs to the output, and then stopping again. For this model, assuming the `Ramp` actor produces the sequence 1, 2, 3, ..., then the output will be 1, 2, 3, 4, 5, followed by five absents, followed by 11, 12, 13, 14, 15, followed by another five absents, etc. The `SR` director is used here to make the iterations and absent values explicit.

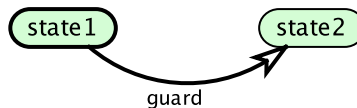


Figure 4.23: A reset transition, if chosen, results in the refinement of the destination state being reset to its initial condition. It is indicated by the open arrowhead at the end of the transition.

This use of final states is sometimes referred to in the literature as **normal termination**. The submodel stops executing when it enters a final state and can be restarted by a reset.

4.2.7 Transition Refinements

A transition may also have refinements. To create a **transition refinement**, right click on the transition and select Add Refinement. The syntax is shown in Figure 4.25. Specifically, a transition with a refinement is shown with a bolder line than a transition

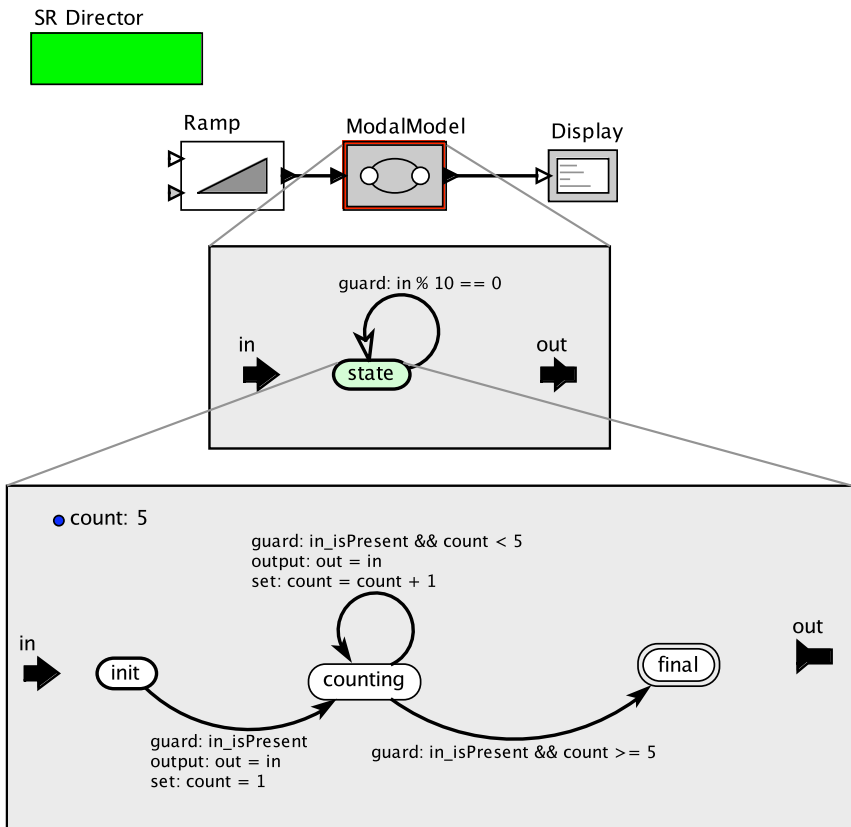


Figure 4.24: A reset transition may be used to restart an FSM after it has reached a final state.

without a refinement. The transition refinement fires when the transition is chosen and postfires when the transition is committed. When it fires, it can produce outputs.

Interestingly, a transition refinement can also take as inputs the outputs of the state refinement from which the transition originated. The way this works is that an output port (named, say *out*, as in the figure) has a sister port with “_in” appended to the name (*out_in* in the figure). That sister port provides whatever data the state refinements produced on that output port prior to the transition being chosen.

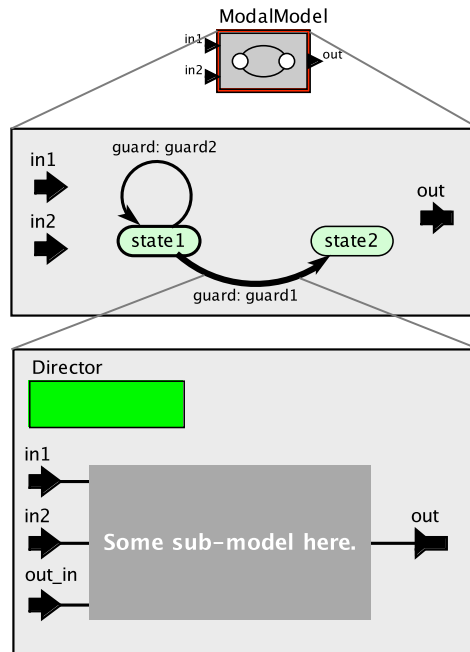


Figure 4.25: A transition refinement is a sub-model that fires when the transition is chosen and postfires when the transition is actually committed in the `postfire()` method of the container.

4.2.8 Execution Policy for Modal Models

Execution of a [ModalModel](#) is similar to the execution of an [FSMActor](#) as described in Section 4.1.1. In outline, in the `fire()` method, the [ModalModel](#) actor

1. reads inputs;
2. evaluates the guards of preemptive transitions out of the current state;
3. if no preemptive transition is enabled, the actor
 1. fires the refinements of the current state (if any); and
 2. evaluates guards on non-preemptive transitions out of the current state;
3. chooses a transition whose guard evaluates to true, giving preference to preemptive transitions;
4. executes the output actions of the chosen transition; and
5. fires the transition refinements of the chosen transition.

In `postfire()`, the [ModalModel](#) actor

1. postfires the refinements of the current state if they were fired;
2. executes the set actions of the chosen transition;
3. postfires the transition refinements of the chosen transition;
4. changes the current state to the destination of the chosen transition; and
5. initializes the refinements of the destination state if the transition is a reset transition.

The [ModalModel](#) actor makes no persistent state changes in its `fire()` method, so as long as the same is true of the refinement directors and actors, a modal model may be used in any domain. How it behaves in each domain, however, can be somewhat subtle, particularly with domains that have fixed-point semantics and when nondeterministic transitions are used. The behavior is exactly as described above for [FSMActor](#), with the only exception being that state and transition refinements are fired and postfired at appropriate times in the execution. One subtlety is that if a preemptive transition is enabled, then the guards of non-preemptive transitions will not be evaluated. Thus, nondeterminism never results from guards of a preemptive and a non-preemptive transition both becoming true in an iteration.

Note that state refinements are fired before any non-preemptive guard is evaluated. One consequence of this is that the outputs resulting from firing the refinement can be referenced in the guards of non-preemptive transitions! Thus, whether a transition is taken can depend on how the current refinement reacts to the inputs. In fact, the astute reader

may have already noticed in the figures here that when the controller `FSMActor` of a `ModalModel` is shown, that the icon for output ports does not look like a normal output port (see for example the *heat* port in the middle diagram of Figure 4.17). This icon, in fact, is the icon for a port that is both an input and an output. In fact, it serves both of these roles for the modal model controller, since it can influence the evaluation of guards and it can provide outputs to the environment.

In a firing, it is possible that the current state refinement produces an output, and a transition that is taken also produces an output on the same port. In this case, only the second of these outputs will appear on the output of the `ModalModel`. However, the first of these output values, the one produced by the refinement, may affect whether the transition is taken. That is, it can affect the guard. If in addition a transition refinement writes to the output, then that value will be produced, overwriting the value produced either by the state refinement or the output action on the transition.

4.2.9 Time and Modal Models

Many Ptolemy II directors implement a timed `model of computation`. The `ModalModel` and `FSMActor` are themselves untimed, but they have certain features to support their use in timed domains.

The FSMs we have described so far are **reactive**, meaning that they only produce outputs in reaction to inputs. In a timed domain, the inputs have `time stamps`. For a reactive FSM, the time stamps of the outputs are the same as the time stamps of the inputs. Thus, the FSM appears to be reacting in zero time. It is instantaneous, from the perspective of the timed domain.

However, in a timed domain, it is also possible to define spontaneous FSMs. A **spontaneous FSM** or **spontaneous modal model** is one that produces outputs even when inputs are absent.

Example 4.11: The model in Figure 4.26 switches between two modes every 2.5 time units. In the regular mode it generates a regularly-spaced clock signal with period 1.0 (and with value 1, the default output value for `DiscreteClock`). In the irregular mode, it generates randomly spaced events using a `PoissonClock` actor with a mean time between events set to 1.0 and value set to 2. The result of a typical run is plotted in Figure 4.27, with a shaded background showing the times

over which it is in the two modes. The output events from the [ModalModel](#) are spontaneous in that they are not necessarily produced in reaction to input events.

This example illustrates a number of subtle points about time. Examining the plot in [Figure 4.27](#), we see that an event with value 1 and another with value 2 is produced at time 0. Why? The initial state is regular, and the execution policy described in [section 4.2.8](#) explains that the refinement of that initial state is fired before guards are evaluated. That

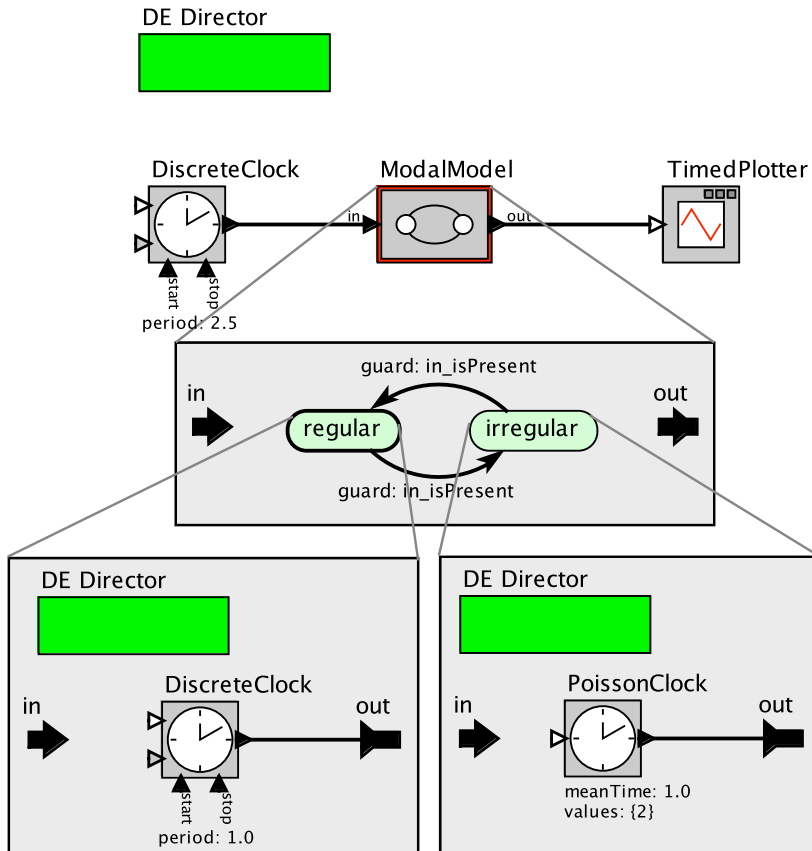


Figure 4.26: A spontaneous modal model, which produces output events that are not triggered by input events.

firing produces the first output of the [DiscreteClock](#). If we had instead used a preemptive transition, as shown in Figure 4.28, then that first output event would not appear.

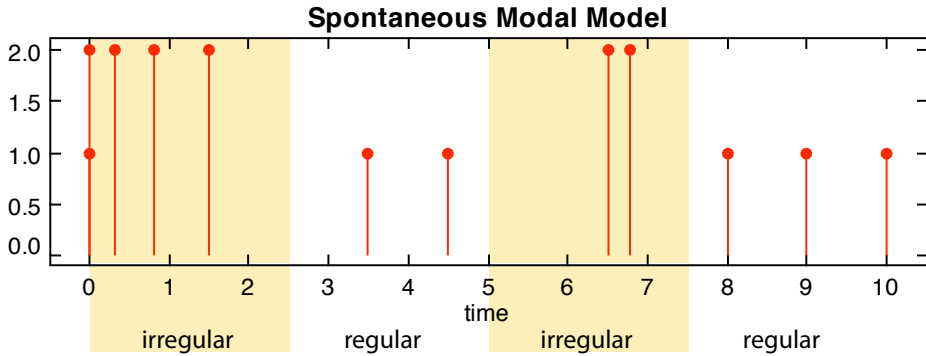


Figure 4.27: A plot of the output from one run of the model in Figure 4.26.

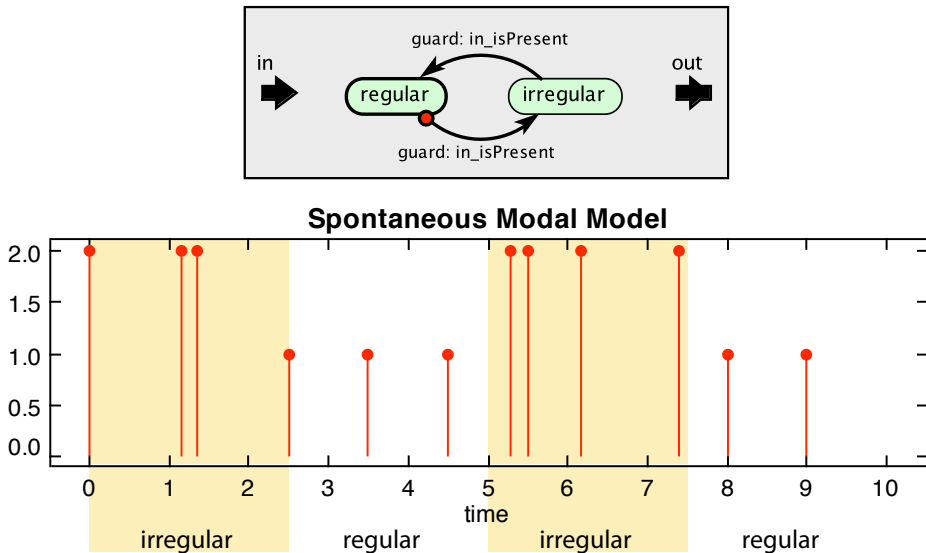


Figure 4.28: A variant of Figure 4.26 where a preemptive transition prevents the initial firing of the [DiscreteClock](#).

The second event in Figure 4.27 (with value 2) at time zero is produced because the `PoissonClock`, by default, produces an event at the time the execution starts, time zero. This event is produced in the second iteration of the `ModalModel`, after entering the irregular state. Although the event has the same time stamp as the first event (both occur at time zero), they have a well-defined ordering. The event with value 1 appears before the event with value 2. In Ptolemy II, the value of time is actually represented by a pair of numbers called a `tag`, $(t, n) \in \mathbb{R} \times \mathbb{N}$, rather than a single number. The first of these numbers, t , is called the `time stamp`. It approximates a real number (it is a quantized real number with a specified precision). See Appendix ?? . We interpret the time stamp t to represent the number of seconds (or any other time unit) since the start of execution of the model. The second of these numbers, n , is called the `time index` or `microstep`, and it represents a sequence number for events that occur at the same time stamp. In our example, the first event (with value 1) has tag $(0, 0)$, and the second event (with value 2) has tag $(0, 1)$. If we had set the `fireAtStart` parameter of the `PoissonClock` actor to `false`, then the second event would not occur.

Notice further that the `DiscreteClock` actor in the regular mode refinement has period 1.0, but produces events at times 0.0, 3.5, and 4.5, 8.0, 9.0, etc.. These are not multiples of 1.0 from the start time of the execution. Why?

The modal begins in the regular mode, but spends zero time there. It immediately transitions to the irregular mode. Hence, at time 0.0, the regular mode becomes inactive. While it is inactive, its local notion of time does not advance. It becomes active again at global time 2.5, but its local notion of time is still 0.0. Therefore, it has to wait one more time unit, until time 3.5, to produce the next output.

This notion of local time is important to understanding timed modal models. Very simply, local time stands still while a mode is inactive. Actors that refer to time, such as `Timed-Plotter` and `CurrentTime`, have a parameter `useLocalTime`, which defaults to `false`. Thus, by default, a plotter will always plot events on the global time line. If no actor accesses global time, however, then a mode refinement will be completely unaware that it was ever suspended. It does not appear as if time has elapsed. See Appendix ?? for a more complete description of local and global time.

Another interesting property of the output of this model is that no event is produced at time 5.0, when the irregular mode becomes active again. This follows from the same principle. The irregular mode became inactive at time 2.5, and hence, from time 2.5 to 5.0, its local notion of time has not advanced. When it becomes active again at time 5.0, it resumes

waiting for the right time (local time) to produce the next output from the [PoissonClock](#) actor.⁵

If an event is desired at time 5.0, then a [reset transition](#) can be used, as shown in Figure 4.29. The `initialize()` method of the [PoissonClock](#) causes an output event to be produced *at the time of the initialization*. A reset transition causes local time to match the environment time. The time lag between local time and environment time goes to zero.

4.2.10 Time Delays in Modal Models

Time delays interact with modal models in interesting ways, as illustrated by the next example.

⁵Interestingly, because of the memoryless property of a Poisson process, the time to the next event after becoming active is statistically identical to the time between events of the Poisson process. But this fact has little to do with the semantics of modal models.

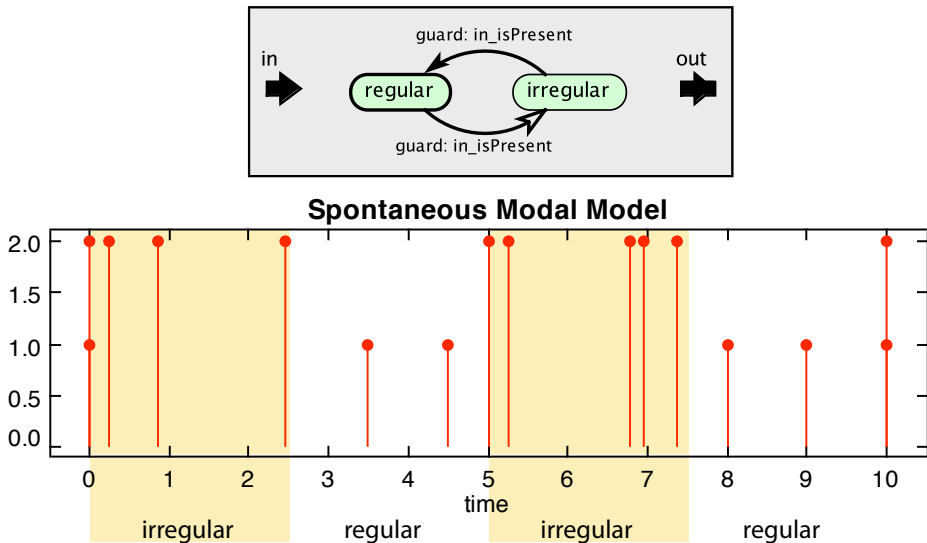


Figure 4.29: A variant of Figure 4.26 where a reset transition causes the Poisson-Clock to produce events when the irregular mode is reactivated.

Example 4.12: Figure 4.30 shows a model that produces a counting sequence of events spaced one time unit apart, and then alternately delays the events by one time unit and does not delay them. In the delay mode, a **TimeDelay** actor imposes a time delay of one time unit. In the noDelay mode, the input is sent directly to the output without delay. The result of executing this model is shown in Figure 4.31. Notice that the value 0 emerges at time 2. Why?

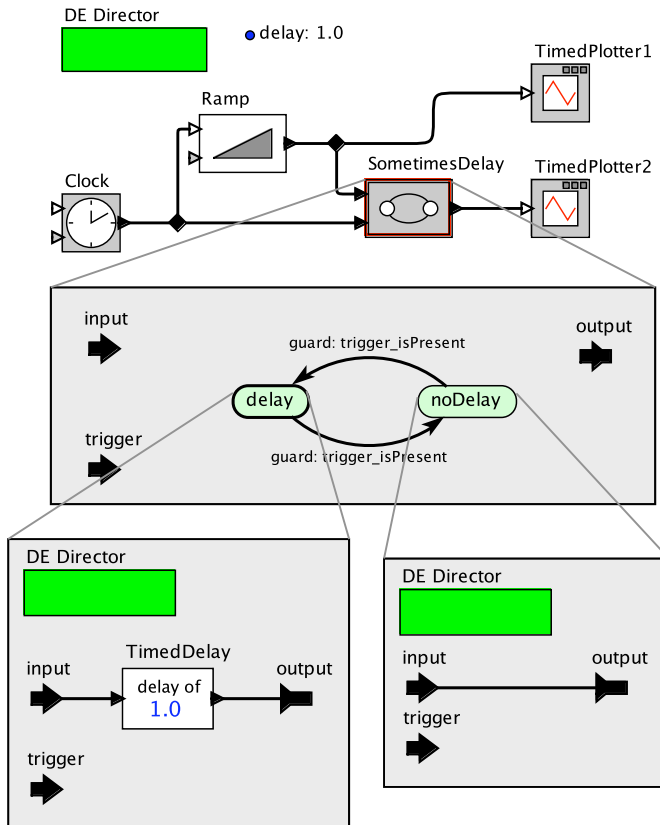


Figure 4.30: A modal model that switches between delaying the input by one time unit and not delaying it.

The model begins in the delay mode, so it is that mode that receives the first input, which has value 0. However, the modal model transitions immediately out of that mode to the noDelay mode. So no time elapses. The delay mode becomes active again at time 1, but at that time, its local time has not advanced. Local time is still 0. Therefore, it still has to delay the input with value 0 by one time unit. It produces that output therefore at time 2, just before transitioning out again to the noDelay mode.

4.2.11 Time in Transition Refinements

A [transition refinement](#) can also make reference to current time, but it makes little sense to have transition refinements that are not [reactive](#). Local time in a transition refinement will always be the same as local time in the environment in which the modal model containing the transition is executing. Hence, any access to local time via, for example, the [CurrentTime](#) actor will yield the time at which the transition is taken.

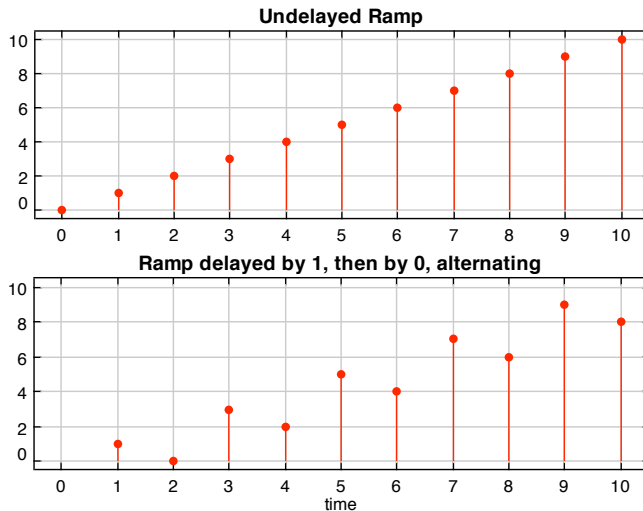


Figure 4.31: The result of executing the model in Figure 4.30.

Conceptually, a transition refinement is always active for exactly zero time. It takes no time to take a transition. As a consequence, it would not make much sense to include in a transition refinement spontaneous event generators like [DiscreteClock](#) or [PoissonClock](#). Neither does it make much sense to include time delay actors.

4.2.12 Time and FSMs

It is possible also to create a simple FSM (not a modal model) that is spontaneous in a sense, though it will not be able to produce outputs at arbitrary times. It can produce outputs at initialization time and can produce outputs with the same time stamp but higher indexes than input events.

Example 4.13: The model shown in Figure [4.32](#) includes an [FSMActor](#) that is both reactive and spontaneous. It produces an output at time zero (with value 0) despite the fact that it receives no input at time zero (the *fireAtStart* parameter of the [PoissonClock](#) actor is `false`). This output is produced because the transition out of the `init` state has guard `true`, and hence is taken immediately on initialization of the state machine. That transition has an output action, `out = 0`.

After initialization, when the first input event arrives (near time 0.7, with value 1), the input enables the transition from the `wait` to the `duplicate` state. That transition copies the value of the input to the output and also records the value of the input in a local variable *recordedInput*. Once the state machine has entered the `duplicate` state, the transition back to `wait` is immediately enabled, so it is taken in the next microstep. Thus, the state machine produces a second output at the same time (around 0.7), as shown in the plot. The second output has twice the value of the first, making both outputs visible in the plot.

A state in which the state machine spends zero time is called a **transient state**. In the previous example, both the `init` and the `duplicate` states are transient. Note that any state that has default transitions (without guards or with guards that evaluate to `true` immediately)

is a transient state, since exiting the state is always immediately enabled after entering the state.

4.2.13 Modal Model Principles

Modal models are clearly subtle and expressive, particularly for timed models. It is useful to step back and ponder the principles that govern the design choices in the Ptolemy II

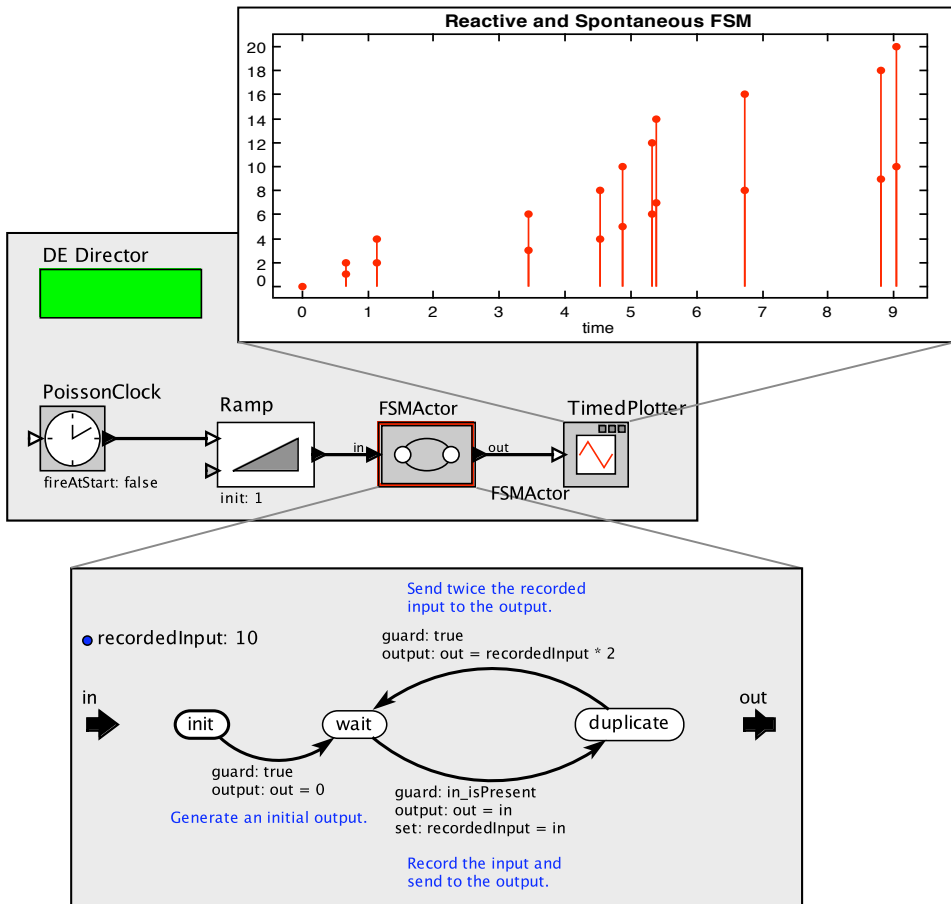


Figure 4.32: An FSM that spontaneously produces events at the start time and at the times of input events.

implementation of modal models. The key idea behind a mode is that it specifies a portion of the system that is active only part of the time. When it is inactive, does it cease to exist? Does time pass? Can its state evolve? These are not easy questions because the desired behavior depends very much on the application.

In Ptolemy II, the guiding principle is that when a mode is inactive, local time stands still, but global time passes. An inactive mode is therefore in a state of suspended animation. Local time within a mode will lag the time in its environment. The amount of lag begins at zero, and increases each time the mode becomes inactive. When an event crosses a hierarchical boundary into or out of the mode, its time stamp is adjusted by the amount of the lag. Thus, within the mode, time seems uninterrupted.

A key point is that being inactive is not necessarily the same as not getting inputs and not being observed. This point is illustrated in the model of Figure 4.33, which shows two instances of [DiscreteClock](#), labeled `DiscreteClock1` and `DiscreteClock2`, which have the same parameter values. `DiscreteClock2` is inside a modal model labeled `ModalClock`, and `DiscreteClock1` is outside of any modal model. The output of `DiscreteClock1` is filtered by a modal model labeled `ModalFilter` that selectively passes the input to the output. The two modal models are controlled by the same `ControlClock`, which determines when they switch between the active and inactive states. Three plots are shown. The top plot is simply the output of `DiscreteClock1` unmodified in any way. The middle plot is the result of switching between observing and not observing the output of `Discrete-`

Probing Further: Implementation of Transient States

When a transition is taken in an FSM, the [FSMActor](#) or [ModalModel](#) calls `fireAtCurrentTime()` on its enclosing director. This method requests a new firing in the next microstep regardless of whether any additional inputs become available. If the director respects this request (normally timed directors do), then the actor will be fired again at the current time, one microstep later. This ensures that if the destination state has a transition that is immediately enabled (in the next microstep), then that transition will be taken immediately. Note also that in a modal model, if the destination state has a refinement, then that refinement will be fired at the current time in the next microstep. This is particularly useful for continuous-time models, since the transition may represent a discontinuity in otherwise continuous signals. The discontinuity translates into two distinct events with the same time stamp. See chapter ??.

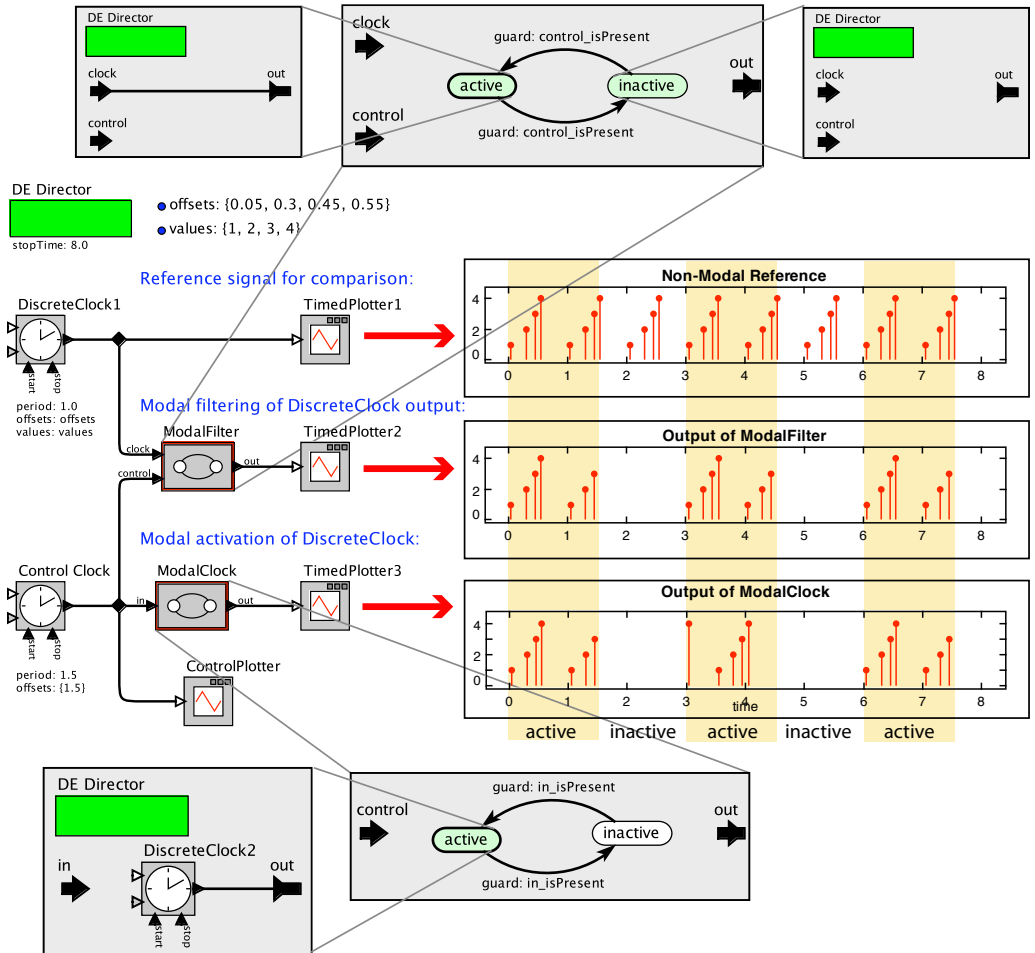


Figure 4.33: A model that illustrates that putting a stateful timed actor such as DiscreteClock inside a modal model is not the same switching between observing and not observing its output.

Clock1. The bottom plot is the result of activating and deactivating DiscreteClock2, which is otherwise identical to DiscreteClock1.

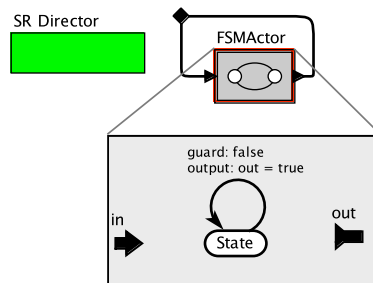
The [DiscreteClock](#) actors in this example are set to produce a sequence of values, 1, 2, 3, 4, cyclically. Consequently, in addition to being timed, these actors have state, since they need to recall which was the last output value in order to produce the next output value. When DiscreteClock2 is inactive, its state does not change, and time does not advance. Thus, when it becomes active again, it simply resumes where it left off.

4.3 Conclusion

FSMs and modal models in Ptolemy II provide a very expressive way to build up complex model behaviors. As a consequence of this expressiveness, it takes some practice to learn to use them well. This chapter is intended to provide a reasonable starting point. Readers who wish to probe further are encouraged to examine the documentation for the Java classes that implement these mechanisms. Many of these are accessible when running Vergil by right clicking and selecting `Documentation`.

Exercises

- Consider a variant of the thermostat of example 4.2. In this variant, there is only one temperature threshold, and to avoid chattering the thermostat simply leaves the heat on or off for at least a fixed amount of time. In the initial state, if the temperature is less than or equal to 20 degrees Celsius, it turns the heater on, and leaves it on for at least 30 seconds. If the temperature is greater than 20 degrees, it turns the heater off and leaves it off for at least 2 minutes. In both cases, once the two minutes have elapsed, it returns to the initial state.
 - Create a Ptolemy II model of this thermostat. You may use an SDF director and assume that it runs at a rate of one iteration per second.
 - How many possible states does your thermostat have? (**Careful!** The number of states should include the number of possible valuations of any local variables.)
 - Does this model thermostat have the [hysteresis](#) property?
- In the following model, the only signal (going from the output of [FSMActor](#) back to its input) has value `absent` at all ticks.



Explain why this is correct.

- Construct a variant of the example in Figure 4.13 where the clean and noisy states share the same refinement, yet the behavior is the same.
- A common scenario in embedded systems is where a component *A* in the system monitors the health of another component *B* and raises an alarm. Assume *B* provides sensor data as timed events. Component *A* will use a local clock to provide a regular stream of local timed events. If component *B* fails to send sensor data to component *A* at least once in each clock interval, then something may be wrong.

- (a) Design an FSM called *MissDetector* with two input ports, *sensor* and *clock*, and two output ports *missed* and *ok*. Your FSM should produce an event on *missed* when two *clock* events arrive without an intervening *sensor* event. It should produce an *ok* event when the first *sensor* event after (or coincident with) a *clock* event arrives.
- (b) Design a second FSM called *StatusClassifier* that takes inputs from your first FSM and decides whether component *B* is operating normally. Specifically, it should enter a warning state if it receives *warningThreshold missed* events without an intervening *ok* event, where *warningThreshold* is a parameter. Moreover, once it enters a warning state, it should remain in that state until at least *normalThreshold ok* events arrive without another intervening *ok*, where *normalThreshold* is another parameter.
- (c) Comment about the precision and clarity of the English-language specification of the behavior in this problem, compared to your state machine implementation. In particular, find at least one ambiguity in the above specification and explain how your model interprets it.

Signal Display

Christopher X. Brooks and Edward A. Lee

Ptolemy II includes a number of **signal plotters**. These can be found in the `Sinks` library, as shown in Figure A.1. This appendix gives an overview of these capabilities, with emphasis on how to customize the plots. Once plots have been customized, saving the model containing the plotter will make the customization persistent.

A.1 Overview of Available Plotters

The plotters shown in Figure A.1 provide a number of different capabilities, and are all built on a common infrastructure. The most basic is **SequencePlotter**, which simply plots data values received on the input port. An example of a plot is shown in Figure A.2. The mechanisms for customizing the title, axes, legend and signal plots will be covered in the following section.

The actors shown in the libraries in Figure A.1 provide a variety of ways of displaying data. Whereas the **SequencePlotter** actor plots all of the input data over the entire run of the the model, the **SequenceScope** actor plots windows of input data, optionally overlaying them, as shown in Figure A.3. The **SequenceScope** actor is more useful with long or infinite runs.

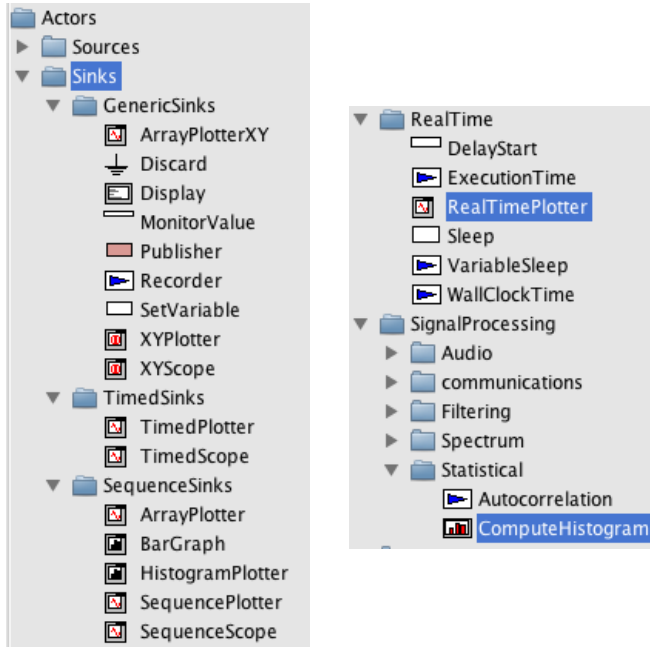


Figure A.1: A variety of signal plotters can be found in the `Sinks` library.

The **TimedPlotter** actor plots input data as a function of the time stamps of the inputs, as shown in Figure A.4. This plotter is only useful in domains that advance *model time*. The **XYPlotter** actor plots input data from one input port vs. input data from its other input port, as shown in Figure A.5. Finally, the **ComputeHistogram** actor which calculates and plots a histogram based on input data, as shown in Figure A.6.

A.2 Customizing a Plot

When used with its default configuration, the `SequencePlotter` will produce a plot like that shown in Figure A.7, which is also shown in Figure ???. The default title is uninformative, the axes are not labeled, and the horizontal axis ranges from 0 to 255, which

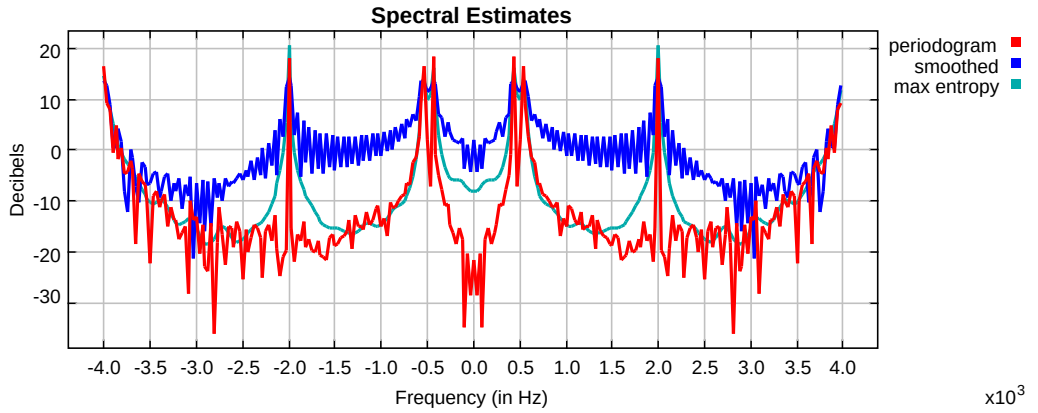


Figure A.2: Example of a plot produced by the SequencePlotter actor.

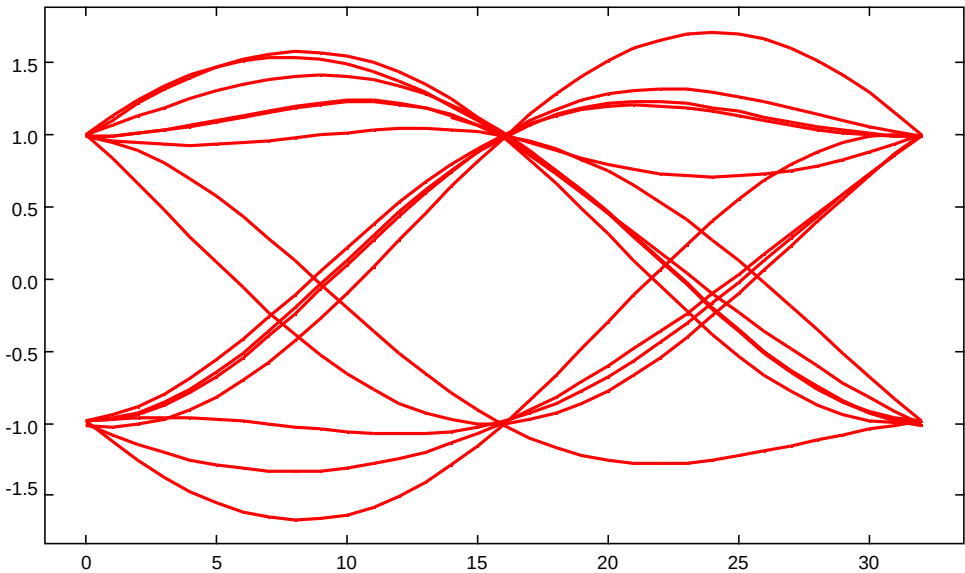


Figure A.3: Example of a plot produced by the SequenceScope actor, which overlays successive windows of data, like what an oscilloscope does.

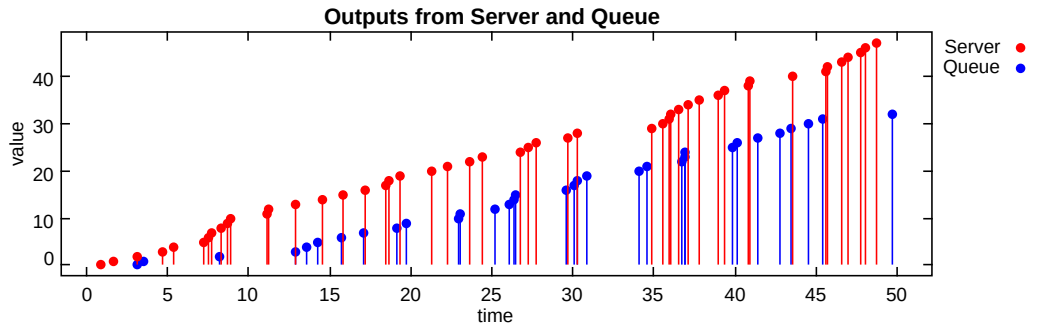


Figure A.4: Example of a plot produced by the TimedPlotter actor, which plots input data as a function of the time stamps of the inputs.

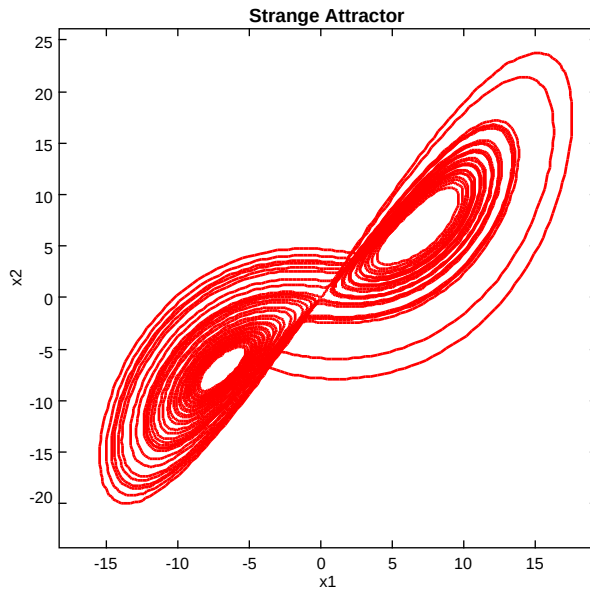


Figure A.5: Example of a plot produced by the XYPlotter actor, which plots input data on one input port vs. input data on the other port.

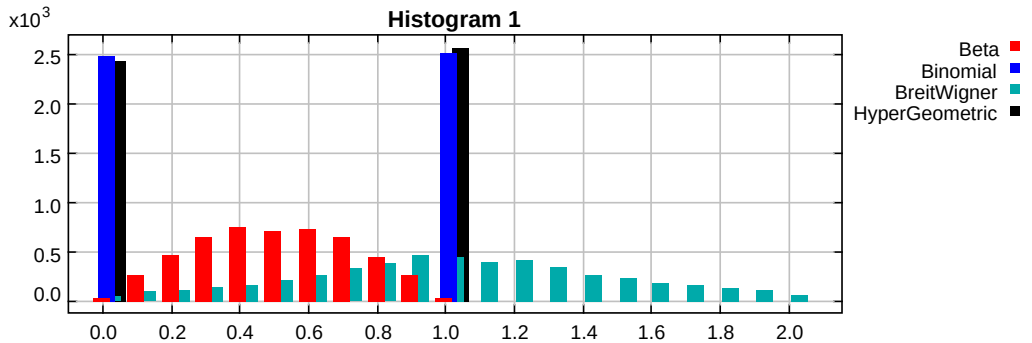


Figure A.6: Example of a plot produced by the ComputeHistogram actor, which calculates and plots a histogram based on input data.

is not meaningful.¹ In the model that created this plot, which is shown in Figure ??, in one iteration, the Spectrum actor produces 256 output tokens. By default, the SequencePlotter just numbers these samples 0 to 255, and uses those numbers as the horizontal axis for the plot. But the horizontal axis may have more meaning in the model. In this particular example plot, the plotted data represent frequency bins that range between $-\pi$ and π radians per second.

The SequencePlotter actor has some pertinent parameters, shown in Figure A.8, that can be used to improve the labeling of the plot. The *xInit* parameter specifies the value to use on the horizontal axis for the first token. The *xUnit* parameter specifies the value to increment this by for each subsequent token. Setting these to “-PI” and “PI/128” respectively results in the plot shown in Figure A.9.

This plot is better, but still missing useful information. To control more precisely the visual appearance of the plot, click on the second button from the right in the row of buttons at the top right of the plot. This button brings up a format control window. It is shown in Figure A.10, filled in with values that result in the plot shown in Figure A.11. Most of these are self-explanatory, but the following pointers may be useful:

- The grid is turned off to reduce clutter.
- Titles and axis labels have been added.

¹Hint: Notice the “ $\times 10^2$ ” at the bottom right, which indicates that the label “2.5” stands for “250”.

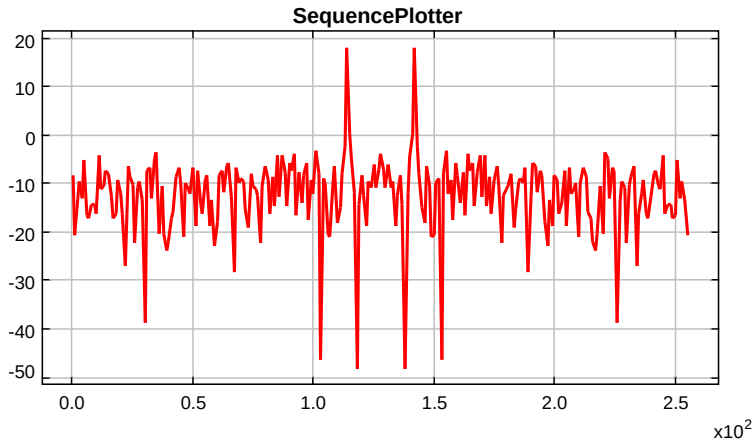


Figure A.7: In its default configuration, a plot produced by SequencePlotter does not have informative labels.

fillOnWrapup:	☒
automaticRescale:	☐
legend:	
startingDataset:	0
xInit:	$-\pi$
xUnit:	$\pi/128$
firingsPerIteration:	256

Figure A.8: Parameters for the SequencePlotter actor.

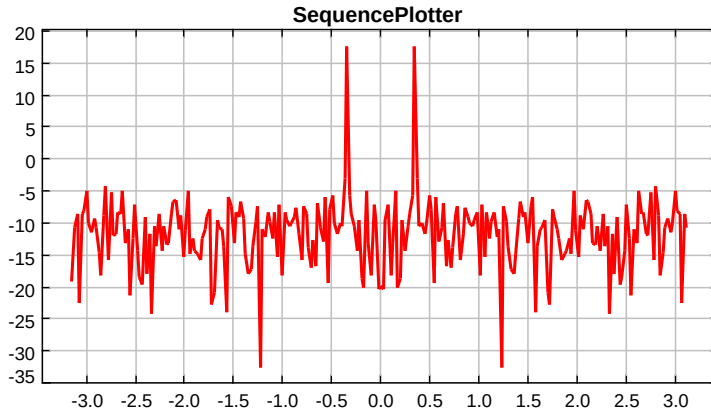


Figure A.9: Better labeled plot, where the horizontal axis now properly represents the frequency values.

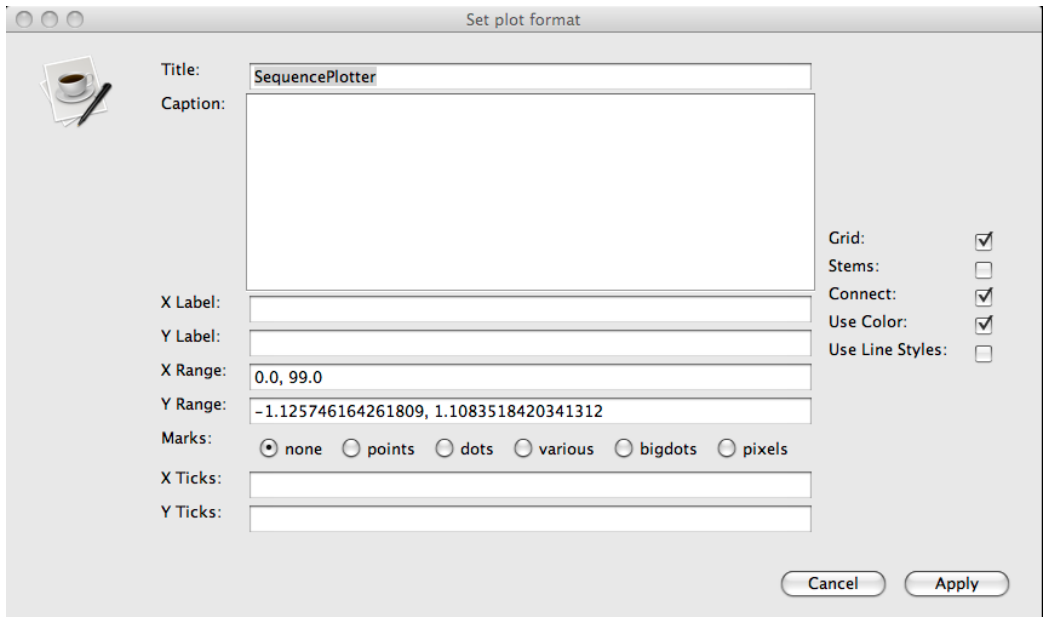


Figure A.10: Format control window for a plot.

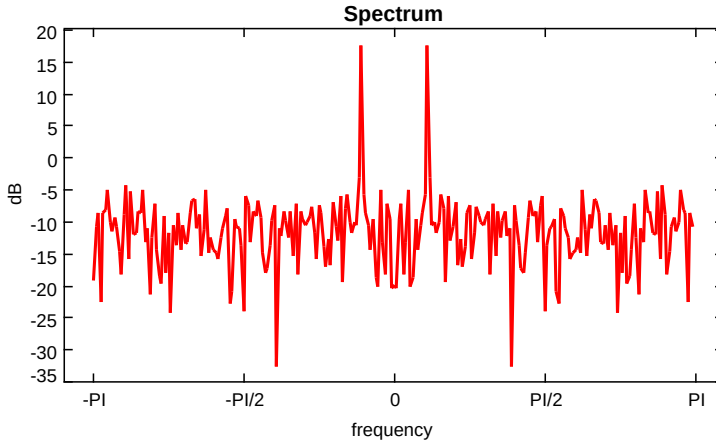


Figure A.11: Still better labeled plot.

- The X range and Y range are determined by the fill button at the upper right of the plot.
- Stem plots can be had by clicking on “Stems”
- Individual tokens can be shown by clicking on “dots”
- Connecting lines can be eliminated by deselecting “connect”
- The X axis label has been changed to symbolically indicate multiples of $\pi/2$. This is done by entering the following in the X Ticks field:

`-PI -3.14159, -PI/2 -1.570795, 0 0.0, PI/2 1.570795, PI 3.14159`

The syntax in general is: *label value, label value, ...*, where the label is any string (enclosed in quotation marks if it includes spaces), and the value is a number.

Bibliography

- Agha, G. A., I. A. Mason, S. F. Smith, and C. L. Talcott, 1997: A foundation for actor computation. *Journal of Functional Programming*, **7**(1), 1–72.
- Alur, R., S. Kannan, and M. Yannakakis, 1999: Communicating hierarchical state machines. In *26th International Colloquium on Automata, Languages, and Programming*, Springer, vol. LNCS 1644, pp. 169–178.
- André, C., 1996: SyncCharts: a visual representation of reactive behaviors. Tech. Rep. RR 95–52, revision: RR (96–56), University of Sophia-Antipolis. Available from: <http://www-sop.inria.fr/members/Charles.Andre/CA%20Publis/SYNCCHARTS/overview.html>.
- Baccelli, F., G. Cohen, G. J. Olster, and J. P. Quadrat, 1992: *Synchronization and Linearity, An Algebra for Discrete Event Systems*. Wiley, New York.
- Baier, C. and M. E. Majster-Cederbaum, 1994: Denotational semantics in the CPO and metric approach. *Theoretical Computer Science*, **135**(2), 171–220.
- Baldwin, P., S. Kohli, E. A. Lee, X. Liu, and Y. Zhao, 2004: Modeling of sensor nets in Ptolemy II. In *Information Processing in Sensor Networks (IPSN)*, Berkeley, CA, USA.
- , 2005: Visualsense: Visual modeling for wireless and sensor network systems. Technical Report UCB/ERL M05/25, EECS Department, University of California.

- Basu, A., M. Bozga, and J. Sifakis, 2006: Modeling heterogeneous real-time components in BIP. In *International Conference on Software Engineering and Formal Methods (SEFM)*, Pune, pp. 3–12.
- Benveniste, A. and G. Berry, 1991: The synchronous approach to reactive and real-time systems. *Proceedings of the IEEE*, **79(9)**, 1270–1282.
- Berry, G., 2003: The effectiveness of synchronous languages for the development of safety-critical systems. White paper, Esterel Technologies. Available from: <http://www.esterel-technologies.com>.
- Berry, G. and G. Gonthier, 1992: The Esterel synchronous programming language: Design, semantics, implementation. *Science of Computer Programming*, **19(2)**, 87–152. Available from: <http://citeseer.nj.nec.com/berry92esterel.html>.
- Bock, C., 2006: SysML and UML 2 support for activity modeling. *Systems Engineering*, **9(2)**, 160–185.
- Booch, G., I. Jacobson, and J. Rumbaugh, 1998: *The Unified Modeling Language User Guide*. Addison-Wesley.
- Brooks, C., C. Cheng, T. H. Feng, E. A. Lee, and R. von Hanxleden, 2008: Model engineering using multimodeling. In *International Workshop on Model Co-Evolution and Consistency Management (MCCM)*, Toulouse, France.
- Broy, M., 1983: Applicative real time programming. In *Information Processing 83, IFIP World Congress*, North Holland Publ. Company, Paris, pp. 259–264.
- Bryant, V., 1985: *Metric Spaces - Iteration and Application*. Cambridge University Press.
- Buck, J. T., S. Ha, E. A. Lee, and D. G. Messerschmitt, 1994: Ptolemy: A framework for simulating and prototyping heterogeneous systems. *Int. Journal of Computer Simulation, special issue on "Simulation Software Development"*, **4**, 155–182. Available from: <http://ptolemy.eecs.berkeley.edu/publications/papers/94/JEurSim/>.
- Burch, J. R., R. Passerone, and A. L. Sangiovanni-Vincentelli, 2001: Overcoming heterophobia: Modeling concurrency in heterogeneous systems. In *International Conference on Application of Concurrency to System Design*, p. 13.

- Carloni, L. P., R. Passerone, A. Pinto, and A. Sangiovanni-Vincentelli, 2006: Languages and tools for hybrid systems design. *Foundations and Trends in Electronic Design Automation*, **1**(1/2). doi:10.1561/10000000001.
- Cassandras, C. G., 1993: *Discrete Event Systems, Modeling and Performance Analysis*. Irwin.
- Cataldo, A., E. A. Lee, X. Liu, E. Matsikoudis, and H. Zheng, 2006: A constructive fixed-point theorem and the feedback semantics of timed systems. In *Workshop on Discrete Event Systems (WODES)*, Ann Arbor, Michigan. Available from: <http://ptolemy.eecs.berkeley.edu/publications/papers/06/constructive/>.
- Chandy, K. M. and J. Misra, 1979: Distributed simulation: A case study in design and verification of distributed programs. *IEEE Trans. on Software Engineering*, **5**(5), 440–452.
- de Alfaro, L. and T. Henzinger, 2001: Interface automata. In *ESEC/FSE 01: the Joint 8th European Software Engineering Conference and 9th ACM SIGSOFT International Symposium on the Foundations of Software Engineering*.
- Eker, J., J. W. Janneck, E. A. Lee, J. Liu, X. Liu, J. Ludvig, S. Neuendorffer, S. Sachs, and Y. Xiong, 2003: Taming heterogeneity—the Ptolemy approach. *Proceedings of the IEEE*, **91**(2), 127–144. Available from: <http://www.ptolemy.eecs.berkeley.edu/publications/papers/03/TamingHeterogeneity/>.
- Encyclopedia Britannica, 2010: Ockham’s razor. *Encyclopedia Britannica Online*, Retrieved June 24, 2010. Available from: <http://www.britannica.com/EBchecked/topic/424706/Ockhams-razor>.
- Feredj, M., F. Boulanger, and A. M. Mbobi, 2009: A model of domain-polymorph component for heterogeneous system design. *The Journal of Systems and Software*, **82**, 112120.
- Fritzson, P., 2003: *Principles of Object-Oriented Modeling and Simulation with Modelica 2.1*. Wiley.
- Fujimoto, R., 2000: *Parallel and Distributed Simulation Systems*. John Wiley and Sons.
- Girault, A., B. Lee, and E. A. Lee, 1999: Hierarchical finite state machines with multiple concurrency models. *IEEE Transactions On Computer-aided Design Of Integrated Circuits And Systems*, **18**(6), 742–760.

- Goderis, A., C. Brooks, I. Altintas, E. A. Lee, and C. Goble, 2009: Heterogeneous composition of models of computation. *Future Generation Computer Systems*, **25(5)**, 552–560. doi:[doi:10.1016/j.future.2008.06.014](https://doi.org/10.1016/j.future.2008.06.014).
- Goessler, G. and A. Sangiovanni-Vincentelli, 2002: Compositional modeling in Metropolis. In *Second International Workshop on Embedded Software (EMSOFT)*, Springer-Verlag, Grenoble, France.
- Hardebolle, C. and F. Boulanger, 2007: ModHel’X: A component-oriented approach to multi- formalism modeling. In *MODELS 2007 Workshop on Multi- Paradigm Modeling*, Elsevier Science B.V., Nashville, Tennessee, USA.
- Harel, D., 1987: Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, **8(3)**, 231–274.
- Harel, D., H. Lachover, A. Naamad, A. Pnueli, M. Politi, R. Sherman, A. Shtull-Trauring, and M. Trakhtenbrot, 1990: STATEMATE: A working environment for the development of complex reactive systems. *IEEE Transactions on Software Engineering*, **16(4)**, 403 – 414. doi:[doi:10.1109/32.54292](https://doi.org/10.1109/32.54292).
- Henzinger, T. A., 2000: The theory of hybrid automata. In Inan, M. and R. Kurshan, eds., *Verification of Digital and Hybrid Systems*, Springer-Verlag, vol. 170 of *NATO ASI Series F: Computer and Systems Sciences*, pp. 265–292.
- Henzinger, T. A., B. Horowitz, and C. M. Kirsch, 2001: Giotto: A time-triggered language for embedded programming. In *EMSOFT 2001*, Springer-Verlag, Tahoe City, CA, vol. LNCS 2211, pp. 166–184.
- Herrera, F. and E. Villar, 2006: A framework for embedded system specification under different models of computation in SystemC. In *Design Automation Conference (DAC)*, ACM, San Francisco.
- Hewitt, C., 1977: Viewing control structures as patterns of passing messages. *Journal of Artificial Intelligence*, **8(3)**, 323–363.
- Hoare, C. A. R., 1978: Communicating sequential processes. *Communications of the ACM*, **21(8)**, 666–677.
- Hopcroft, J. and J. Ullman, 1979: *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, Reading, MA.

- Jantsch, A., 2003: *Modeling Embedded Systems and SoCs - Concurrency and Time in Models of Computation*. Morgan Kaufmann.
- Jefferson, D., 1985: Virtual time. *ACM Trans. Programming Languages and Systems*, **7(3)**, 404–425.
- Kahn, G. and D. B. MacQueen, 1977: Coroutines and networks of parallel processes. In Gilchrist, B., ed., *Information Processing*, North-Holland Publishing Co., pp. 993–998.
- Kienhuis, B., E. Deprettere, P. van der Wolf, and K. Vissers, 2001: A methodology to design programmable embedded systems. In Deprettere, E., J. Teich, and S. Vassiliadis, eds., *Systems, Architectures, Modeling, and Simulation (SAMOS)*, Springer-Verlag, vol. LNCS 2268.
- Lee, E. A., 1999: Modeling concurrent real-time processes using discrete events. *Annals of Software Engineering*, **7**, 25–45. doi:10.1023/A:1018998524196.
- , 2006: The problem with threads. *Computer*, **39(5)**, 33–42. doi:10.1109/MC.2006.180.
- , 2008: Threadedcomposite: A mechanism for building concurrent and parallel ptolemy ii models. Technical Report UCB/EECS-2008-151, EECS Department, University of California, Berkeley. Available from: <http://www.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-151.html>.
- , 2009: Finite state machines and modal models in Ptolemy II. Tech. Rep. UCB/EECS-2009-151, EECS Department, University of California, Berkeley. Available from: <http://www.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-151.html>.
- Lee, E. A., X. Liu, and S. Neuendorffer, 2009: Classes and inheritance in actor-oriented design. *ACM Transactions on Embedded Computing Systems (TECS)*, **8(4)**, 29:1–29:26. doi:10.1145/1550987.1550992.
- Lee, E. A. and E. Matsikoudis, 2009: The semantics of dataflow with firing. In Huet, G., G. Plotkin, J.-J. Lévy, and Y. Bertot, eds., *From Semantics to Computer Science: Essays in memory of Gilles Kahn*, Cambridge University Press. Available from: <http://ptolemy.eecs.berkeley.edu/publications/papers/08/DataflowWithFiring/>.

- Lee, E. A. and D. G. Messerschmitt, 1987: Synchronous data flow. *Proceedings of the IEEE*, **75(9)**, 1235–1245. doi:10.1109/PROC.1987.13876.
- Lee, E. A. and S. Neuendorffer, 2000: MoML - a modeling markup language in XML. Tech. Rep. UCB/ERL M00/12, UC Berkeley.
- Lee, E. A., S. Neuendorffer, and M. J. Wirthlin, 2003: Actor-oriented design of embedded hardware and software systems. *Journal of Circuits, Systems, and Computers*, **12(3)**, 231–260.
- Lee, E. A. and T. M. Parks, 1995: Dataflow process networks. *Proceedings of the IEEE*, **83(5)**, 773–801. doi:10.1109/5.381846.
- Lee, E. A. and A. Sangiovanni-Vincentelli, 1998: A framework for comparing models of computation. *IEEE Transactions on Computer-Aided Design of Circuits and Systems*, **17(12)**, 1217–1229. Available from: <http://ptolemy.eecs.berkeley.edu/publications/papers/98/framework/>.
- Lee, E. A. and S. Tripakis, 2010: Modal models in Ptolemy. In *3rd International Workshop on Equation-Based Object-Oriented Modeling Languages and Tools (EOOLT)*, Linköping University Electronic Press, Linköping University, Oslo, Norway, vol. 47, pp. 11–21. Available from: <http://chess.eecs.berkeley.edu/pubs/700.html>.
- Lee, E. A. and H. Zheng, 2005: Operational semantics of hybrid systems. In Morari, M. and L. Thiele, eds., *Hybrid Systems: Computation and Control (HSCC)*, Springer-Verlag, Zurich, Switzerland, vol. LNCS 3414, pp. 25–53. doi:10.1007/978-3-540-31954-2_2.
- , 2007: Leveraging synchronous language principles for heterogeneous modeling and design of embedded systems. In *EMSOFT*, ACM, Salzburg, Austria, pp. 114 – 123. doi:10.1145/1289927.1289949.
- Liu, X. and E. A. Lee, 2008: CPO semantics of timed interactive actor networks. *Theoretical Computer Science*, **409(1)**, 110–125. doi:10.1016/j.tcs.2008.08.044.
- Liu, X., E. Matsikoudis, and E. A. Lee, 2006: Modeling timed concurrent systems. In *CONCUR 2006 - Concurrency Theory*, Springer, Bonn, Germany, vol. LNCS 4137, pp. 1–15. doi:10.1007/11817949_1.

- Lynch, N., R. Segala, F. Vaandrager, and H. Weinberg, 1996: Hybrid I/O automata. In Alur, R., T. Henzinger, and E. Sontag, eds., *Hybrid Systems III*, Springer-Verlag, vol. LNCS 1066, pp. 496–510.
- Maler, O., Z. Manna, and A. Pnueli, 1992: From timed to hybrid systems. In *Real-Time: Theory and Practice, REX Workshop*, Springer-Verlag, pp. 447–484.
- Manna, Z. and A. Pnueli, 1993: Verifying hybrid systems. In *Hybrid Systems*, vol. LNCS 736, pp. 4–35.
- Maraninchi, F. and T. Bhouhadiba, 2007: 42: Programmable models of computation for a component-based approach to heterogeneous embedded systems. In *6th ACM International Conference on Generative Programming and Component Engineering (GPCE)*, Salzburg, Austria, pp. 1–3.
- Mosterman, P. J. and H. Vangheluwe, 2004: Computer automated multi-paradigm modeling: An introduction. *Simulation: Transactions of the Society for Modeling and Simulation International Journal of High Performance Computing Applications*, **80**(9), 433450.
- Object Management Group (OMG), 2008a: System modeling language specification v1.1. Tech. rep., OMG. Available from: <http://www.sysmlforum.com>.
- , 2008b: A UML profile for MARTE, beta 2. OMG Adopted Specification ptc/08-06-09, OMG. Available from: <http://www.omg.org/omgmarte/>.
- Patel, H. D. and S. K. Shukla, 2004: *SystemC Kernel Extensions for Heterogeneous System Modelling*. Kluwer.
- Pree, W. and J. Templ, 2006: Modeling with the timing definition language (TDL). In *Automotive Software Workshop San Diego (ASWSD) on Model-Driven Development of Reliable Automotive Services*, Springer, San Diego, CA, LNCS.
- Prochnow, S. and R. von Hanxleden, 2007: Statechart development beyond WYSIWYG. In *International Conference on Model Driven Engineering Languages and Systems (MoDELS)*, ACM/IEEE, Nashville, TN, USA.
- Ramadge, P. and W. Wonham, 1989: The control of discrete event systems. *Proceedings of the IEEE*, **77**(1), 81–98.

- Reed, G. M. and A. W. Roscoe, 1988: Metric spaces as models for real-time concurrency. In *3rd Workshop on Mathematical Foundations of Programming Language Semantics*, London, UK, pp. 331–343.
- Sander, I. and A. Jantsch, 2004: System modeling and transformational design refinement in ForSyDe. *IEEE Transactions on Computer-Aided Design of Circuits and Systems*, **23**(1), 17–32.
- Shapiro, F. R., 2006: *The Yale Book of Quotations*. Yale University Press.
- Smith, N. K., 1929: *Immanuel Kant's Critique of Pure Reason*. Macmillan and Co. Available from: <http://www.hkbu.edu.hk/~ppp/cpr/toc.html>.
- von der Beeck, M., 1994: A comparison of Statecharts variants. In Langmaack, H., W. P. de Roever, and J. Vytöpil, eds., *Third International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems*, Springer-Verlag, Lübeck, Germany, vol. 863 of *Lecture Notes in Computer Science*, pp. 128–148.
- Xiong, Y., 2002: An extensible type system for component-based design. Ph.D. Thesis Technical Memorandum UCB/ERL M02/13, University of California, Berkeley, CA 94720. Available from: <http://ptolemy.eecs.berkeley.edu/papers/02/typeSystem>.
- Yates, R. K., 1993: Networks of real-time processes. In Best, E., ed., *Proc. of the 4th Int. Conf. on Concurrency Theory (CONCUR)*, Springer-Verlag, vol. LNCS 715.
- Zeigler, B., 1976: *Theory of Modeling and Simulation*. Wiley Interscience, New York.
- Zeigler, B. P., H. Praehofer, and T. G. Kim, 2000: *Theory of Modeling and Simulation*. Academic Press, 2nd ed.

Index

*charts, [134](#)
[double] data type, [44](#)
{ x=double, y=double } data type, [44](#)
[42](#), [13](#)
abstract semantics, [10](#)
abstract syntax, [6](#), [20](#)
actor, [3](#), [4](#), [13](#)
actor model, [3](#)
actor-oriented classes, [52](#)
actor-oriented model, [3](#)
Add Refinement, [131](#)
AddSubtract actor, [36](#), [78](#)
algorithm, [4](#)
alpha parameter, [49](#)
amorphous heterogeneity, [5](#)
and states, [134](#)
annotation, [49](#)
Annotation parameter, [49](#)
antenna gain, [97](#)
array, [45](#)
ArrayAverage actor, [66](#)
arrayLength parameter, [66](#)
arrayType(int), [44](#)
arrayType(int) data type, [44](#)
arrayType(int, 5) data type, [44](#)
atomic actor, [6](#)
atomic entity, [6](#)
atomos, [6](#)
Ball Model actor, [127](#)
Bernoulli actor, [133](#)
BIP, [134](#)
black box, [9](#)
blue halo, [54](#)
boolean data type, [44](#), [82](#)
BooleanSelect actor, [87](#)
BooleanSwitch actor, [87](#)
capacity parameter, [96](#)
causal, [20](#)
causality loop, [119](#)
channel, [35](#), [82](#)
chattering, [103](#)
class, [52](#)

- CLASSPATH, 60
- classpath, 60
- clock, 74, 95
- code generation, 10
- comment, 49
- Commutator actor, 68
- complete iteration, 94
- complete partial order, 83
- complex* data type, 44
- complex type, 44
- composite actor, 6, 26, 40, 42, 94, 102, 121, 123
- composite entity, 6
- composite structure diagram, 7
- CompositeActor actor, 40
- ComputeHistogram actor, 156
- concrete semantics, 10
- concrete syntax, 6, 109
- concurrency, 5
- concurrere, 5
- connection, 42
- Const actor, 28, 28, 36, 68
- context menu, 26, 40
- Continuous, 15, 20, 118
- continuous dynamics, 4
- continuous time, 14, 134
- ContinuousDirector, 132
- contraction map, 83
- control key, 35, 42
- coolingRate parameter, 105, 112
- count parameter, 112, 113
- CPO, 83
- CPS, 4
- creative process, 3
- curly braces, 45
- current model time, 81
- CurrentTime actor, 75, 76, 144, 147
- custom icon, 49
- cyber-physical systems, 4
- DAE, 22
- data types, 36
- dataflow, 19, 76, 95
- DDF, 19, 94
- DE, 19, 72, 124
- DE with SDF submodel, 94, 99
- DE within SDF, 95
- debugging, 36
- default parameter, 114
- default transition, 114, 115, 117, 121
- defaultTransition parameter, 130
- delay parameter, 93
- DelayChannel actor, 95
- denotational, 83
- determinate, 19, 73, 87
- determinism, 19, 78
- DEVS, 83
- differential equation, 4
- differential-algebraic equation, 22
- direction of a port, 42
- director, 7, 10, 28
- discard parameter, 82
- discrete event, 19, 72
- Discrete Event System Specification, 83
- DiscreteClock actor, 72, 74, 86, 95, 99, 141, 143, 144, 148, 150, 152
- Display actor, 28, 28, 36, 113
- Display name parameter, 40
- Display Stack Trace button, 37
- diversity, 54
- diversity parameter, 66
- domain, 18, 21
- domain polymorphic, 14
- dotted name, 37
- double* data type, 36, 40, 44, 45, 75

- double type, 44
- dynamic dataflow, 19
- edit parameters dialog, 46
- EmbeddedCA actor, 102
- Esterel, 134
- event, 72
- event queue, 20
- EventFilter actor, 82
- exception, 36
- exponential random variable, 74
- Expression actor, 25, 26, 91
- expression language, 9, 26, 36
- extended FSM, 104, 112, 113
- extended state machine, 104
- fault, 72
- feedback, 82, 86
- FIFO queue, 15
- FileWriter actor, 84
- final state, 114, 137, 138
- final states, 104, 113
- finite state machine, 5, 103
 - hierarchical, 133
- finite-state machine, 20
- fire, 13, 15
- fireAtStart parameter, 144, 148
- fixed point, 15, 83
- fixed-point iteration, 107, 118
- fixed-point semantics, 14
- fixedpoint* data type, 44
- fixedpoint type, 44
- float* data type, 44
- float type, 44
- ForSyDe, 13
- free.initialPosition parameter, 130
- free.initialVelocity parameter, 130
- FSM, 20, 73, 103
 - hierarchical, 133
- FSMA actor, 103, 109, 111, 113, 117, 121, 136, 140, 141, 148, 150, 153
- FSMDirector actor, 136
- Gaussian actor, 42, 46
- Gaussian noise, 122
- general* data type, 44
- general type, 44
- Giotto, 22
- global time, 150
- guard, 20, 124
- guard expression, 105, 129
- guard1 parameter, 136
- guard2 parameter, 136
- guardExpression parameter, 129
- GUI, 24
- hardware description language, 83
- Harel, David, 134
- heatingRate parameter, 105
- heatOffThreshold parameter, 105, 108
- hierarchical FSM, 126, 127, 133
- hierarchical heterogeneity, 9, 14
- hierarchical models, 40
- hierarchical multimodeling, 5
- higher-order actor, 91
- higher-order components, 64
- HigherOrderActors library, 68
- histogram, 77, 156, 159
- hybrid system, 13, 21, 134
- hysteresis, 108, 153
- icon editor, 49
- IID, 74
- IllegalActionException, 95
- independent and identically distributed, 74
- index, 17

- inheritance, [52](#), [57](#)
- Inhibit actor, [82](#)
- init parameter, [40](#), [102](#)
- initial state, [103](#)
- initialize, [10](#)
- initialPosition parameter, [130](#)
- initialValue parameter, [82](#)
- instance, [54](#)
- instance parameter, [64](#)
- instances, [52](#)
- int* data type, [36](#), [40](#), [44](#), [74](#), [89](#)
- int type, [44](#)
- InteractiveShell actor, [91](#)
- interarrival times, [77](#)
- interference, [97](#)
- interferenceAmplitude parameter, [59](#)
- interferenceFrequency parameter, [59](#)
- internal block diagram, [7](#)
- isFinalState parameter, [113](#)
- isPresent, [108](#)
- iterate, [10](#)
- IterateOverArray actor, [66](#), [66](#), [67](#)
- iteration, [13](#)
- iteration in DE, [87](#)
- iterations parameter, [31](#), [36](#), [44](#)
- LabVIEW, [22](#)
- level, [83](#)
- lexicographic order, [18](#)
- local time, [144](#), [145](#), [150](#)
- long* data type, [40](#), [44](#)
- long type, [44](#)
- loose actor semantics, [14](#)
- loosest actor semantics, [14](#)
- Macintosh, [34](#), [40](#), [42](#), [131](#)
- mailbox, [15](#)
- MARTE, [7](#)
- MathWorks, [22](#)
- matrix, [45](#)
- matrix* data type, [44](#)
- matrix type, [44](#)
- Mealy machine, [104](#), [112](#)
- meanTime parameter, [73](#)
- memoryless property, [145](#)
- Merge actor, [82](#), [87](#)
- meta model, [6](#)
- metric space, [83](#)
- Metropolis, [22](#)
- microstep, [17](#), [74](#), [76](#), [80–82](#), [144](#)
- mobile transmitters and receivers, [97](#)
- MobileFunction actor, [67](#)
- MobileModel actor, [67](#)
- MoC, [5](#), [7](#), [19](#), [73](#)
- modal model, [17](#), [20](#), [104](#), [121](#)
- ModalModel actor, [64](#), [73](#), [111](#), [121](#), [127](#), [136](#), [137](#), [140–142](#), [144](#), [150](#)
- mode, [123](#), [150](#)
- mode refinement, [123](#)
- model of computation, [5](#), [72](#), [141](#)
- model time, [15](#), [17](#), [74–76](#), [76](#), [80](#), [156](#)
- Modelica, [22](#)
- ModelReference actor, [68](#)
- ModHel’X, [13](#)
- MoML, [10](#), [109](#)
- monotonic functions, [83](#)
- Moore machine, [104](#), [112](#)
- multi-core machine, [19](#)
- multi-paradigm modeling, [13](#)
- multi-view modeling, [5](#)
- multicore, [90](#), [94](#)
- MultiInstanceComposite actor, [64](#), [64](#), [66](#)
- MultiplyDivide actor, [36](#)
- multiport, [35](#), [39](#), [82](#), [111](#)
- multiport type constraint, [40](#)

- name, 37
- National Instruments, 22
- natural number, 18
- network, 95
- Newton's cradle, 18
- nInstances parameter, 64, 66
- noise, 97
- noisePower parameter, 46, 55, 66
- nondeterminate, 19, 75, 85, 93
- nondeterminism, 19, 21
- nondeterministic, 115
- nondeterministic FSM, 133
- Nondeterministic FSM error, 116
- nondeterministic parameter, 116, 130
- nondeterministic transition, 116, 119, 120
- NonStrictDelay actor, 119
- normal termination, 138
- normalThreshold parameter, 154
- ns-2, 83
- ns-3, 83
- numberOfTimes parameter, 66
- object* data type, 44
- Object Management Group, 7
- object type, 44
- object-oriented class, 6
- object-oriented software architecture, 2
- Occam's razor, 2
- occlusion, 97
- ODE, 20, 134
- OMG, 7
- opaque, 9, 14, 64, 102, 123
- operational, 83
- OPNET Modeler, 83
- ordinary differential equation, 20
- ordinary differential equations, 134
- oscilloscope, 157
- output action, 106, 132
- output actions, 111, 129, 132
- output actions parameter, 132, 133
- outputActions parameter, 111, 129
- outsideChannel parameter, 96
- pan, 49
- pan window, 49
- parameter, 46
- period parameter, 74, 95, 99
- persistentTrigger parameter, 96
- physical process, 4
- pink halo, 55, 57
- PN, 19, 94
- Poisson process, 74, 121
- PoissonClock actor, 72, 73, 74, 76, 97, 121, 141, 144, 145, 148
- polymorphic actor, 36
- port buttons in the toolbar, 41
- port direction, 42
- port types, 44
- port visibility, 42
- postfire, 13, 15, 86
- power loss, 97
- predicate, 124
- preemptive parameter, 130, 132, 133
- preemptive transition, 133, 136, 143
- prefire, 13
- preinitialize, 10
- Previous actor, 77, 80
- Priority parameter, 85
- process network, 19
- propagationSpeed parameter, 97
- Ptolemy Classic, 13
- Ptolemy II model, 6
- Ptolemy Project, 4
- ptolemy.ptII.dir, 60
- Python, 102
- PythonScript actor, 102

- quantity manager, 22
- Queue actor, 96
- queueing, 96

- Ramp actor, 33, 86, 102, 137
- random times, 74
- reactive, 141, 147
- real number, 18
- real time, 74, 75, 76, 80
- receiver, 14
- record, 45
- recordedInput parameter, 148
- refinement, 20
 - mode, 123
 - multiple, 125
 - shared, 125
- refinementName parameter, 125, 126, 130
- Register actor, 82, 86
- relation, 9, 33
- relation group, 35
- Rendezvous, 21
- rendezvous, 15, 134
- Repeat actor, 66, 66
- reset parameter, 130, 132
- reset transition, 137, 138, 145
- ResettableTimer actor, 74
- right, 40
- run button, 25
- run window, 25
- RunCompositeActor actor, 68

- SampleAndHold actor, 82
- Sampler actor, 79, 82
- SCADE, 22
- scalar* data type, 44
- scalar type, 44
- scheduling, 10
- SDF, 19, 94, 114, 123
 - SDF with DE submodel, 95
 - SDF within DE, 94, 99
 - SDFDirector, 28, 31, 33, 67
 - semantics, 5, 28, 83
 - SequencePlotter actor, 33, 42, 155, 159
 - SequenceScope actor, 155
 - SequenceToArray actor, 66
 - Server actor, 96, 97
 - serviceTime parameter, 96
 - set action, 106
 - set actions, 113, 130, 132
 - set actions parameter, 132, 133
 - setActions parameter, 130
 - setup, 10
 - SharedQueue actor, 96
 - short* data type, 44
 - short type, 44
 - show name parameter, 49
 - showClones parameter, 64
 - signal, 3, 17, 19, 76
 - signal plotter, 155
 - Simulink, 22
 - simultaneity, 17
 - simultaneous, 17, 77, 82
 - Sinewave actor, 42
 - single port, 35
 - SingleEvent actor, 74, 90
 - Spectrum actor, 159
 - Spectrum model, 24
 - speculative execution, 15, 21
 - spontaneous FSM, 141
 - spontaneous modal model, 141, 142
 - sporadic source, 97
 - square braces, 45
 - SR, 118, 134, 137
 - standard deviation, 46
 - standard out, 84

- standardDeviation parameter, 46
- starCharts, 134
- state, 14, 20, 102
- state machine, 103, 121
- state variable, 102
- Statecharts, 5, 134
- StateFlow, 22
- Statemate, 22, 134
- step parameter, 40, 102
- stochastic state machine, 133
- stopWhenQueueIsEmpty parameter, 94
- strict actor semantics, 14, 21
- string, 44
- string* data type, 40, 44
- strongly simultaneous events, 17, 77
- stuttering Zeno, 89
- subclass, 57
- subclasses, 52
- superdense time, 17, 76, 83
- SynchCharts, 134
- Synchronizer actor, 80, 82
- synchronizeToRealTime parameter, 76, 94
- synchronous, 5
- synchronous dataflow, 19
- synchronous language, 134
- synchronous reactive, 14
- synchronousreactive, 83
- SysML, 7
- SystemC, 13, 83
- tag, 18, 144
- tagged-signal model, 18
- TDL, 22
- termination, normal, 138
- thermostat, 103
- ThreadedComposite actor, 90, 91
- time, 15
- time index, 144
- time resolution, 17
- time stamp, 17, 19, 75, 76, 80, 141, 144
- TimeCompare actor, 75
- TimeDelay actor, 80, 86, 87, 146
- TimedPlotter actor, 73, 144, 156
- TimeGap actor, 75, 80
- token, 36
- toolbar, 41
- top level, 6
- topological sort, 79, 83
- transient state, 148
- transition, 104, 124
- transition refinement, 131, 138, 139, 147
- transitions, 129
- transparency, 49
- transparent, 9
- type, 36
- type checking, 10
- type error, 37
- type inference, 10, 42
- type of a port, 42
- type system, 36
- UML, 2, 134
- UML 2, 7
- UML class diagram, 2, 6, 8
- under the table, 84
- unified modeling language, 2, 134
- unknown, 44
- unknown* data type, 44
- unsignedByte, 44
- unsignedByte* data type, 44
- useLocalTime parameter, 144
- UserLibrary, 62
- value parameter, 31, 36
- VariableDelay actor, 80, 86, 90
- variance, 46

Vergil, 6, **24**, 111, 136
 Verilog, 83
 VHDL, 83
 VINT, 83
 Virtual Internetwork Testbed Project, 83
 visual syntax, **6**, 7, 95
 VisualModelReference actor, **68**

 WaitingTime actor, **75**
 wall-clock time, **76**
 WallClockTime actor, **75**, 76
 warningThreshold parameter, 154
 weakly simultaneous events, 17, 77
 welcome window, 28
 white box, 10
 white halo, 67
 William of Ockham, 2
 wireless channel, 96
 wireless domain, **95**
 WirelessComposite actor, **95**
 WirelessDirector, 95
 wrapup, **10**

 XML schema, 10
 xmlToken, 44
xmlToken data type, 44
 XYPlotter actor, **156**

 Y-chart, 22

 Zeno, 88, **89**
 zero delay, 80
 zero-delay actor, 77
 zooming, 49